AtomVM Documentation

Release 0.6.0-dev+git.29d5d7a

© 2017-2023, AtomVM

May 05, 2023

Table of Contents

1 Welcome to AtomVM! 3
1.1 What is AtomVM? . . . 3
1.2 Why Erlang/Elixir? . . . 3
1.3 Design Philosophy . . . 4
1.4 Licensing . . . 4
1.5 Source Code. . . . 4
1.6 Contributing . . . 5
1.7 Where to go from here. . . . 5

2 Getting Started Guide 7
2.1 Getting Started on the ESP32 platform . . . 7
2.2 Getting Started on the STM32 platform . . . 11
2.3 Getting Started on the Generic UNIX platform . . . 11
2.4 Where to go from here . . . 11

3 Programmers Guide 13
3.1 AtomVM Features . . . 13
3.2 AtomVM Development . . . 14
3.3 Applications. . . . 16
3.4 Core APIs . . . 18
3.5 ESP32-specific APIs. . . . 23
3.6 Peripherals. . . . 25
3.7 Protocols . . . 32

4 Example Programs 39
4.1 Erlang Examples . . . 39
4.2 ESP32 Examples. . . . 42
4.3 Flashing AtomVM Examples for ESP32 . . . 42

5 Network Programming Guide 49
5.1 Station (STA) mode . . . 49
5.2 AP mode . . . 51
5.3 STA+AP mode. . . . 53
5.4 SNTP Support. . . . 53
5.5 NVS Credentials . . . 53
5.6 Stopping the Network . . . 54

6 Build Instructions 55
6.1 Downloading AtomVM . . . 55
6.2 Source code organization . . . 55
6.3 Building for Generic UNIX . . . 56

6.4 Building for ESP32 . . . 57
6.5 Building for STM32 . . . 63

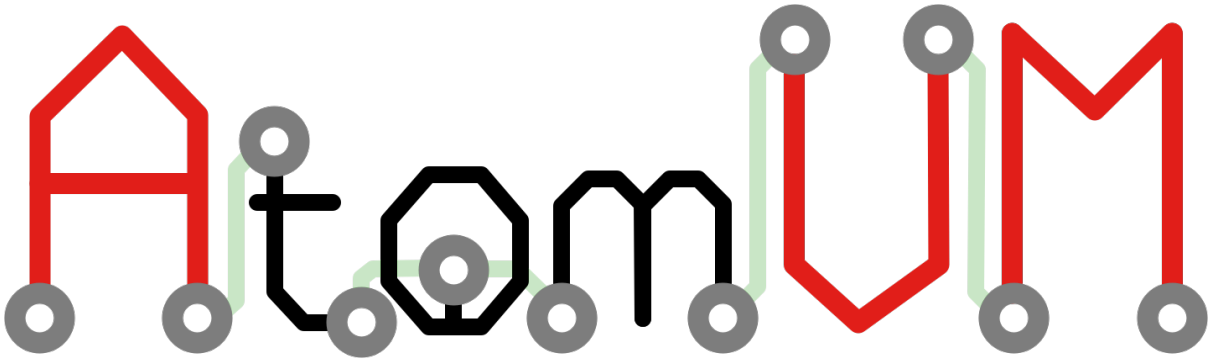
7 AtomVM Internals **65**
7.1 What is an Abstract Machine? . . . 65
7.2 AtomVM Data Structures . . . 66
7.3 Runtime Execution Loop . . . 67
7.4 Module Loading . . . 67
7.5 Function Calls and Return Values . . . 67
7.6 Exception Handling . . . 67
7.7 The Scheduler . . . 67
7.8 Stacktraces . . . 67

8 Memory Management **69**
8.1 The Context structure . . . 69
8.2 Simple Terms . . . 72
8.3 Boxed terms . . . 73
8.4 Lists . . . 79
8.5 Special Stack Types . . . 80
8.6 Garbage Collection . . . 81

9 Packbeam Format **87**
9.1 Overview . . . 87
9.2 Packbeam Header . . . 87
9.3 File encodings . . . 88

10 API Reference Documentation **91**
10.1 Erlang Libraries . . . 91
10.2 AtomVM 'C' Internal Libraries . . . 136

11 Contributing Guidelines **137**
11.1 Git Recommended Practices . . . 137
11.2 Coding Style . . . 137



Welcome to AtomVM, the Erlang virtual machine for IoT devices!

AtomVM is a lightweight implementation of the the Bogdan Erlang Abstract Machine (_aka_, the BEAM), a virtual machine that can execute byte-code instructions compiled from Erlang or Elixir source code. AtomVM supports a limited but functional subset of the BEAM opcodes, and also includes a small subset of the Erlang/OTP standard libraries, all optimized to run on tiny micro-controllers. With AtomVM, you can write your IoT applications in a functional programming language, using a modern actor-based concurrency model, making them vastly easier to write and understand!

AtomVM includes many advanced features, including process spawning, monitoring, message passing, pre-emptive scheduling, and efficient garbage collection. It can also interface directly with peripherals and protocols supported on micro-controllers, such as GPIO, I2C, SPI, and UART. It also supports WiFi networking on devices that support it, such as the Espressif ESP32. All of this on a device that can cost as little as $2!

Warning AtomVM is currently in Alpha status. Software may contain bugs and should not be used for mission-critical applications. Application Programming Interfaces may change without warning.

Chapter 1

Welcome to AtomVM!

Welcome to AtomVM, the Erlang virtual machine for IoT devices!

1.1 What is AtomVM?

AtomVM is a ground-up implementation of the Bogdan Erlang Abstract Machine (a.k.a the BEAM) and is designed specifically to run on small systems, such as the Espressif ESP32 and ST Microelectronics STM32 micro-controllers. It allows developers to implement IoT applications in the Erlang or Elixir programming languages and to deploy those applications onto tiny devices. (Users may also target their applications for fully-fledged operating systems, such as Linux, FreeBSD, and MacOS, though in most cases deployment to traditional computers is done for development and testing purposes, only.)

AtomVM features include:

- An Erlang runtime, capable of executing bytecode instructions in compiled BEAM files;
- Support for all the major Erlang and Elixir types, including integers, strings, lists, maps, binaries, Enums, and more;
- A memory-managed environment, with efficient garbage collection and shared data, where permissible;
- Support for truly functional programming languages, making your programs easier to understand and debug;
- A concurrency-oriented platform, allowing users to spawn, monitor, and communicate with lightweight processes, making it easy for your IoT devices to perform tasks simultaneously;
- A rich set of networking APIs, for writing robust IoT applications that communicate over IP networks;
- A rich set of APIs for interfacing with standard device protocols, such as GPIO, I2C, SPI, and UART;
- And more!

1.2 Why Erlang/Elixir?

The environments on which AtomVM applications are deployed are significantly more constrained than typical programming environments. For example, the typical ESP32 ships with 520K of RAM and 4MB of flash storage, roughly the specs of a mind 1980s desktop computer. Moreover, most micro-controller environments do not support native POSIX APIs for interfacing with an operating system, and in many cases, common operating system abstractions, such as processes, threads, or files, are simply unavailable.

However, because the BEAM is provides a pre-emptive multitasking environment for your applications, many of the common operating system abstractions, particularly involving threading and concurrency, are simply not needed. As concurrently-oriented languages, Erlang and Elixir support lightweight "processes", with message passing as the mechanism for inter-(erlang)process communication, pre-emptive multi-tasking, and per-process heap allocation and garbage collection.

In many ways, the programming model for Erlang and Elixir is closer to that of an operating system and multiple concurrent processes running on it, where operating system processes are single execution units, communicate through message passing (signals), and don't share any state with one another. Contrast that with most popular programming languages today (C, C++, Java, Python, etc), which use threading abstractions to achieve concurrency within a single memory space, and which subsequently require close attention to cases in which multiple CPUs operate on a shared region of memory, requiring threads, locks, semaphores, and so forth.

As an implementation of the BEAM, AtomVM provides a modern, memory managed, and concurrency-oriented environment for developing applications on small devices. This makes writing concurrent code for micro-controllers (e.g., and application that reads sensor data, services HTTP requests, and updates the system clock, all at the same time) incredibly simple and natural – far easier than writing programs that use concurrency than C, C++, or even, for example, Micropython.

In addition, because it is targeted for micro-controller environments, AtomVM provides interfaces for integrating with features commonly seen on micro-controllers, such as GPIO pins, analog-to-digital conversion, and common industry peripheral interfaces, such as I2C, SPI, and UART, making AtomVM a rich platform for developing IoT applications.

Finally, one of the exciting aspects about modern micro-controllers, such as the ESP32, is their integration with modern networking technologies, such as WiFi and Bluetooth. AtomVM leverages Erlang and Elixir's natural affinity with telecommunications technologies to open up further possibilities for developing networked and wireless IoT devices.

We think you will agree that AtomVM provides a compelling environment not only for Erlang and Elixir development, but also as a home for interesting and fun IoT projects.

1.3 Design Philosophy

AtomVM is designed from the start to run on small, cheap embedded devices, where system resources (memory, cpu, storage) are tightly constrained. The smallest environment in which AtomVM runs has around 512k of addressable RAM, some of which is used by the underlying runtime (FreeRTOS), and some of which is used by the AtomVM virtual machine, itself, leaving even less RAM for your own applications. Where there is a tradeoff between memory consumption and performance, minimizing memory consumption (and heap fragmentation) always wins.

From the developer's point of view, AtomVM is designed to make use of the existing tool chain from the Erlang and Elixir ecosystems. This includes the Erlang and Elixir compilers, which will compile Erlang and Elixir source code to BEAM bytecode. Where possible, AtomVM makes use of existing tool chains to reduce the amount of unnecessary features in AtomVM, thus reducing complexity, as well as the amount of system resources in use by the runtime. AtomVM is designed to be as small and lean as possible, providing as many resources to user applications, as possible.

1.4 Licensing

AtomVM is licensed under the terms of the Apache2 and LGPLv2 licenses.

1.5 Source Code

The AtomVM Github Repository contains the AtomVM source code, including the AtomVM virtual

machine and core libraries. The AtomVM Build Instructions contains instructions for building AtomVM for Generic UNIX, ESP32, and STM32 platforms.

1.6 Contributing

The AtomVM community welcomes contributions to the AtomVM code base and upstream and downstream projects. Please see the contributing guidelines for information about how to contribute.

AtomVM developers can be reached on the #AtomVM discord server or on Telegram at AtomVM - Erlang and Elixir on Microcontrollers.

1.7 Where to go from here

The following guides provide more detailed information about getting started with the AtomVM virtual machine, how to develop and deploy applications, and implementation information, for anyone interested in getting more involved:

- Getting Started Guide
- Programmers Guide
- Example Programs
- Build Instructions

Chapter 2

Getting Started Guide

The getting started is broken up into the following sections:

- Getting Started on the ESP32 platform
- Getting Started on the STM32 platform
- Getting Started on the Generic UNIX platform

2.1 Getting Started on the ESP32 platform

The AtomVM virtual machine is supported on the Espressif ESP32 platform, allowing users to write Erlang and Elixir programs and deploy them to the ESP32 micro-controller.

These instructions cover how to get the AtomVM virtual machine flashed to your ESP32 device, as well as how to flash your Erlang and Elixir programs that will be executed by the virtual machine running on the device.

For most applications, you should only need to install the VM once (or at least once per desired AtomVM release). Once the VM is uploaded, you can then begin development of Erlang or Elixir applications, which can then be flashed as part of your routine development cycle.

2.1.1 Requirements

Deployment of AtomVM applications requires the following components:

- A computer running MacOS or Linux (Windows support is not currently supported);
- An ESP32 module with a USB/UART connector (typically part of an ESP32 development board);
- A USB cable capable of connecting the ESP32 module or board to your development machine (laptop or PC);
- The `esptool` program, for flashing the AtomVM image and AtomVM programs;
- An Erlang/OTP release (21, 22, or 23);
- A serial console program, such as `minicom` or `screen`, so that you can view console output from your AtomVM application.
- (optional) For Erlang programs, `rebar3`;
- (optional) For Elixir programs, `mix`, which ships with the Elixir runtime;

2.1.2 Deployment Overview

The ES32 AtomVM virtual machine is an IDF application that runs on the ESP32 platform. As an IDF application, it provides the object code to boot the ESP device and execute the AtomVM virtual machine code, which in turn is responsible for execution of an Erlang/Elixir application.

The AtomVM virtual machine is implemented in C, and the AtomVM binary image contains the binary object code compiled from C source files, as well as the ESP boot loader and partition map, which tells the ESP32 how the flash module is laid out.

AtomVM developers will typically write their applications in Erlang or Elixir. These source files are compiled into BEAM bytecode, which is then assembled into AtomVM "packbeam" (`.avm`) files. This packbeam file is flashed onto the ESP32 device, starting at the data partition address `0x210000`. When AtomVM starts, it will look in this partition for the first occurrence of a BEAM module that exports a `start/0` function. Once that module is located, execution of the BEAM bytecode will commence at that point.

The following diagram provides a simplified overview of the layout of the AtomVM virtual machine and Erlang/Elixir applications on the ESP32 flash module.

```
|               |
+---------------+  ----------- 0x1000
| boot loader   |           ^
+---------------+           |
| partition map |           | AtomVM
+---------------+           | binary
|               |           | image
|   AtomVM      |           |
|   Virtual     |           |
|   Machine     |           |
|               |           v
+---------------+  ----------- 0x210000
|               |           ^
|               |           |
|     data      |           | Erlang/Elixir
|   partition   |           | Application
|               |           |
|               |           v
+---------------+  ----------- end
```

Deploying an AtomVM application to an ESP32 device typically involved two steps:

1. Connecting the ESP32 device;
2. Deploying the AtomVM virtual machine;
3. Deploying an AtomVM application (typically an iterative process)

These steps are described in more detail below.

2.1.3 Connecting the ESP32 device

Connect the ESP32 to your development machine (e.g., laptop or PC) via a USB cable.

```
+---------------+
| laptop or PC  |
|               |           +-------+
|               | USB       |       |
|               x-----------x       |
|               |           |       |
|               |           +-------+
+---------------+           ESP32
```

> Note. There are a wide variety of ESP32 modules, ranging from home-made breadboard solutions to all-in-one development boards. For simplicity, we assume a development board that can both be powered by a USB cable and which can be simultaneously flashed using the same cable, e.g., the Espressif ESP32 DevKit.

Consult your local development board documentation for instructions about how to connect your device to your development machine.

2.1.4 Deploying the AtomVM virtual machine

The following methods can be used to deploy the AtomVM virtual machine to an ESP32 device:

1. Flashing a binary image;
2. Building from source.

Flashing a binary images

Flashing the ESP32 using a pre-built binary image is by far the easiest path to getting started with development on the ESP32. Binary images contain the virtual machine image and all of the necessary components to run your application.

We recommend first erasing any existing applications on the ESP32 device. E.g.,

```
shell$ esptool.py --chip esp32 --port /dev/ttyUSB0 --baud 115200 erase_flash
...
```

> Note. Specify the device port and baud settings and AtomVM image name to suit your particular environment.

Next, download the latest stable or nightly ESP32 release image from the AtomVM web site.

> Note. Nightly images may be unstable and may result in unpredictable behavior.

Finally, use the `esptool` to flash the image to the start address `0x1000` on the ESP32. E.g.,

```
shell$ esptool.py --chip esp32 --port /dev/ttyUSB0 --baud 115200 \
    --before default_reset --after hard_reset \
    write_flash -u --flash_mode dio --flash_freq 40m --flash_size detect \
    0x1000 atomvm-esp32-v0.1.0.bin
...
```

Once completed, your ESP32 device is ready to run Erlang or Elixir programs targeted for AtomVM.

Building from source

You may optionally build AtomVM from source and deploy the AtomVM virtual machine to your ESP32 device manually. Building AtomVM from source is slightly more involved, as it requires the installation of the Espressif IDF SDK and tool chain and is typically recommended only for users who are doing development on the AtomVM virtual machine, or for developers implementing custom Nifs or ports.

Instructions for building AtomVM from source are covered in the AtomVM Build Instructions

2.1.5 Deploying an AtomVM application

An AtomVM application is a collection of BEAM files, which have been compiled using the Erlang or Elixir compiler. These BEAM files are assembled into an AtomVM "packbeam" (`.avm`) file, which in turn is flashed to the `main` data partition on the ESP32 flash module, starting at address `0x210000`.

When the AtomVM virtual machine starts, it will search for the first module that contains an exported `start/0` function in this partition, and it will begin execution of the BEAM bytecode at that function.

AtomVM applications can be written in Erlang or Elixir, or a combination of both. The AtomVM community has provided tooling for both platforms, making deployment of AtomVM applications as seamless as possible.

This section describes both Erlang and Elixir tooling for deploying AtomVM applications to ESP32 devices.

Erlang Tooling

Deployment of AtomVM applications written in the Erlang programming language is supported via the `atomvm_rebar3_plugin` plugin, a community-supported plugin to the `rebar3` Erlang build

tool.

You can generate a simple application from scratch using the `atomvm_rebar3_plugin` template, as follows:

Edit or create the `$HOME/.config/rebar3/rebar.config` file to include the `atomvm_rebar3_plugin` plugin:

```
%% $HOME/.config/rebar3/rebar.config
{plugins, [
    {atomvm_rebar3_plugin, "0.3.0"},
    ...
]}.
```

In any directory in which you have write permission, issue

```
shell$ rebar3 new atomvm_app <app-name>
```

where `<app-name>` is the name of the application you would like to create (e.g., `myapp`). This command will generate a rebar project under the directory `<app-name>`.

The generated application will contain the proper `rebar.config` configuration and will contain the `<app-name>.erl` module, which exports the `start/0` function with a stubbed implementation.

Specifically, note the following stanza in the generated `rebar.config` file:

```
%% rebar.config
{plugins, [
    {atomvm_rebar3_plugin, "0.3.0"},
    ...
]}.
```

And note the `myapp` application exports a `start/0` function, e.g.,

```
%% erlang
-module(myapp).
-export([start/0]).

start() ->
    ok.
```

With this plugin installed, you have access to the `esp32_flash` target, which will build an AtomVM packbeam

```
shell$ rebar3 esp32_flash --port /dev/ttyUSB0
===> Fetching atomvm_rebar3_plugin v0.3.0
===> Fetching rebar3_hex v6.11.3
===> Fetching hex_core v0.7.1
===> Fetching verl v1.0.2
===> Analyzing applications...
===> Compiling verl
===> Compiling hex_core
===> Compiling rebar3_hex
===> Fetching atomvm_packbeam v0.3.0
===> Analyzing applications...
===> Compiling atomvm_rebar3_plugin
===> Compiling packbeam
===> Verifying dependencies...
===> Analyzing applications...
===> Compiling myapp
===> AVM file written to : myapp.avm
===> esptool.py --chip esp32 --port /dev/ttyUSB0 --baud 115200 --before
default_reset --after hard_reset write_flash -u --flash_mode dio --flash_freq 40m
--flash_size detect 0x210000 /home/frege/myapp/_build/default/lib/myapp.avm
```

Note. Consult the the `atomvm_rebar3_plugin` plugin documentation, for more detailed

information about how to use this tool.

Once the application has been flashed, you may connect to the ESP32 over the serial port using `minicom`, `screen`, or equivalent.

Elixir Tooling

TODO mix + https://github.com/atomvm/ExAtomVM + hex

2.2 Getting Started on the STM32 platform

TODO will se distribute a binary image for STM32?

2.3 Getting Started on the Generic UNIX platform

AtomVM may be run on UNIX-like platforms using the `AtomVM` executable.

You may specify one or more AVM files on the command line when running the `AtomVM` command. BEAM modules defined in earlier AVM modules on the command line take higher precedence that BEAM modules included in AVM files later in the argument list.

```
shell$ AtomVM /path/to/myapp.avm
```

Currently, the `AtomVM` executable must be built from source.

See the AtomVM Build Instructions for instructions about how to build AtomVM on the Generic UNIX platform.

2.4 Where to go from here

The following resources may be useful for understanding how to develop Erlang or Elixir applications for the AtomVM platform:

- Example Programs
- Programmers Guide

Chapter 3

Programmers Guide

This guide is intended for programmers who develop applications targeted for AtomVM.

As an implementation of the Erlang virtual machine, AtomVM is designed to execute unmodified byte-code instructions compiled into BEAM files, either by the Erlang or Elixir compilers. This allow developers to write programs in their BEAM programming language of choice, and to use the common Erlang community tool-chains specific to their language platform, and to then deploy those applications onto the various devices that AtomVM supports.

This document describes the development workflow when writing AtomVM applications, as well as a high-level overview of the various APIs that are supported by AtomVM. With an understanding of this guide, you should be able to design, implement, and deploy applications onto a device running the AtomVM virtual machine.

3.1 AtomVM Features

Currently, AtomVM implements a strict subset of the BEAM instruction set, as of Erlang/OTP R21. Previous versions of Erlang/OTP are not supported.

A high level overview of the supported language features include:

- All the major Erlang types, including
  - integers (with size limits)
  - floats
  - tuples
  - lists
  - binaries
  - maps
- support for many Erlang BIFs and guard expressions to support the above types
- pattern matching (case statements, function clause heads, etc)
- `try ... catch ... finally` constructs
- anonymous functions
- process `spawn` and `spawn_link`
- send (`!`) and `receive` messages
- bit syntax (with some restrictions)
- reference counted binaries

In addition, several features are supported specifically for integration with micro-controllers, includ-

ing:including:

- Wifi networking (`network`)
- UDP and TCP/IP support (`inet`, `gen_tcp` and `gen_udp`)
- Peripheral and system support on micro-controllers, including
  - GPIO, including pins reads, writes, and interrupts
  - I2C interface
  - SPI interface
  - UART interface
  - LEDC (PWM)
  - non-volatile storage (NVS)
  - deep sleep

3.1.1 Limitations

While the list of supported features is long and growing, the currently unsupported Erlang/OTP and BEAM features include (but are not limited to):

- Bingnums. Integer values are restricted to 64-bit values.
- Bit Syntax. While packing and unpacking of arbitrary (but less than 64-) bit values is support, packing and unpacking of integer values at the start of end of a binary, or bordering binary packing or extraction must align on 8-bit boundaries. Arbitrary-bitlength binaries are not currently supported.
- SMP support. The AtomVM VM is currently a single-threaded process.
- The `epmd` and the `disterl` protocols are not supported.
- There is no support for code hot swapping.
- There is no support for a Read-Eval-Print-Loop. (REPL)
- Numerous modules and functions from Erlang/OTP standard libraries (`kernel`, `stdlib`, `sasl`, etc) are not implemented.

AtomVM bit syntax is restricted to alignment on 8-bit boundaries. Little-endian and signed insertion and extraction of integer values is restricted to 8, 16, and 32-bit values. Only unsigned big and little endian 64-bit values can be inserted into or extracted from binaries.

It is highly unlikely that an existing Erlang program targeted for Erlang/OTP will run unmodified on AtomVM. And indeed, even as AtomVM matures and additional features are added, it is more likely than not that Erlang applications will need to targeted specifically for the AtomVM platform. The intended target environment (small, cheap micro-controllers) differs enough from desktop or server-class systems in both scale and APIs that special care and attention is needed to target applications for such embedded environments.

That being said, many of the features of the BEAM are supported and provide a rich and compelling development environment for embedded devices, which Erlang and Elixir developers will find natural and productive.

3.2 AtomVM Development

This section describes the typical development environment and workflow most AtomVM developers are most likely to use.

3.2.1 Development Environment

In general, for most development purposes, you should be able to get away with an Erlang/OTP development environment (OTP21 or later), and for Elixir developers, and Elixir version TODO development environment. We assume most development will take place on some UNIX-like environment (e.g., Linux, FreeBSD, or MacOS). Consult your local package manager for installation of these development environments.

Developers will want to make use of common Erlang or Elixir development tools, such as `rebar3` for Erlang developers or `mix` for Elixir developers.

Developers will need to make use of some AtomVM tooling. Fortunately, there are several choices for developers to use:

1. AtomVM `PackBEAM` executable (described below)
2. `atomvm_rebar3_plugin`, for Erlang development using `rebar3`.
3. `ExAtomVM` Mix plugin, Elixir development using `Mix`.

Some testing can be performed on UNIX-like systems, using the `AtomVM` executable that is suitable for your development environment. AtomVM applications that do not make use of platform-specific APIs are suitable for such tests.

Deployment and testing on micro-controllers is slightly more involved, as these platforms require additional hardware and software, described below.

ESP32 Deployment Requirements

In order to deploy AtomVM applications to and test on the ESP32 platform, developers will need:

- A computer running MacOS or Linux (Windows support is TBD);
- An ESP32 module with a USB/UART connector (typically part of an ESP32 development board);
- A USB cable capable of connecting the ESP32 module or board to your development machine (laptop or PC);
- The `esptool` program, for flashing the AtomVM image and AtomVM programs;
- (Optional, but recommended) A serial console program, such as `minicom` or `screen`, so that you can view console output from your AtomVM application.

STM32 Deployment Requirements

TODO

3.2.2 Development Workflow

For the majority of users, AtomVM applications are written in the Erlang or Elixir programming language. These applications are compiled to BEAM (`.beam`) files using standard Erlang or Elixir compiler tool chains (`erlc`, `rebar`, `mix`, etc). The generated BEAM files contain byte-code that can be executed by the Erlang/OTP runtime, or by the AtomVM virtual machine.

> Note. In a small number of cases, it may be useful to write parts of an application in the C programming language, as AtomVM nifs or ports. However, writing AtomVM nifs and ports is outside of the scope of this document.

Once Erlang and/or Elixir files are compiled to BEAM files, AtomVM provides tooling for processing and aggregating BEAM files into AtomVM Packbeam (`.avm`) files, using AtomVM tooling, distributed as part of AtomVM, or as provided through the AtomVM community.

AtomVM packbeam files are the applications and libraries that run on the AtomVM virtual machine. For micro-controller devices, they are "flashed" or uploaded to the device; for command-line use of AtomVM (e.g., on Linux, FreeBSD, or MacOS), they are supplied as the first parameter to the AtomVM command.

The following diagram illustrates the typical development workflow, starting from Erlang or Elixir source code, and resulting in a deployed Packbeam file:

```
*.erl or *.ex                  *.beam
+-------+                   +-------+
|       |+                  |       |+
|       ||+                 |       ||+
|       |||     -------->   |       |||
|       |||  Erlang/Elixir  |       |||
+-------+||     Compiler    +-------+||
 +-------+|                  +-------+|
  +-------+                   +-------+
     ^                           |
     |                           | packbeam
     |                           |
     |                           v
     |                        +-------+
     |                        |       |
     | test                   |       |
     | debug                  |       |
     | fix                    |       |
     |                        +-------+
     |                         app.avm
     |                           |
     |                           | flash/upload
     |                           |
     |                           v
     +-------------------- Micro-controller
                              device
```

The typical compile-test-debug cycle can be summarized in the following steps:

1. Deploy the AtomVM virtual machine to your device
2. Develop an AtomVM application in Erlang or Elixir
    1. Write application
    2. Deploy application to device
    3. Test/Debug/Fix application
    4. Repeat

Deployment of the AtomVM virtual machine and an AtomVM application currently require a USB serial connection. There is currently no support for over-the-air (OTA) updates.

For more information about deploying the AtomVM image and AtomVM applications to your device, see the Getting Started Guide

3.3 Applications

An AtomVM application is a collection of BEAM files, aggregated into an AtomVM "Packbeam" (`.avm`) file, and typically deployed (flashed) to some device. These BEAM files be be compiled from Erlang, Elixir, or any other language that targets the Erlang VM.

> Note. The return value from the `start/0` function is ignored.

Here, for example is one of the smallest AtomVM applications you can write:

```
%% erlang
-module(myapp).

-export([start/0]).
```

```
start() ->
    ok.
```

This particular application doesn't do much, of course. The application will start and immediately terminate, with a return value of `ok`. Typical AtomVM applications will be more complex than this one, and the AVM file that contains the application BEAM files will be considerably larger and more complex than the above program.

Most applications will spawn processes, send and receive messages between processes, and wait for certain conditions to apply before terminating, if they terminate at all. For applications that spawn processes and run forever, you may need to add an empty `receive ... end` block, to prevent the AtomVM from terminating prematurely, e.g.,

```
%% erlang
wait_forever() ->
    receive X -> X end.
```

3.3.1 Packbeam files

AtomVM applications are packaged into Packbeam (`.avm`) files, which contain collections of files, typically BEAM (`.beam`) files that have been generated by the Erlang or Elixir compiler.

At least one BEAM module in this file must contain an exported `start/0` function. The first module in a Packbeam file that contain this function is the entry-point of your application and will be executed when the AtomVM virtual machine starts.

Not all files in a Packbeam need to be BEAM modules – you can embed any type of file in a Packbeam file, for consumption by your AtomVM application.

> Note. The Packbeam format is described in more detail in the AtomVM PackBEAM format.

The AtomVM community has provided several tools for simplifying your experience, as a developer. These tools allow you to use standard Erlang and Elixir tooling (such as `rebar3` and `mix`) to build Packbeam files and deploy then to your device of choice.

3.3.2 `PackBEAM` tool

The `PackBEAM` tool is a command-line application that can be used to create Packbeam files from a collection of input files:

```
shell$ PackBEAM -h
Usage: PackBEAM [-h] [-l] <avm-file> [<options>]
    -h                                                  Print this help menu.
    -i                                                  Include file and line
information.
    -l <input-avm-file>                                 List the contents of an AVM
file.
    [-a] <output-avm-file> <input-beam-or-avm-file>+  Create an AVM file (archive
if -a specified).
```

To create a packbeam file, specify the name of the AVM file to created (by convention, ending in `.avm`), followed by a list of BEAM files:

```
shell$ PackBEAM foo.avm path/to/foo.beam path/to/bar.beam
```

You can also specify another AVM file to include. Thus, for example, to add to BEAM file to an existing AVM file, you might enter:

```
shell$ PackBEAM foo.avm foo.avm path/to/gnu.beam
```

To list the contents of an AVM file, use the `-l` flag:

```
shell% PackBEAM -l foo.avm
foo.beam *
```

```
bar.beam
gnu.beam
```

Any BEAM files that export a `start/0` function will contain an asterisk (`*`) in the AVM file contents.

3.3.3 Running AtomVM

AtomVM is executed in different ways, depending on the platform. On most microcontrollers (e.g., the ESP32), the VM starts when the device is powered on. On UNIX platforms, the VM is started from the command-line using the `AtomVM` executable.

AtomVM will use the first module in the supplied AVM file that exports a `start/0` function as the entrypoint for the application.

`AtomVM` program syntax

On UNIX platforms, you can specify a BEAM file or AVM file as the first argument to the executable, e.g.,

```
shell$ AtomVM foo.avm
```

> Note. If you start the `AtomVM` executable with a BEAM file, then the corresponding module may not make any calls to external function in other modules, with the exception of built-in functions and Nifs that are included in the VM.

3.4 Core APIs

The AtomVM virtual machine provides a set of Erlang built-in functions (BIFs) and native functions (NIFs), as well as a collection of Erlang and Elixir libraries that can be used from your applications.

This section provides an overview of these APIs. For more detailed information about specific APIs, please consult the API reference documentation.

3.4.1 Standard Libraries

AtomVM provides a limited implementations of standard library modules, including:

- `base64`
- `gen_server`
- `gen_statem`
- `io` and `io_lib`
- `lists`
- `maps`
- `proplists`
- `supervisor`
- `timer`

In addition AtomVM provides limited implementations of standard Elixir modules, including:

- `List`
- `Tuple`
- `Enum`
- `Kernel`
- `Module`
- `Process`

- `Console`

For detailed information about these functions, please consult the API reference documentation. These modules provide a strict subset of functionality from their Erlang/OTP counterparts. However, they aim to be API-compatible with the Erlang/OTP interfaces, at least for the subset of provided functionality.

3.4.2 Console Output

There are several mechanisms for writing data to the console.

For common debugging, many users will find `erlang:display/1` sufficient for debugging:

```
%% erlang
erlang:display({foo, [{bar, tapas}]}).
```

The output parameter is any Erlang term, and a newline will be appended automatically.

Users may prefer using the `io:format/1,2` functions for more controlled output:

```
%% erlang
io:format("The ~p did a ~p~n", [friddle, frop]).
```

Note that the `io_lib` module can be used to format string data, as well.

> Note. Formatting parameters are currently limited to `~p`, `~s`, and `~n`.

3.4.3 Process Management

You can obtain a list of all processes in the system via `erlang:processes/0`:

```
%% erlang
Pids = erlang:processes().
```

And for each process, you can get detailed process information via the `erlang:process_info/1` function:

```
%% erlang
[io:format("Process info for Pid ~p: ~p~n", [Pid, erlang:process_info(Pid)]) ||
Pid <- Pids].
```

The return value is a property list containing values for `heap_size`, `stack_size`, `message_queue_len`,and `memory` consumed by the process.

3.4.4 System APIs

You can obtain system information about the AtomVM virtual machine via the `erlang:system_info/1` function, which takes an atom parameter designating the desired datum. Allowable parameters include

- `process_count` The number of processes running in the system.
- `port_count` The number of ports running in the system.
- `atom_count` The number of atoms allocated in the system.
- `word_size` The word size (in bytes) on the current platform (typically 4 or 8).
- `atomvm_version` The version of AtomVM currently running (as a binary).

For example,

```
%% erlang
io:format("Atom Count: ~p~n", [erlang:system_info(atom_count)]).
```

> Note. Additional platform-specific information is supported, depending on the platform type.

See below.

Use the `atomvm:platform/0` to obtain the system platform on which your code is running. The return value of this function is an atom who's value will depend on the platform on which your application is running.

```
%% erlang
case atomvm:platform() of
    esp32 ->
        io:format("I am running on an ESP32!~n");
    stm32 ->
        io:format("I am running on an STM32!~n");
    generic_unix ->
        io:format("I am running on a UNIX box!~n")
end.
```

Use `erlang:garbage_collect/0` or `erlang:garbage_collect/1` to force the AtomVM garbage collector to run on a give process. Garbage collection will in general happen automatically when additional free space is needed and is rarely needed to be called explicitly.

The 0-arity version of this function will run the garbage collector on the currently executing process.

```
%% erlang
Pid = ... %% get a reference to some pid
ok = erlang:garbage_collect(Pid).
```

3.4.5 System Time

AtomVM supports numerous function for accessing the current time on the device.

Use `erlang:timestamp/0` to get the current time since the UNIX epoch (Midnight, Jan 1, 1970, UTC), at microsecond granularity, expressed as a triple (mega-seconds, seconds, and micro-seconds):

```
%% erlang
{MegaSecs, Secs, MicroSecs} = erlang:timestamp().
```

User `erlang:system_time/1` to obtain the seconds, milliseconds or microseconds since the UNIX epoch (Midnight, Jan 1, 1970, UTC):

```
%% erlang
Seconds = erlang:system_time(second).
MilliSeconds = erlang:system_time(millisecond).
MicroSeconds = erlang:system_time(microsecond).
```

Use `erlang:universaltime/0` to get the current time at second resolution, to obtain the year, month, day, hour, minute, and second:

```
%% erlang
{{Year, Month, Day}, {Hour, Minute, Second}} = erlang:universaltime().
```

Note. Setting the system time is done in a platform-specific manner. For information about how to set system time on the ESP32, see the Network Programming Guide.

3.4.6 Miscellaneous

Use the `erlang:md5/1` function to compute the MD5 hash of an input binary. The output is a fixed-length binary ()

```
%% erlang
Hash = erlang:md5(<<foo>>).
```

Use `atomvm:random/0` to generate a random unsigned 32-bit integer in the range `0..4294967295`:

```
%% erlang
```

```
RandomInetger = atomvm:random().
```

Use `atomvm:random_bytes/1` to return a randomly populated binary of a specified size:

```
%% erlang
RandomBinary = erlang:random_bytes(32).
```

Use `base64:encode/1` and `base64:decode/1` to encode to and decode from Base64 format. The input value to these functions may be a binary or string. The output value from these functions is an Erlang binary.

```
%% erlang
Encoded = base64:encode(<<"foo">>).
<<"foo">> = base64:decode(Encoded).
```

You can Use `base64:encode_to_string/1` and `base64:decode_to_string/1` to perform the same encoding, but to return values as Erlang list structures, instead of as binaries.

3.4.7 StackTraces

You can obtain information about the current state of a process via stacktraces, which provide information about the location of function calls (possibly including file names and line numbers) in your program.

Currently in AtomVM, stack traces can be obtained in one of following ways:

- via try-catch blocks
- via catch blocks, when an error has been raised via the `error` Bif.

> Note. AtomVM does not support `erlang:get_stacktrace/0` which was deprecated in Erlang/OTP 21 and 22, stopped working in Erlang/OTP 23 and was removed in Erlang/OTP 24. Support for accessing the current stacktrace via `erlang:process_info/2` may be added in the future.

For example a stack trace can be bound to a variable in the catch clause in a try-catch block:

```
try
    do_something()
catch
    _Class:_Error:Stacktrace ->
        io:format("Stacktrace: ~p~n", [Stacktrace])
end
```

Alternatively, a stack trace can be bound to the result of a `catch` expression, but only when the error is raised by the `error` Bif. For example,

```
{'EXIT', {foo, Stacktrace}} = (catch error(foo)),
io:format("Stacktrace: ~p~n", [Stacktrace])
```

Stack traces are printed to the console in a crash report, for example, when a process dies unexpectedly.

Stacktrace data is represented as a list of tuples, each of which represents a stack "frame". Each tuple is of the form:

```
[{Module :: module(), Function :: atom(), Arity :: non_neg_integer(), AuxData ::
aux_data()}]
```

where `aux_data()` is a (possibly empty) properties list containing the following elements:

```
[{file, File :: string(), line, Line :: pos_integer()}]
```

Stack frames are ordered from the frame "closest" to the point of failure (the "top" of the stack) to the frame furthest from the point of failure (the "bottom" of the stack).

Stack frames will contain file and line information in the AuxData list if the BEAM files (typically embedded in AVM files) include <<"Line">> chunks generated by the compiler. Otherwise, the AuxData will be an empty list.

> Note. Adding line information to BEAM files not only increases the size of BEAM files in storage, but calculation of file and line information can have a non-negligible impact on memory usage. Memory-sensitive applications should consider not including line information in BEAM files.

The PackBEAM tool does not include file and line information in the AVM files it creates, but file and line information can be included via a command line option. For information about the PackBEAM too, see the `PackBEAM` tool.

3.4.8 Math

AtomvVM supports the following standard functions from the OTP `math` module:

- `cos/1`
- `acos/1`
- `acosh/1`
- `asin/1`
- `asinh/1`
- `atan/1`
- `atan2/2`
- `atanh/1`
- `ceil/1`
- `cosh/1`
- `exp/1`
- `floor/1`
- `fmod/2`
- `log/1`
- `log10/1`
- `log2/1`
- `pow/2`
- `sin/1`
- `sinh/1`
- `sqrt/1`
- `tan/1`
- `tanh/1`
- `pi/0`

The input values for these functions may be `float` or `integer` types. The return value is always a value of `float` type.

Input values that are out of range for the specific mathematical function or which otherwise are invalid or yield an invalid result (e.g., division by 0) will result in a `badarith` error.

> Note. If the AtomVM virtual machine is built with floating point arithmetic support disabled, these functions will result in a `badarg` error.

3.5 ESP32-specific APIs

Certain APIs are specific to and only supported on the ESP32 platform. This section describes these APIs.

3.5.1 System-Level APIs

As noted above, the `erlang:system_info/1` function can be used to obtain system-specific information about the platform on which your application is deployed.

You can request ESP32-specific information using using the following input atoms:

- `esp_free_heap_size` Returns the available free space in the ESP32 heap.
- `esp_largest_free_block` Returns the size of the largest free continuous block in the ESP32 heap.
- `esp_get_minimum_free_size` Returns the smallest ever free space available in the ESP32 heap since boot, this will tell you how close you have come to running out of free memory.
- `esp_chip_info` Returns map of the form `#{features := Features, cores := Cores, revision := Revision, model := Model}`, where `Features` is a list of features enabled in the chip, from among the following atoms: `[emb_flash, bgn, ble, bt]`; `Cores` is the number of CPU cores on the chip; `Revision` is the chip version; and `Model` is one of the following atoms: `esp32`, `esp32_s2`, `esp32_s3`, `esp32_c3`.
- `esp_idf_version` Return the IDF SDK version, as a string.

For example,

```
%% erlang
FreeHeapSize = erlang:system_info(esp_free_heap_size).
```

3.5.2 Non-volatile Storage

AtomVM provides functions for setting, retrieving, and deleting key-value data in binary form in non-volatile storage (NVS) on an ESP device. Entries in NVS survive reboots of the ESP device, and can be used a limited "persistent store" for key-value data.

> Note. NVS storage is limited in size, and NVS keys are restricted to 15 characters. Try to avoid writing frequently to NVS storage, as the flash storage may degrade more rapidly with repeated writes to the medium.

NVS entries are stored under a namespace and key, both of which are expressed as atoms. AtomVM uses the namespace `atomvm` for entries under its control. Applications may read from and write to the `atomvm` namespace, but they are strongly discouraged from doing so, except when explicitly stated otherwise.

To set a value in non-volatile storage, use the `esp:set_binary/3` function, and specify a namespace, key, and value:

```
%% erlang
Namespace = <<"my-namespace">>,
Key = <<"my-key">>,
esp:set_binary(Namespace, Key, <<"some-value">>).
```

To retrieve a value in non-volatile storage, use the `esp:get_binary/2` function, and specify a namespace and key. You can optionally specify a default value (of any desired type), if an entry does not exist in non-volatile storage:

```
%% erlang
```

```
Value = esp:get_binary(Namespace, Key, <<"default-value">>).
```

To delete an entry, use the `esp:erase_key/2` function, and specify a namespace and key:

```
%% erlang
ok = esp:erase_key(Namespace, Key).
```

You can delete all entries in a namespace via the `esp:erase_all/1` function:

```
%% erlang
ok = esp:erase_all(Namespace).
```

Finally, you can delete all entries in all namespaces on the NVS partition via the `esp:reformat/0` function:

```
%% erlang
ok = esp:reformat().
```

Applications should use the `esp:reformat/0` function with caution, in case other applications are making using the non-volatile storage.

> Note. NVS entries are currently stored in plaintext and are not encrypted. Applications should exercise caution if sensitive security information, such as account passwords, are stored in NVS storage.

3.5.3 Restart and Deep Sleep

You can use the `esp:restart/0` function to immediately restart the ESP32 device. This function does not return a value.

```
%% erlang
esp:restart().
```

Use the `esp:reset_reason/0` function to obtain the reason for the ESP32 restart. Possible values include:

- `esp_rst_unknown`
- `esp_rst_poweron`
- `esp_rst_ext`
- `esp_rst_sw`
- `esp_rst_panic`
- `esp_rst_int_wdt`
- `esp_rst_task_wdt`
- `esp_rst_wdt`
- `esp_rst_deepsleep`
- `esp_rst_brownout`
- `esp_rst_sdio`

Use the `esp:deep_sleep/1` function to put the ESP device into deep sleep for a specified number of milliseconds. Be sure to safely stop any critical processes running before this function is called, as it will cause an immediate shutdown of the device.

```
%% erlang
esp:deep_sleep(60*1000).
```

Use the `esp:sleep_get_wakeup_cause/0` function to inspect the reason for a wakeup. Possible return values include:

- `sleep_wakeup_ext0`

- `sleep_wakeup_ext1`
- `sleep_wakeup_timer`
- `sleep_wakeup_touchpad`
- `sleep_wakeup_ulp`
- `sleep_wakeup_gpio`
- `sleep_wakeup_uart`
- `sleep_wakeup_wifi`
- `sleep_wakeup_cocpu`
- `sleep_wakeup_cocpu_trag_trig`
- `sleep_wakeup_bt`
- `undefined` (no sleep wakeup)
- `error` (unknown other reason)

The values matches the semantics of `esp_sleep_get_wakeup_cause`.

```
%% erlang
case esp:sleep_get_wakeup_cause() of
    sleep_wakeup_timer ->
        io:format("Woke up from a timer~n");
    sleep_wakeup_ext0 ->
        io:format("Woke up from ext0~n");
    sleep_wakeup_ext1 ->
        io:format("Woke up from ext1~n");
    _ ->
        io:format("Woke up for some other reason~n")
end.
```

Use the `esp:sleep_enable_ext0_wakeup/2` and `esp:sleep_enable_ext1_wakeup` functions to configure ext0 and ext1 wakeup mechanisms. They follow the semantics of `esp_sleep_enable_ext0_wakeup` and `esp_sleep_enable_ext1_wakeup`.

```
%% erlang
-spec shutdown() -> no_return().
shutdown() ->
    % Configure wake up when GPIO 37 is set to low (M5StickC main button)
    ok = esp:sleep_enable_ext0_wakeup(37, 0),
    % Deep sleep for 1 hour
    esp:deep_sleep(60*60*1000).
```

3.5.4 Miscellaneous

The `freq_hz` function can be used to retrieve the clock frequency of the chip.

- `esp:freq_hz/0`

3.6 Peripherals

The AtomVM virtual machine and libraries support APIs for interfacing with peripheral devices connected to the ESP32. This section provides information about these APIs.

3.6.1 GPIO

You can read and write digital values on GPIO pins using the `gpio` module, using the `digital_read/1` and `digital_write/2` functions. You must first set the direction of the pin

using the `gpio:set_direction/2` function, using `input` or `output` as the direction parameter.

To read the value of a GPIO pin (`high` or `low`), use `gpio:digital_read/1`:

```
%% erlang
Pin = 2,
gpio:set_direction(Pin, input),
case gpio:digital_read(Pin) of
    high ->
        io:format("Pin ~p is high ~n", [Pin]);
    low ->
        io:format("Pin ~p is low ~n", [Pin])
end.
```

To set the value of a GPIO pin (`high` or `low`), use `gpio:digital_write/2`:

```
%% erlang
Pin = 2,
gpio:set_direction(Pin, output),
gpio:digital_write(Pin, low).
```

Interrupt Handling

You can get notified of changes in the state of a GPIO pin by using the `gpio:set_int/2` function. This function takes a reference to a GPIO Pin and a trigger. Allowable triggers are `rising`, `falling`, `both`, `low`, `high`, and `none` (to disable an interrupt).

When a trigger event occurs, such as a pin rising in voltage, a tuple will be delivered to the process containing the atom `gpio_interrupt` and the pin.

```
%% erlang
Pin = 2,
gpio:set_direction(Pin, input),
GPIO = gpio:open(),
ok = gpio:set_int(GPIO, Pin, rising),
receive
    {gpio_interrupt, Pin} ->
        io:format("Pin ~p is rising ~n", [Pin])
end.
```

Interrupts can be removed by using the `gpio:remove_int/2` function.

Use the `gpio:close/1` function to close the GPIO driver and free any resources in use by it, supplying a reference to a previously opened GPIO driver instance. Any references to the closed GPIO instance are no longer valid after a successful call to this function, and all interrupts will be removed.

```
%% erlang
ok = gpio:close(GPIO).
```

Since only one instance of the GPIO driver is allowed, you may also simply use `gpio:stop/0` to remove all interrupts, free the resouces, and close the GPIO driver port.

```
%% erlang
ok = gpio:stop().
```

3.6.2 I2C

The `i2c` module encapsulates functionality associated with the 2-wire Inter-Integrated Circuit (I2C) interface.

> Note. Information about the ESP32 I2C interface can be found in the IDF SDK I2C Documentation.

The AtomVM I2C implementation uses the AtomVM Port mechanism and must be initialized using

the `i2c:open/1` function. The single parameter contains a properties list, with the following elements:

Key	Value Type	Required	Description
`scl_io_num`	`integer()`	yes	I2C clock pin (SCL)
`sda_io_num`	`integer()`	yes	I2C data pin (SDA)
`i2c_clock_hz`	`integer()`	yes	I2C clock frequency (in hertz)
`i2c_num`	`0 .. I2C_NUM_MAX - 1`	no (default: `0`)	I2C port number. `I2C_NUM_MAX` is defined by the device SDK. On ESP32, this value is 1.

For example,

```
%% erlang
I2C = i2c:open([
    {scl_io_num, 21}, {sda_io_num, 22}, {i2c_clock_hz, 40000}
])
```

Once the port is opened, you can use the returned `I2C` instance to read and write bytes to the attached device.

Both read and write operations require the I2C bus address from which data is read or to which data is written. A devices address is typically hard-wired for the specific device type, or in some cases may be changed by the addition or removal of a resistor.

In addition, you may optionally specify a register to read from or write to, as some devices require specification of a register value. Consult your device's data sheet for more information and the device's I2C bus address and registers, if applicable.

There are two patterns for writing data to an I2C device:

1. Queuing `i2c:qwrite_bytes/2,3` write operations between calls to `i2c:begin_transmission/1` and `i2c:end_transmission/1`. In this case, write operations are queued locally and dispatched to the target device when the `i2c:end_transmission/1` operation is called;
2. Writing a byte or sequence of bytes in one `i2c:write_bytes/2,3` operation.

The choice of which pattern to use will depend on the device being communicated with. For example, some devices require a sequence of write operations to be queued and written in one atomic write, in which case the first pattern is appropriate. E.g.,

```
%% erlang
ok = i2c:begin_transmission(I2C),
ok = i2c:qwrite_bytes(I2C, DeviceAddress, Register1, <<"some sequence of
bytes">>),
ok = i2c:qwrite_bytes(I2C, DeviceAddress, Register2, <<"some other of bytes">>),
ok = i2c:end_transmission(I2C),
```

In other cases, you may just need to write a byte or sequence of bytes in one operation to the device:

```
%% erlang
ok = i2c:write_bytes(I2C, DeviceAddress, Register1, <<"write it all in one go">>),
```

Reading bytes is more straightforward. Simply use `i2c:read_bytes/3,4`, specifying the port instance, device address, optionally a register, and the number of bytes to read:

```
%% erlang
BinaryData = i2c:read_bytes(I2C, DeviceAddress, Register, Len)
```

To close the I2C driver and free any resources in use by it, use the `i2c:close/1` function, supplying a reference to the I2C driver instance created via `i2c:open/1`:

```
%% erlang
ok = i2c:close(I2C)
```

Once the I2C driver is closed, any calls to `i2c` functions using a reference to the I2C driver instance should return with the value `{error, noproc}`.

3.6.3 SPI

The `spi` module encapsulates functionality associated with the 4-wire Serial Peripheral Interface (SPI) in leader mode.

> Note. Information about the ESP32 SPI leader mode interface can be found in the IDF SDK SPI Documentation.

The AtomVM SPI implementation uses the AtomVM Port mechanism and must be initialized using the `spi:open/1` function. The single parameter to this function is a properties list containing two elements:

- `bus_config` – a properties list containing entries for the SPI bus
- `device_config` – a properties list containing entries for each device attached to the SPI Bus

The `bus_config` properties list contains the following entries:

Key	Value Type	Required	Description
`miso_io_num`	`integer()`	yes	SPI leader-in, follower-out pin (MOSI)
`mosi_io_num`	`integer()`	yes	SPI leader-out, follower-in pin (MISO)
`sclk_io_num`	`integer()`	yes	SPI clock pin (SCLK)

The `device_config` entry is a properties list containing entries for each device attached to the SPI Bus. Each entry in this list contains the user-selected name (as an atom) of the device, followed by configuration for the named device.

Each device configuration is a properties list containing the following entries:

Key	Value Type	Required	Description
`spi_clock_hz`	`integer()`	yes	SPI clock frequency (in hertz)
`spi_mode`	`0..3`	yes	SPI mode, indicating clock polarity (`CPOL`) and clock phase (`CPHA`). Consult the SPI specification and data sheet for your device, for more information about how to control the behavior of the SPI clock.
`spi_cs_io_num`	`integer()`	yes	SPI chip select pin (CS)
`address_len_bits`	`0..64`	yes	number of bits in the address field of a read-/write operation (for example, 8, if the transaction address field is a single byte)
`command_len_bits`	`0..16`	default: 0	number of bits in the command field of a read-/write operation (for example, 8, if the transaction command field is a single byte)

For example,

```
%% erlang
SPIConfig = [
    {bus_config, [
        {miso_io_num, 19},
        {mosi_io_num, 27},
        {sclk_io_num, 5}
    ]},
    {device_config, [
```

```
        {my_device_1, [
            {spi_clock_hz, 1000000},
            {spi_mode, 0},
            {spi_cs_io_num, 18},
            {address_len_bits, 8}
        ]}
        {my_device_2, [
            {spi_clock_hz, 1000000},
            {spi_mode, 0},
            {spi_cs_io_num, 15},
            {address_len_bits, 8}
        ]}
    ]}
],
SPI = spi:open(SPIConfig),
...
```

In the above example, there are two SPI devices, one using pin 18 chip select (named `my_device_1`), and once using pin 15 chip select (named `my_device_2`).

Once the port is opened, you can use the returned `SPI` instance, along with the selected device name, to read and write bytes to the attached device.

To read a byte at a given address on the device, use the `spi:read_at/4` function:

```
%% erlang
{ok, Byte} = spi:read_at(SPI, DeviceName, Address, 8)
```

To write a byte at a given address on the device, use the `spi_write_at/5` function:

```
%% erlang
write_at(SPI, DeviceName, Address, 8, Byte)
```

> Note. The `spi:write_at/5` takes integer values as inputs and the `spi:read_at/4` returns integer values. You may read and write up to 32-bit integer values via these functions.

Consult your local device data sheet for information about various device addresses to read from or write to, and their semantics.

The above functions are optimized for small reads and writes to an SPI device, typically one byte at a time.

The SPI interface also supports a more generic way to read and write from an SPI device, supporting arbitrary-length reads and writes, as well as a number of different "phases" of writes, per the SPI specification.

These phases include:

- Command phase – write of an up-to 16-bit command to the SPI device
- Address Phase – write of an up-to 64-bit address to the SPI device
- Data Phase – read or write of an arbitrary amount of of data to and from the device.

Any one of these phases may be included or omitted in any given SPI transaction.

In order to achieve this level of flexibility, these functions allow users to specify the SPI transaction through a map structure, which includes fields that specify the behavior of an SPI transaction.

The following table enumerates the permissible fields in this structure:

Key	Value Type	Description
`command`	`integer()` (16-bit)	(Optional) SPI command. The low-order `command_len_bits` are written to the device.
`address`	`integer()` (64-bit)	(Optional) Device address. The low-order `address_len_bits` are written to the device.

`write_data`	`binary()`	(Optional) Data to write
`write_bits`	`non_neg_integer()`	Number of bits to write from `write_data'. If not included,then all bits will be written.
`read_bits`	`non_neg_integer()`	Number of bits to read from the SPI device. If not included, then the same number of bits will be read as were written.

To write a blob of data to the SPI device, for example, you would use:

```
WriteData = <<"some binary data">>,
ok = spi:write(SPI, DeviceName, #{write_data => WriteData})
```

To write and simultaneously read back a blob of data to the SPI device, you would use:

```
{ok, ReadData} = spi:write_read(SPI, DeviceName, #{write_data => WriteData})
```

The size of the returned data is the same as the size of the written data, unless otherwise specified by the `read_bits` field.

Use the `spi:close/1` function to close the SPI driver and free any resources in use by it, supplying a reference to a previously opened SPI driver instance. Any references to the closed SPI instance are no longer valid after a successful call to this function.

```
%% erlang
ok = spi:close(SPI).
```

3.6.4 UART

The `uart` module encapsulates functionality associated with the Universal Asynchronous Receiver/-Transmitter (UART) interface supported on ESP32 devices. Some devices, such as NMEA GPS receivers, make use of this interface for communicating with an ESP32.

> Note. Information about the ESP32 UART interface can be found in the IDF SDK UART Documentation.

The AtomVM UART implementation uses the AtomVM Port mechanism and must be initialized using the `uart:open/2` function.

The first parameter indicates the ESP32 UART hardware interface. Legal values are:

```
"UART0" | "UART1" | "UART2"
```

The selection of the hardware interface dictates the default RX and TX pins on the ESP32:

Port	RX pin	TX pin
`UART0`	GPIO_3	GPIO_1
`UART1`	GPIO_9	GPIO_10
`UART2`	GPIO_16	GPIO_17

The second parameter is a properties list, containing the following elements:

Key	Value Type	Required	Default Value	Description
`speed`	`integer()`	no	115200	UART baud rate (bits/sec)
`data_bits`	`5 | 6 | 7 | 8`	no	8	UART data bits
`stop_bits`	`1 | 2`	no	1	UART stop bits
`flow_control`	`hardware | software | none`	no	`none`	Flow control
`parity`	`even | odd | none`	no	`none`	UART parity check

For example,

```
%% erlang
UART = uart:open("UART0", [{speed, 9600}])
```

Once the port is opened, you can use the returned `UART` instance to read and write bytes to the attached device.

To read data from the UART channel, use the `uart:read/1` function. The return value from this function is a binary:

```
%% erlang
Bin = uart:read(UART)
```

To write data to the UART channel, use the `uart_write/2` function. The input data is any Erlang I/O list:

```
%% erlang
uart:write(UART, [<<"any">>, $d, $a, $t, $a, "goes", <<"here">>])
```

Consult your local device data sheet for information about the format of data to be read from or written to the UART channel.

To close the UART driver and free any resources in use by it, use the `uart:close/1` function, supplying a reference to the UART driver instance created via `uart:open/2`:

```
%% erlang
ok = uart:close(UART)
```

Once the UART driver is closed, any calls to `uart` functions using a reference to the UART driver instance should return with the value `{error, noproc}`.

3.6.5 LED Control

The LED Control API can be used to drive LEDs, as well as generate PWM signals on GPIO pins.

The LEDC API is encapsulated in the `ledc` module and is a direct translation of the IDF SDK LEDC API, with a natural mapping into Erlang. This API is intended for users with complex use-cases, and who require low-level access to the LEDC APIs.

The `ledc.hrl` module should be used for common modes, channels, duty cycle resolutions, and so forth.

```
%% erlang
-include("ledc.hrl").

...

%% create a 5khz timer
SpeedMode = ?LEDC_HIGH_SPEED_MODE,
Channel = ?LEDC_CHANNEL_0,
ledc:timer_config([
    {duty_resolution, ?LEDC_TIMER_13_BIT},
    {freq_hz, 5000},
    {speed_mode, ?LEDC_HIGH_SPEED_MODE},
    {timer_num, ?LEDC_TIMER_0}
]).

%% bind pin 2 to this timer in a channel
ledc:channel_config([
    {channel, Channel},
    {duty, 0},
    {gpio_num, 2},
    {speed_mode, ?LEDC_HIGH_SPEED_MODE},
    {hpoint, 0},
    {timer_sel, ?LEDC_TIMER_0}
```

```
]).

%% set the duty cycle to 0, and fade up to 16000 over 5 seconds
ledc:set_duty(SpeedMode, Channel, 0).
ledc:update_duty(SpeedMode, Channel).
TargetDuty = 16000.
FadeMs = 5000.
ok = ledc:set_fade_with_time(SpeedMode, Channel, TargetDuty, FadeMs).
```

3.7 Protocols

AtomVM supports network programming on devices that support it, specifically the ESP32 platform, with its built-in support for WIFI networking, and of course on the UNIX platform.

This section describes the network programming APIs available on AtomVM.

3.7.1 Network (ESP32 only)

The ESP32 supports WiFi connectivity as part of the built-in WiFi and Bluetooth radio (and in most modules, an integrated antenna). The WIFI radio on an ESP32 can operate in several modes:

- STA (Station) mode, whereby it acts as a member of an existing WiFi network;
- AP (Access Point) mode, whereby the ESP32 acts as an access point for other devices; or
- AP+STA mode, whereby the ESP32 behaves both as a member of an existing WiFi network and as an access point for other devices.

AtomVM supports these modes of operation via the `network` module, which is used to initialize the network and allow applications to respond to events within the network, such as a network disconnect or reconnect, or a connection to the ESP32 from another device.

> Note. Establishment and maintenance of network connections on roaming devices is a complex and subtle art, and the AtomVM `network` module is designed to accommodate as many IoT scenarios as possible. This section of the programmer's guide is deliberately brief and only addresses the most basic scenarios. For a more detailed explanation of the AtomVM `network` module and its many use-cases, please refer to the AtomVM Network Programming Guide.

STA mode

To connect your ESP32 to an existing WiFi network, use the `network:wait_for_sta/1,2` convenience function, which abstracts away some of the more complex details of ESP32 STA mode.

This function takes a station mode configuration, as a properties list, and optionally a timeout (in milliseconds) before connecting to the network should fail. The default timeout, if unspecified, is 15 seconds.

The station mode configuration supports the following options:

Key	Value Type	Required	Default Value	Description
`ssid`	`string()` | `binary()`	yes	-	WiFi AP SSID
`psk`	`string()` | `binary()`	yes, if network is encrypted	-	WiFi AP password
`dhcp_hostname`	`string()` | `binary()`	no	`atomvm-<MAC>` where `<MAC>` is the factory-assigned MAC-address of the device	DHCP hostname for the connecting device

> Note. The WiFi network to which you are connecting must support DHCP and IPv4. IPv6

addressing is not yet supported on AtomVM.

If the ESP32 device connects to the specified network successfully, the device's assigned address, netmask, and gateway address will be returned in an `{ok, ...}` tuple; otherwise, an error is returned.

For example:

```
%% erlang
Config = [
    {ssid, <<"myssid">>},
    {psk,  <<"mypsk">>},
    {dhcp_hostname, <<"mydevice">>}
],
case network:wait_for_sta(Config, 15000) of
    {ok, {Address, _Netmask, _Gateway}} ->
        io:format("Acquired IP address: ~p~n", [Address]);
    {error, Reason} ->
        io:format("Network initialization failed: ~p~n", [Reason])
end
```

Once connected to a WiFi network, you may begin TCP or UDP networking, as described in more detail below.

For information about how to handle disconnections and reconnections to a WiFi network, see the AtomVM Network Programming Guide.

AP mode

To turn your ESP32 into an access point for other devices, you can use the `network:wait_for_ap/1,2` convenience function, which abstracts away some of the more complex details of ESP32 AP mode. When the network is started, the ESP32 device will assign itself the `192.168.4.1` address. Any devices that connect to the ESP32 will take addresses in the `192.168.4/24` network.

This function takes an access point mode configuration, as a properties list, and optionally a timeout (in milliseconds) before starting the network should fail. The default timeout, if unspecified, is 15 seconds.

The access point mode configuration supports the following options:

Key	Value Type	Required	Default Value	Description
`ssid`	`string()` | `binary()`	no	`atomvm-<MAC>` where `<MAC>` is the factory-assigned MAC-address of the device	WiFi AP SSID
`ssid_hidden`	`boolean()`	no	`false`	Whether the AP SSID should be hidden (i.e., not broadcast)

psk	string() | binary()	yes, if network is encrypted	-	WiFi AP password. Warning: If this option is not specified, the network will be an open network, to which anyone who knows the SSID can connect and which is not encrypted.
ap_max_connections	non_neg_integer()	no	4	Maximum number of devices that can be connected to this AP

If the ESP32 device starts the AP network successfully, the `ok` atom is returned; otherwise, an error is returned.

For example:

```
%% erlang
Config = [
    {psk,  <<"mypsk">>}
],
case network:wait_for_ap(Config, 15000) of
    ok ->
        io:format("AP network started at 192.168.4.1~n");
    {error, Reason} ->
        io:format("Network initialization failed: ~p~n", [Reason])
end
```

Once the WiFi network is started, you may begin TCP or UDP networking, as described in more detail below.

For information about how to handle connections and disconnections from attached devices, see the AtomVM Network Programming Guide.

STA+AP mode

For information about how to run the AtomVM network in STA and AP mode simultaneously, see the AtomVM Network Programming Guide.

3.7.2 UDP

AtomVM supports network programming using the User Datagram Protocol (UDP) via the `gen_udp` module. This modules obeys the syntax and semantics of the Erlang/OTP `gen_udp` interface.

> Note. Not all of the Erlang/OTP `gen_udp` functionality is implemented in AtomVM. For details, consults the AtomVM API documentation.

To open a UDP port, use the `gen_udp:open/1,2` function. Supply a port number, and if your application plans to receive UDP messages, specify that the port is active via the `{active, true}` prop-

ertyin the optional properties list.

For example:

```
%% erlang
Port = 44404,
case gen_udp:open(Port, [{active, true}, binary]) of
    {ok, Socket} ->
        {ok, SockName} = inet:sockname(Socket)
        io:format("Opened UDP socket on ~p.~n", [SockName])
    Error ->
        io:format("An error occurred opening UDP socket: ~p~n", [Error])
end
```

If the port is active, you can receive UDP messages in your application. They will be delivered as a 5-tuple, starting with the `udp` atom, and containing the socket, address and port from which the message was sent, as well as the datagram packet, itself, as a list (by default) or a binary. To choose the format, pass `list` or `binary` in options, as with Erlang/OTP.

```
%% erlang
receive
    {udp, _Socket, Address, Port, Packet} ->
        io:format("Received UDP packet ~p from address ~p port ~p~n", [Packet,
Address, Port)])
end,
```

With a reference to a UDP `Socket`, you can send messages to a target UDP endpoint using the `gen_udp:send/4` function. Specify the UDP socket returned from `gen_udp:open/1,2`, the address (as a 4-tuple of octets), port number, and the datagram packet to send:

```
Packet = <<":?
```

> Note. IPv6 networking is not currently supported in AtomVM.

3.7.3 TCP

AtomVM supports network programming using the Transport Connection Protocol (TCP) via the `gen_tcp` module. This modules obeys the syntax and semantics of the Erlang/OTP `gen_tcp` interface.

> Note. Not all of the Erlang/OTP `gen_tcp` functionality is implemented in AtomVM. For details, consults the AtomVM API documentation.

Server-side TCP

Server side TCP requires opening a listening socket, and then waiting to accept connections from remote clients. Once a connection is established, the application may then use a combination of sending and receiving packets over the established connection to or from the remote client.

> Note. Programming TCP on the server-side using the `gen_tcp` interface is a subtle art, and this portion of the documentation will not go into all of the design choices available when designing a TCP application.

Start by opening a listening socket using the `gen_tcp:listen/2` function. Specify the port number on which the TCP server should be listening:

```
%% erlang
case gen_tcp:listen(44405, []) of
    {ok, ListenSocket} ->
        {ok, SockName} = inet:sockname(Socket),
        io:format("Listening for connections at address ~p.~n", [SockName]),
        spawn(fun() -> accept(ListenSocket) end);
    Error ->
        io:format("An error occurred listening: ~p~n", [Error])
end.
```

In this particular example, the server will spawn a new process to wait to accept a connection from a remote client, by calling the `gen_tcp:accept/1` function, passing in a reference to the listening socket. This function will block until a client has established a connection with the server.

When a client connects, the function will return a tuple `{ok, Socket}`, where `Socket` is a reference to the connection between the client and server:

```
%% erlang
accept(ListenSocket) ->
    io:format("Waiting to accept connection...~n"),
    case gen_tcp:accept(ListenSocket) of
        {ok, Socket} ->
            {ok, SockName} = inet:sockname(Socket),
            {ok, Peername} = inet:peername(Socket),
            io:format("Accepted connection.  local: ~p peer: ~p~n", [SockName,
Peername]),
            spawn(fun() -> accept(ListenSocket) end),
            echo();
        Error ->
            io:format("An error occurred accepting connection: ~p~n", [Error])
    end.
```

> Note that immediately after accepting a connection, this example code will spawn a new process to accept any new connections from other clients.

The socket returned from `gen_tcp:accept/1` can then be used to send and receive messages to the connected client:

```
%% erlang
echo() ->
    io:format("Waiting to receive data...~n"),
    receive
        {tcp_closed, _Socket} ->
            io:format("Connection closed.~n"),
            ok;
        {tcp, Socket, Packet} ->
            {ok, Peername} = inet:peername(Socket),
            io:format("Received packet ~p from ~p.  Echoing back...~n", [Packet,
Peername]),
            gen_tcp:send(Socket, Packet),
            echo()
    end.
```

In this case, the server program will continuosuly echo the received input back to the client, until the client closes the connection.

For more information about the `gen_tcp` server interface, consult the AtomVM API Reference Documentation.

Client-side TCP

Client side TCP requires establishing a connection with an endpoint, and then using a combination of sending and receiving packets over the established connection.

Start by opening a connection to another TCP endpoint using the `gen_tcp:connect/3` function. Supply the address and port of the TCP endpoint.

For example:

```
%% erlang
Address = {192, 168, 1, 101},
Port = 44405,
case gen_tcp:connect(Address, Port, []) of
    {ok, Socket} ->
        {ok, SockName} = inet:sockname(Socket),
```

```
        {ok, Peername} = inet:peername(Socket),
        io:format("Connected to ~p from ~p~n", [Peername, SockName]);
    Error ->
        io:format("An error occurred connecting: ~p~n", [Error])
end
```

Once a connection is established, you can use a combination of

```
%% erlang
SendPacket = <<":?
```

For more information about the `gen_tcp` client interface, consults the AtomVM API documentation.

Chapter 4

Example Programs

AtomVM includes a collection of useful examples for getting started. This section describes what these examples do, and how to run them, for example, on an ESP32 device.

4.1 Erlang Examples

Erlang examples may be run in the UNIX shell or on supported microcontroller devices.

4.1.1 `hello_world`

This example program prints the string "Hello World" and quits.

Command line

The `hello_world.avm` file will get created as part of a build. This file may be supplied as an argument to the `AtomVM` command:

```
shell$ ./src/AtomVM ./examples/erlang/hello_world.avm
Hello World
Return value: ok
```

4.1.2 `udp_server`

This example program listens on UDP port 44444 and will print information about the received message, including the source IP, (ephemeral) source port, and packet received, to the console.

Command line

The `udp_server.avm` file will get created as part of a build. This file may be supplied as an argument to the `AtomVM` command:

```
shell$ ./src/AtomVM ./examples/erlang/udp_server.avm
Opened UDP socket on "0.0.0.0:44404".
Waiting to receive data...
```

You can send UDP packets to the AtomVM instance using `netcat` (or `nc` on some platforms), in a separate terminal window:

```
shell$ nc -u localhost 44404
```

This command will wait for you to enter a line of text, e.g.,

```
testing 1 2 3
```

In the AtomVM termianl window, you see:

```
Received UDP packet <<116,101,115,116,105,110,103,32,49,32,50,32,51,10>> from
```

```
"127.0.0.1:55261"
Waiting to receive data...
```

Note. Netcat appends a newline character at the end of the input, so the packet binary does not display as printable text.

4.1.3 udp_client

This example program send the packet of data (":?.) to the console, every time it sends a message.

This command may be used in tandem with the `udp_server` program to illustrate sending messages between AtomVM processes over UDP.

Command line

The `udp_client.avm` file will get created as part of a build. This file may be supplied as an argument to the `AtomVM` command:

```
shell$ ./src/AtomVM ./examples/erlang/udp_client.avm
Opened UDP socket on "0.0.0.0:63665".
Sent <<58,-94,-56,-32,54,45>>
Sent <<58,-94,-56,-32,54,45>>
Sent <<58,-94,-56,-32,54,45>>
...
```

If you are running the `udp_server` program, you should see messages like the following printed to the console:

```
Received UDP packet <<58,-94,-56,-32,54,45>> from "127.0.0.1:63665"
Waiting to receive data...
Received UDP packet <<58,-94,-56,-32,54,45>> from "127.0.0.1:63665"
Waiting to receive data...
Received UDP packet <<58,-94,-56,-32,54,45>> from "127.0.0.1:63665"
Waiting to receive data...
...
```

Note. AtomVM does not currently treat characters outside of the printable ASCII character set as printable characters.

4.1.4 tcp_server

This example program listens on TCP port 44404 and accept connections on that port. Once accepted, it will wait for packets to be sent from the client. Once received, the server will print the packet received to the console, and then echo the packet back to the calling client.

Command line

The `tcp_server.avm` file will get created as part of a build. This file may be supplied as an argument to the `AtomVM` command:

```
shell$ ./src/AtomVM ./examples/erlang/tcp_server.avm
Listening on "0.0.0.0:44404".
Waiting to accept connection...
```

You can send TCP packets to the AtomVM instance using `netcat` (or `nc` on some platforms), in a separate terminal window:

```
shell$ nc localhost 44404
```

This will open a TCP connection to the `tcp_server`, and you should see the following on the console:

```
Accepted connection.  local: "127.0.0.1:44404" peer: "127.0.0.1:56628"
Waiting to receive data...
```

```
Waiting to accept connection...
```

The netcat command will wait for you to enter a line of text, e.g.,

```
testing 1 2 3
```

In the AtomVM terminal window, you see:

```
Received packet [116,101,115,116,105,110,103,32,49,32,50,32,51,10] from
"127.0.0.1:56628".  Echoing back...
Waiting to receive data...
```

> Note. Netcat appends a newline character at the end of the input, so the packet binary does not display as printable text.

You may enter as much data as you like, though by default, the packet size will be limited to 128 bytes.

If you stop the `netcat` command (via ^C), you should

```
Connection closed.
```

printed to the AtomVM console.

Note that you can have multiple, concurrent TCP/IP connections to your AtomVM server.

4.1.5 `tcp_client`

This example program send the packet of data (":?

This command may be used in tandem with the `tcp_server` program to illustrate sending messages between AtomVM processes over TCP.

> Note. You will need to change the `Address` variable in the `tcp_client.erl` program in order to test against an AtomVM server running on a different host or device.

Command line

The `tcp_client.avm` file will get created as part of a build. This file may be supplied as an argument to the `AtomVM` command:

```
shell$ ./src/AtomVM ./examples/erlang/tcp_client.avm
Connected to "127.0.0.1:44404" from "127.0.0.1:56741"
Sent <<58,-94,-56,-32,54,45>> to "127.0.0.1:44404"
Received [58,162,200,224,54,45] from "127.0.0.1:44404"
Sent <<58,-94,-56,-32,54,45>> to "127.0.0.1:44404"
Received [58,162,200,224,54,45] from "127.0.0.1:44404"
Sent <<58,-94,-56,-32,54,45>> to "127.0.0.1:44404"
Received [58,162,200,224,54,45] from "127.0.0.1:44404"
...
```

If you are running the `tcp _server` program, you should see messages like the following printed to the console:

```
Accepted connection.  local: "127.0.0.1:44404" peer: "127.0.0.1:56741"
Waiting to receive data...
Waiting to accept connection...
Received packet [58,162,200,224,54,45] from "127.0.0.1:56741".  Echoing back...
Waiting to receive data...
Received packet [58,162,200,224,54,45] from "127.0.0.1:56741".  Echoing back...
Waiting to receive data...
Received packet [58,162,200,224,54,45] from "127.0.0.1:56741".  Echoing back...
Waiting to receive data...
...
```

> Note. AtomVM does not currently treat characters outside of the printable ASCII character set as printable characters.

You may run multiple concurrent instances of the `tcp_client` against a single `tcp_server` instance.

4.2 ESP32 Examples

AtomVM includes examples that are specifically designed for the ESP32 and other microcontrollers.

4.3 Flashing AtomVM Examples for ESP32

In order to run the ESP32 examples, you will need to flash the example AVM files that are created as part of the build to your device.

In the remainder of this document, we assume the `flash.sh` script, located in the `tools/dev` directory of the AtomVM source tree.

> Note. You must set the `ESP_IDF` environment variable to the root directory of the ESP IDF SDK installation on your development machine.

You can control the serial port and baud rate via the `FLASH_SERIAL_PORT` and `FLASH_BAUD_RATE` environment variables, e.g.,

```
shell$ export FLASH_SERIAL_PORT="/dev/tty.SLAB_USBtoUART"
shell$ export FLASH_BAUD_RATE=921600
```

The default values for these variables, if not set, are `/dev/ttyUSB0` and `115200`, respectively.

> Note. Experiment with baud rates (e.g., 921600). You may find you can shorten the flash-debug-flash cycle with higher rates.

You can montor the console output of these examples by issuing the `monitor` target to `make`, in the `src/platforms/esp32` directory of the AtomVM source tree:

```
shell$ make monitor
MONITOR
--- WARNING: Serial ports accessed as /dev/tty.* will hang gdb if launched.
--- Using /dev/cu.SLAB_USBtoUART instead...
--- idf_monitor on /dev/cu.SLAB_USBtoUART 115200 ---
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H ---
ets Jun  8 2016 00:22:57
...
```

4.3.1 `blink`

The `blink` example will turn the blue LED on an ESP32 SoC (pin 2) on and off, once every second.

Flash the example program to your device as follows:

```
shell$ ./tools/dev/flash.sh build/examples/erlang/esp32/blink.avm
...
Hard resetting via RTS pin...
```

You should see the blue LED turn on and off on your ESP32 device.

4.3.2 `esp_random`

This demo program illustrates use of the ESP32 `random`, `restart`, and `reset_reason` functions.

The program will generate a random binary of a random size (at most 127 bytes) every 5 seconds. If a 0-length byte sequence is generated (1:128 probability), the ESP will restart.

Flash the example program to your device as follows:

```
shell$ ./tools/dev/flash.sh build/examples/erlang/esp32/esp_random.avm
...
Hard resetting via RTS pin...
```

You should see something like the following output when monitoring the ESP32 output (truncated for brevity):

```
shell$ make monitor
...
Found AVM partition: size: 1048576, address: 0x110000
Booting file mapped at: 0x3f420000, size: 1048576
Starting: esp_random.beam...
---
Reset reason: esp_rst_poweron
Random bytes: <<71,13,221,24,8,15,...,197,120,152,205>>
Random bytes: <<37,155,124,177,44,141,40,106,...,43,48,62,109,2,78,39,107>>
Random bytes:
<<217,210,239,183,...,78,68,253,146,212,71,17,208,219,126,240,218,34,0,152,80,20,166,194,106>>
Random bytes: <<112,77,123,249,162,...,238,237,128,227,58,29,64,74>>
...
<<"">>
ets Jun  8 2016 00:22:57
...
Found AVM partition: size: 1048576, address: 0x110000
Booting file mapped at: 0x3f420000, size: 1048576
Starting: esp_random.beam...
---
esp_rst_sw
Random bytes:
<<155,174,204,143,232,202,136,...,118,177,77,230,10,21,72,91,92,160,198,115,249,217,206,52,102,3
...
```

4.3.3 esp_nvs

This demo program illustrates the use of ESP32 non-volatile storage (NVS).

The program will store the number of times the device has been rebooted, along with the start time, in NVS.

Flash the example program to your device as follows:

```
shell$ ./tools/dev/flash.sh build/examples/erlang/esp32/esp_nvs.avm
...
Hard resetting via RTS pin...
```

You should see the following output when monitoring the ESP32 output (truncated for brevity):

```
shell$ make monitor
...
Found AVM partition: size: 1048576, address: 0x110000
Booting file mapped at: 0x3f420000, size: 1048576
Starting: esp_nvs.beam...
---
Saving count 0 to NVS...
Reset device to increment.
AtomVM finished with return value = ok
going to sleep forever..
```

Hit the reset button on your device, and the ESP device will reboot, and display something like the following:

```
Found AVM partition: size: 1048576, address: 0x110000
Booting file mapped at: 0x3f420000, size: 1048576
```

```
Starting: esp_nvs.beam...
---
Saving count 1 to NVS...
Reset device to increment.
AtomVM finished with return value = ok
going to sleep forever..
```

4.3.4 reformat_nvs

This demo program will reformat the non-volatile storage (NVS) partition.

Flash the example program to your device as follows:

```
shell$ ./tools/dev/flash.sh build/examples/erlang/esp32/reformat_nvs.avm
...
Hard resetting via RTS pin...
```

You should see the following output when monitoring the ESP32 output (truncated for brevity):

```
shell$ make monitor
...
Found AVM partition: size: 1048576, address: 0x110000
Booting file mapped at: 0x3f420000, size: 1048576
Starting: esp_nvs.beam...
---
Warning: Reformatted NVS partition!
AtomVM finished with return value = ok
going to sleep forever..
```

The NVS partition on your ESP device should be reformatted.

> Note. This program will irrevocably delete all existing key-values stored on the NVS partition. Use with caution.

4.3.5 set_network_config

This demo program can be used to set the WIFI credentials in NVS. Setting WIFI credentials in NVS can greatly simplify the task of running ESP programs that require connectivity to WIFI networks.

> Note. Credentials are stored unencrypted and in plaintext and should not be considered secure. Future versions may use encrypted NVS storage.

Edit the `sta_network_config.erl` program and set the `Ssid` binary with your WIFI AP SSID, and `Psk` binary with the password used to access your WIFI network. Save the file, rebuild, and flash to your device:

```
shell$ make
...
shell$ ./tools/dev/flash.sh build/examples/erlang/esp32/sta_network_config.avm
...
Hard resetting via RTS pin...
```

You should see the following output when monitoring the ESP32 output (truncated for brevity):

```
shell$ make monitor
...
Found AVM partition: size: 1048576, address: 0x110000
Booting file mapped at: 0x3f420000, size: 1048576
Starting: set_network_config.beam...
---
{atomvm,sta_ssid,<<"myssid">>}
{atomvm,sta_psk,<<"xxxxxx">>}
AtomVM finished with return value = ok
going to sleep forever..
```

You may now run programs that use your WIFI network (see below) without needing to enter WIFI credentials.

4.3.6 sta_network

The `sta_network` example will connect to your local WiFi network and obtain and IP address. Once a connection is established, a `connected` message will be displayed. Once an IP address is obtained, the device IP address, netmask, and gateway will be displayed on the console.

> Note. AtomVM currently only supports station mode (STA).
>
> Note. AtomVM currently only supports IPv4 addresses.
>
> Note. If you have not set WIFI credentials in NVS (see above), you will need to edit the `examples/erlang/esp32/sta_network.erl` source file and set the `ssid` and `psk` parameters to match your local WiFi network, and then rebuild the example.

Flash the example program to your device as follows:

```
shell$ ./tools/dev/flash.sh build/examples/erlang/esp32/sta_network.avm
...
Hard resetting via RTS pin...
```

You should see the following output when monitoring the ESP32 output (truncated for brevity):

```
shell$ make monitor
...
Starting: sta_network.beam...
---
I (220) wifi: wifi driver task: 3ffc3b54, prio:23, stack:3584, core=0
I (220) wifi: wifi firmware version: d5da5a5
I (220) wifi: config NVS flash: enabled
I (220) wifi: config nano formatting: disabled
I (230) system_api: Base MAC address is not set, read default base MAC address
from BLK0 of EFUSE
I (240) system_api: Base MAC address is not set, read default base MAC address
from BLK0 of EFUSE
I (270) wifi: Init dynamic tx buffer num: 32
I (270) wifi: Init data frame dynamic rx buffer num: 32
I (270) wifi: Init management frame dynamic rx buffer num: 32
I (270) wifi: Init static rx buffer size: 1600
I (280) wifi: Init static rx buffer num: 10
I (280) wifi: Init dynamic rx buffer num: 32
I (290) NETWORK: starting wifi: SSID: [myssid], password: [XXXXXXXX].
I (360) phy: phy_version: 4000, b6198fa, Sep  3 2018, 15:11:06, 0, 0
I (370) wifi: mode : sta (3c:71:bf:84:d9:08)
I (370) NETWORK: SYSTEM_EVENT_STA_START received.
I (490) wifi: n:1 0, o:1 0, ap:255 255, sta:1 0, prof:1
I (1470) wifi: state: init -> auth (b0)
I (1480) wifi: state: auth -> assoc (0)
I (1490) wifi: state: assoc -> run (10)
I (1500) wifi: connected with myssid, channel 1
I (1500) wifi: pm start, type: 1
I (1500) NETWORK: SYSTEM_EVENT_STA_CONNECTED received.
I (3690) event: sta ip: 192.168.1.236, mask: 255.255.255.0, gw: 192.168.1.1
I (3690) NETWORK: SYSTEM_EVENT_STA_GOT_IP: 192.168.1.236
connected
{{192,168,1,236},{255,255,255,0},{192,168,1,1}}
```

4.3.7 udp_server_blink

The `udp_server_blink` example will connect to your local WiFi network and obtain and IP address. It will then start a UDP server on port 44444. When a UDP message is received, the blue LED on the ESP32 SoC (pin 2) will toggle on and off.

Note. AtomVM currently only supports station mode (STA).

Note. AtomVM currently only supports IPv4 addresses.

Note. You will need to edit the `examples/erlang/esp32/udp_server_blink.erl` source file and set the `ssid` and `psk` parameters to match your local WiFi network, and then rebuild the example.

Flash the example program to your device as follows:

```
shell$ ./tools/dev/flash.sh build/examples/erlang/esp32/udp_server_blink.avm
...
Hard resetting via RTS pin...
```

You should see the following output when monitoring the ESP32 output (truncated for brevity):

```
shell$ make monitor
...
Found AVM partition: size: 1048576, address: 0x110000
Booting file mapped at: 0x3f420000, size: 1048576
Starting: udp_server_blink.beam...
---
I (222) wifi: wifi driver task: 3ffc3de8, prio:23, stack:3584, core=0
I (222) wifi: wifi firmware version: d5da5a5
I (222) wifi: config NVS flash: enabled
I (232) wifi: config nano formatting: disabled
I (232) system_api: Base MAC address is not set, read default base MAC address
from BLK0 of EFUSE
I (242) system_api: Base MAC address is not set, read default base MAC address
from BLK0 of EFUSE
I (312) wifi: Init dynamic tx buffer num: 32
I (312) wifi: Init data frame dynamic rx buffer num: 32
I (312) wifi: Init management frame dynamic rx buffer num: 32
I (312) wifi: Init static rx buffer size: 1600
I (322) wifi: Init static rx buffer num: 10
I (322) wifi: Init dynamic rx buffer num: 32
I (332) NETWORK: starting wifi: SSID: [myssid], password: [XXXXXXXX].
I (442) phy: phy_version: 4000, b6198fa, Sep  3 2018, 15:11:06, 0, 0
I (442) wifi: mode : sta (3c:71:bf:84:d9:08)
I (442) NETWORK: SYSTEM_EVENT_STA_START received.
I (572) wifi: n:1 0, o:1 0, ap:255 255, sta:1 0, prof:1
I (1552) wifi: state: init -> auth (b0)
I (1552) wifi: state: auth -> assoc (0)
I (1562) wifi: state: assoc -> run (10)
I (1582) wifi: connected with myssid, channel 1
I (1582) wifi: pm start, type: 1
I (1582) NETWORK: SYSTEM_EVENT_STA_CONNECTED received.
I (2212) event: sta ip: 192.168.1.236, mask: 255.255.255.0, gw: 192.168.1.1
I (2212) NETWORK: SYSTEM_EVENT_STA_GOT_IP: 192.168.1.236
Acquired IP address: "192.168.1.236" Netmask: "255.255.255.0" Gateway:
"192.168.1.1"
Opened UDP socket on "0.0.0.0:44404".
Waiting to receive data...
```

You can send UDP packets to the AtomVM instance using `netcat` (or `nc` on some platforms), in a separate terminal window:

```
shell$ nc -u 192.168.1.236 44404
```

Every time you enter a line of text, the blue LED on the ESP32 SoC (pin 2) should toggle on and off, and you should see output on the console, such as

```
Received UDP packet <<100,115,102,115,100,10>> from "192.168.1.237:53291"
Waiting to receive data...
```

4.3.8 tcp_server_blink

The `tcp_server_blink` example will connect to your local WiFi network and obtain and IP address. It will then start a TCP server on port 44404. When a TCP message is received, the blue LED on the ESP32 SoC (pin 2) will toggle on and off.

> Note. AtomVM currently only supports station mode (STA).
>
> Note. AtomVM currently only supports IPv4 addresses.
>
> Note. You will need to edit the `examples/erlang/esp32/tcp_server_blink.erl` source file and set the `ssid` and `psk` parameters to match your local WiFi network, and then rebuild the example.

Flash the example program to your device as follows:

```
shell$ ./tools/dev/flash.sh build/examples/erlang/esp32/tcp_server_blink.avm
...
Hard resetting via RTS pin...
```

You should see the following output when monitoring the ESP32 output (truncated for brevity):

```
shell$ make monitor
Found AVM partition: size: 1048576, address: 0x110000
Booting file mapped at: 0x3f420000, size: 1048576
Starting: tcp_server_blink.beam...
---
start
I (296) wifi: wifi driver task: 3ffc650c, prio:23, stack:3584, core=0
I (296) wifi: wifi firmware version: 9415913
I (296) wifi: config NVS flash: enabled
I (296) wifi: config nano formatting: disabled
I (296) system_api: Base MAC address is not set, read default base MAC address
from BLK0 of EFUSE
I (306) system_api: Base MAC address is not set, read default base MAC address
from BLK0 of EFUSE
I (346) wifi: Init dynamic tx buffer num: 32
I (346) wifi: Init data frame dynamic rx buffer num: 32
I (346) wifi: Init management frame dynamic rx buffer num: 32
I (346) wifi: Init static rx buffer size: 1600
I (356) wifi: Init static rx buffer num: 10
I (356) wifi: Init dynamic rx buffer num: 32
I (366) NETWORK: starting wifi: SSID: [myssid], password: [XXXXXXXX].
I (446) phy: phy_version: 4008, c9ae59f, Jan 25 2019, 16:54:06, 0, 0
I (446) wifi: mode : sta (3c:71:bf:84:d9:08)
I (446) NETWORK: SYSTEM_EVENT_STA_START received.
I (1176) wifi: n:6 0, o:1 0, ap:255 255, sta:6 0, prof:1
I (2156) wifi: state: init -> auth (b0)
I (2166) wifi: state: auth -> assoc (0)
I (2166) wifi: state: assoc -> run (10)
I (2196) wifi: connected with myssid, channel 6
I (2196) wifi: pm start, type: 1
I (2196) NETWORK: SYSTEM_EVENT_STA_CONNECTED received.
I (2746) event: sta ip: 192.168.1.236, mask: 255.255.255.0, gw: 192.168.1.1
I (2746) NETWORK: SYSTEM_EVENT_STA_GOT_IP: 192.168.1.236
Acquired IP address: "192.168.1.236" Netmask: "255.255.255.0" Gateway:
"192.168.1.1"
Listening on "0.0.0.0:44404".
Waiting to accept connection...
```

You can send TCP packets to the AtomVM instance using `netcat` (or `nc` on some platforms), in a separate terminal window, e.g.,

```
shell$ nc 192.168.1.236 44404
```

On the ESP32 console, you should see:

```
Accepted connection.  local: "192.168.1.236:44404" peer: "192.168.1.237:55275"
Waiting to receive data...
Waiting to accept connection...
```

Every time you enter a line of text, the blue LED on the ESP32 SoC (pin 2) should toggle on and off, and the data you entered should get echoed back to the `netcat` console.

On the ESP32 console, you should see:

```
Received packet [115,100,102,115,100,102,10] from "192.168.1.237:55275".  Echoing
back...
Waiting to receive data...
```

every time a packet is sent to the server.

Chapter 5

Network Programming Guide

One of the exciting features of the ESP32 is its support for WiFi networking, allowing ESP32 micro-controllers to communicate with the outside world over common IP networking protocols, such as TCP or IDP. The ESP32 and ISF SDK supports configuring an ESP32 in station mode (STA), whereby the device connects to an existing access point, as well as "softAP" mode (AP), whereby it functions as an access point, to which other stations can connect. The ESP32 also supports a combined STA+softAP mode, which allows the device to function in both STA and softAP mode simultaneously.

AtomVM provides an Erlang API interface for interacting with the WiFi networking layer on ESP32 devices, providing support for configuring your ESP32 device in STA mode, AP mode, or a combined STA+AP mode, allowing Erlang/Elixir applications to send and receive data from other devices on a network. This interface is encapsulated in the `network` module, which implements a simple interface for connecting to existing WiFi networks or for functioning as a WiFi access point.

Once the network has been set up (in STA or AP mode), AtomVM can use various socket interfaces to interact with the socket layer to create a client or server application. For example, AtomVM supports the `gen_udp` and `gen_tcp` APIs, while AtomVM extensions may support HTTP, MQTT, and other protocols built over low-level networking interfaces.

The AtomVM networking API leverages callback functions, allowing applications to be responsive to changes in the underlying network, which can frequently occur in embedded applications, where devices can easily lose and then regain network connectivity. In such cases, it is important for applications to be resilient to changes in network availability, by closing or re-opening socket connections in response to disconnections and re-connections in the underlying network.

This document describes the basic design of the AtomVM network interfaces, and how to interact programmatically with it.

5.1 Station (STA) mode

In STA mode, the ESP32 connects to an existing WiFi network.

In this case, the input configuration should be a properties list containing a tuple of the form `{sta, <sta-properties>}`, where `<sta-properties>` is a property list containing configuration properties for the device in station mode.

The `<sta-properties>` property list should contain the following entries:

- `{ssid, string() | binary()}` The SSID to which the device should connect.
- `{psk, string() | binary()}` The password required to authenticate to the network, if required.

  Note that the station mode SSID and password *may* be stored in non-volatile storage, in which case these parameters may be skipped. See the "NVS Credentials" section below, for more information about using non-volatile storage to store credentials that persist across device

reboots.

The `network:start/1` will immediately return `{ok, Pid}`, where `Pid` is the process ID of the network server instance, if the network was properly initialized, or `{error, Reason}`, if there was an error in configuration. However, the application may want to wait for the device to connect to the target network and obtain an IP address, for example, before starting clients or services that require network access.

Applications can specify callback functions, which get triggered as events emerge from the network layer, including connection to and disconnection from the target network, as well as IP address acquisition.

Callback functions can be specified by the following configuration parameters:

- `{connected, fun(() -> term())}` A callback function which will be called when the device connects to the target network.
- `{disconnected, fun(() -> term())}` A callback function which will be called when the device disconnects from the target network.
- `{got_ip, fun((ip_info()) -> term())}` A callback function which will be called when the device obtains an IP address. In this case, the IPv4 IP address, net mask, and gateway are provided as a parameter to the callback function.

Note. IPv6 addresses are not yet supported in AtomVM.

Callback functions are optional, but are highly recommended for building robust WiFi applications. The return value from callback functions is ignored, and AtomVM provides no guarantees about the execution context (i.e., BEAM process) in which these functions are invoked.

In addition, the following optional parameters can be specified to configure the AP network:

- `{dhcp_hostname, string()|binary()}` The DHCP hostname as which the device should register (`<<"atomvm-<hexmac>">>`, where `<hexmac>` is the hexadecimal representation of the factory-assigned MAC address of the device).

The following example illustrates initialization of the WiFi network in STA mode. The example program will configure the network to connect to a specified network. Events that occur during the lifecycle of the network will trigger invocations of the specified callback functions.

```
%% erlang
Config = [
    {sta, [
        {ssid, <<"myssid">>},
        {psk,  <<"mypsk">>},
        {connected, fun connected/0,
        {got_ip, fun got_ip/1},
        {disconnected, fun disconnected/0}
        {dhcp_hostname, <<"myesp32">>}
    ]}
],
{ok, Pid} = network:start(Config),
...
```

The following callback functions will be called when the corresponding events occur during the lifetime of the network connection.

```
%% erlang
connected()) ->
    io:format("Connected to AP.~n").

gotIp(IpInfo) ->
    io:format("Got IP: ~p~n", [IpInfo]).

disconnected() ->
```

```
    io:format("Disconnected from AP.~n").
```

In a typical application, the network should be configured and an IP address should be acquired first, before starting clients or services that have a dependency on the network.

5.1.1 Convenience Functions

The `network` module supports the `network:wait_for_sta/1,2` convenience functions for applications that do not need robust connection management. These functions are synchronous and will wait until the device is connected to the specified AP. Supply the properties list specified in the `{sta, [...]}` component of the above configuration, in addition to an optional timeout (in milliseconds).

For example:

```
%% erlang
Config = [
    {ssid, <<"myssid">>},
    {psk,  <<"mypsk">>},
    {dhcp_hostname, <<"mydevice">>}
],
case network:wait_for_sta(Config, 15000) of
    {ok, {Address, _Netmask, _Gateway}} ->
        io:format("Acquired IP address: ~p~n", [Address]);
    {error, Reason} ->
        io:format("Network initialization failed: ~p~n", [Reason])
end
```

5.2 AP mode

In AP mode, the ESP32 starts a WiFi network to which other devices (laptops, mobile devices, other ESP32 devices, etc) can connect. The ESP32 will create an IPv4 network, and will assign itself the address `192.168.4.1`. Devices that attach to the ESP32 in AP mode will be assigned sequential addresses in the `192.168.4.0/24` range, e.g., `192.168.4.2`, `192.168.4.3`, etc.

To initialize the ESP32 device in AP mode, the input configuration should be a properties list containing a tuple of the form `{ap, <ap-properties>}`, where `<ap-properties>` is a property list containing configuration properties for the device in AP mode.

The `<ap-properties>` property list may contain the following entries:

- `{ssid, string() | binary()}` The SSID to which the device should connect.
- `{psk, string() | binary()}` The password required to authenticate to the network, if required. Note that this password must be a minimum of 8 characters.

If the SSID is omitted in configuration, the SSID name `atomvm-<hexmac>` will be created, where `<hexmac>` is the hexadecimal representation of the factory-assigned MAC address of the device. This name should be sufficiently unique to disambiguate it from other reachable ESP32 devices, but it may also be difficult to read or remember.

If the password is omitted, then an *open network* will be created, and a warning will be printed to the console. Otherwise, the AP network will be started using WPA+WPA2 authentication.

> Note that the station mode SSID and password *may* be stored in non-volatile storage, in which case these parameters may be skipped. See the "NVS Credentials" section below, for more information about using non-volatile storage to store credentials that persist across device reboots.

The `network:start/1` will immediately return `{ok, Pid}`, where `Pid` is the process id of the network server, if the network was properly initialized, or `{error, Reason}`, if there was an error in configuration. However, the application may want to wait for the device to to be ready to

accept connections from other devices, or to be notified when other devices connect to this AP.

Applications can specify callback functions, which get triggered as events emerge from the network layer, including when a station connects or disconnects from the AP, as well as when a station is assigned an IP address.

Callback functions can be specified by the following configuration parameters:

- `{ap_started, fun(() -> term())}` A callback function which will be called when the AP endpoint has started and is ready to be connected to.
- `{sta_connected, fun((Mac::binary()) -> term())}` A callback function which will be called when a device connects to the AP. The MAC address of the connected station, as a 6-byte binary, is passed to the callback function.
- `{sta_disconnected, fun((Mac::binary()) -> term())}` A callback function which will be called when a device disconnects from the AP. The MAC address of the disconnected station, as a 6-byte binary, is passed to the callback function.
- `{sta_ip_assigned, fun((ipv4_address()) -> term())}` A callback function which will be called when the AP assigns an IP address to a station. The assigned IP address is passed to the callback function.

Note. IPv6 addresses are not yet supported in AtomVM.

Callback functions are completely optional, but are highly recommended for building robust WiFi applications. The return value from callback functions is ignored, and AtomVM provides no guarantees about the execution context (i.e., BEAM process) in which these functions are invoked.

In addition, the following optional parameters can be specified to configure the AP network:

- `{ssid_hidden, boolean()}` Whether the AP network should be not be broadcast (false, by default)
- `{max_connections, non_neg_integer()}` The maximum number of devices that can connect to this network (by default, 4)

The following example illustrates initialization of the WiFi network in AP mode. The example program will configure the network to connect to start a WiFi network with the name `myssid` and password `mypsk`. Events that occur during the lifecycle of the network will trigger invocations of the specified callback functions.

```
%% erlang
Config = [
    {ap, [
        {ssid, <<"myssid">>},
        {psk,  <<"mypsk">>},
        {ap_started, fun ap_started/0},
        {sta_connected, fun sta_connected/1},
        {sta_ip_assigned, fun sta_ip_assigned/1},
        {sta_disconnected, fun sta_disconnected/1},
    ]}
],
{ok, Pid} = network:start(Config),
...
```

The following callback functions will be called when the corresponding events occur during the lifetime of the network connection.

```
%% erlang
ap_started() ->
    io:format("AP started.~n").

sta_connected(Mac) ->
    io:format("STA connected with mac ~p~n", [Mac]).
```

```
sta_disconnected(Mac) ->
    io:format("STA disconnected with mac ~p~n", [Mac]).

sta_ip_assigned(Address) ->
    io:format("STA assigned address ~p~n", [Address]).
```

In a typical application, the network should be configured and the application should wait for the AP to report that it has started, before starting clients or services that have a dependency on the network.

5.2.1 Convenience Functions

The `network` module supports the `network:wait_for_ap/1,2` convenience functions for applications that do not need robust connection management. These functions are synchronous and will wait until the device is successfully starts an AP. Supply the properties list specified in the `{ap, [...]}` component of the above configuration, in addition to an optional timeout (in milliseconds).

For example:

```
%% erlang
Config = [
    {psk,  <<"mypsk">>}
],
case network:wait_for_ap(Config, 15000) of
    ok ->
        io:format("AP network started at 192.168.4.1~n");
    {error, Reason} ->
        io:format("Network initialization failed: ~p~n", [Reason])
end
```

5.3 STA+AP mode

The `network` module can be started in both STA and AP mode. In this case, the ESP32 device will both connect to an access point in its STA mode, and will simultaneously serve as an access point in its role in AP mode.

In order to enable both STA and AP mode, simply provide valid configuration for both modes in the configuration structure supplied to the `network:start/1` function.

5.4 SNTP Support

You may configure the networking layer to automatically synchronize time on the ESP32 with an NTP server accessible on the network.

To synchronize time with an NTP server, add a property list with the tag `sntp` at the top level configuration passed into the `network:start/1` function. Specify the NTP hostname or IP address with which your device should sync using the `endpoint` property tag, e.g.,

```
{sntp, [{endpoint, <<"pool.ntp.org">>}]}
```

The endpoint value can be a string or binary.

5.5 NVS Credentials

It can become tiresome to enter an SSID and password for every application, and in general it is bad security practice to hard-wire WiFi credentials in your application source code.

You may instead store an STA or AP SSID and PSK in non-volatile storage (NVS) on and ESP32 device under the `atomvm` namespace. The following entries may be specified in non-volatile storage:

namespace	mode	key	type	value
`atomvm`	STA	`sta_ssid`	`binary()`	Station ID
`atomvm`	STA	`sta_psk`	`binary()`	Station password (if applicable)
`atomvm`	AP	`ap_ssid`	`binary()`	Access Point ID
`atomvm`	AP	`ap_psk`	`binary()`	Access Point password (if applicable)

If set in NVS storage, you may remove the corresponding `ssid` and `psk` parameters from the configuration used to initialize the network, and the SSID and PSK configured in NVS will be used, instead. An SSID or PSK defined explicitly in configuration will override any values in NVS.

You can set these credentials once, as follows:

```
esp:nvs_set_binary(atomvm, sta_ssid, <<"myssid">>).
esp:nvs_set_binary(atomvm, sta_psk, <<"mypsk">>).
```

or

```
esp:nvs_set_binary(atomvm, ap_ssid, <<"myssid">>).
esp:nvs_set_binary(atomvm, ap_psk, <<"mypsk">>).
```

With these settings, you can run ESP programs that initialize the network without configuring your SSID and PSK explicitly in source code.

> Note. Credentials are stored un-encrypted and in plaintext and should not be considered secure. Future versions may use encrypted NVS storage.

5.6 Stopping the Network

To stop the network and free any resources in use, issue the `stop/0` function:

```
network:stop().
```

> Note. This function is currently not well tested.

Chapter 6

Build Instructions

This guide is intended for anyone interested in the implementation of AtomVM, including developers who would like to provide enhancements or bug fixes to the core AtomVM virtual machine, as well as providing third-party extensions via the implementation of AtomVM Nifs or Ports. Understanding the virtual machine and how it is implemented is also interesting in its own right, and having an understanding of the underlying mechanics can be helpful for writing better Erlang and Elixir programs.

The AtomVM virtual machine itself, including the runtime code execution engine, as well as built-in functions and Nifs is implemented in C. The core standard and AtomVM libraries are implemented in Erlang and Elixir.

The native C parts of AtomVM compile to machine code on MacOS, Linux, and FreeBSD platforms. The C code also compiles to run on the ESP32 and STM32 platforms. Typically, binaries for these platforms are created on a UNIX-like environment (MacOS or Linux, currently) using tool-chains provided by device vendors to cross-compile and target specific device architectures.

The Erlang and Elixir parts are compiled to BEAM byte-code using the Erlang (`erlc`) and Elixir compilers. Currently, Erlang/OTP versions 21 and 22 are supported.

This guide provides information about how to build AtomVM for the various supported platforms (Generic UNIX, ESP32, and STM32).

> Note. In order to build AtomVM AVM files for ESP32 and STM32 platforms, you will also need to build AtomVM for the Generic UNIX platform of your choice.

6.1 Downloading AtomVM

The AtomVM source code is available by cloning the AtomVM github repository:

```
shell$ git clone https://github.com/atomvm/AtomVM
```

> Note. Downloading the AtomVM github repository requires the installation of the `git` program. Consult your local OS documentation for installation of the `git` package.

6.2 Source code organization

Source code is organized as follows:

- `src` Contains the core AtomVM virtual machine source code;
- `lib` Contains the Erlang and Elixir core library source code;
- `tools` Contains AtomVM tooling, including the `PackBEAM` executable, as well as build support tooling;
- `examples` Contains sample programs for demonstration purposes;

- `tests` Contains test code run as part of test qualification;
- `doc` Contains documentation source code and content.

The `src` directory is broken up into the core platform-independent AtomVM library (`libAtomVM`), and platform-dependent code for each of the supported platforms (Generic UNIX, ESP32, and STM32).

For information about porting to new platforms, see Porting to new platforms, below.

6.3 Building for Generic UNIX

The following instructions apply to unix-like environments, including Linux, FreeBSD, and MacOS.

> Note. The Generic UNIX is useful for running and testing simple AtomVM programs. Not all of the AtomVM APIs, specifically, APIs that are dependent on various device integration, are supported on this platform.

6.3.1 Build Requirements

The following software is required in order to build AtomVM in generic UNIX systems:

- `gcc` or `llvm` tool chains
- `cmake`
- `make`
- `gperf`
- `zlib`
- Erlang/OTP 21

Consult your local OS documentation for instructions about how to install these components.

6.3.2 Build Instructions

The AtomVM build for generic UNIX systems makes use of the `cmake` tool for generating `make` files from the top level AtomVM directory. With CMake, you generally create a separate directory for all output files (make files, generated object files, linked binaries, etc). A common pattern is to create a clcal `build` directory, and then point `cmake` to the parent directory for the root of the source tree:

```
shell$ mkdir build
shell$ cd build
shell$ cmake ..
...
```

This command will create all of the required make files for creating the AtomVM binary, tooling, and core libraries. You can create all of these object using the `make` command:

```
shell$ make -j 8
...
```

> Note. You may specify `-j <n>`, where `<n>` is the number of CPUs you would like to assign to run the build in parallel.

Upon completion, the `AtomVM` executable can be found in the `build/src` directory.

The AtomVM core Erlang library can be found in the generated `libs/atomvmlib.avm` AVM file.

Special Note for MacOS users

You may build an Apple Xcode project, for developing, testing, and debugging in the Xcode IDE, by specifying the Xcode generator. For example, from the top level AtomVM directory:

```
shell$ mkdir xcode
shell$ cmake -G Xcode ..
...
shell$ open AtomVM.xcodeproj
```

The above commands will build and open an AtomVM project in the Xcode IDE.

6.3.3 Running tests

There are currently two sets of suites of tests for AtomVM:

- Erlang tests (`erlang_tests`) A set of unit tests for basic Erlang functionality, exercising support BEAM opcodes, built-in functions (Bifs) and native functions (Nifs).
- Library tests, exercising functionality in the core Erlang and Elixir libraries.

To run the Erlang tests, run the `test-erlang` executable in the `tests` directory:

```
shell$ ./tests/test-erlang
```

This will run a suite of several score unit tests. Check the status of the executable after running the tests. A non-zero return value indicates a test failure.

To run the Library tests, run the corresponding AVM module in the `tests/libs` directory using the `AtomVM` executable. For example:

```
shell$ ./src/AtomVM ./tests/libs/estdlib/test_estdlib.avm
```

This will run a suite of several unit tests for the specified library. Check the status of the executable after running the tests. A non-zero return value indicates a test failure.

Tests for the following libraries are supported:

- `estdlib`
- `eavmlib`
- `alisp`

6.4 Building for ESP32

Building AtomVM for ESP32 must be done on either a Linux or MacOS build machine.

In order to build a complete AtomVM image for ESP32, you will also need to build AtomVM for the Generic UNIX platform (typically, the same build machine you are suing to build AtomVM for ESP32).

6.4.1 Build Requirements

The following software is required in order to build AtomVM for the ESP32 platform:

- Espressif Xtensa tool chains
- Espressif IDF SDK version 3.x (4.x support is currently *experimental*)
- `cmake`
- GNU `make` (for ESP-IDF versions lower than 4)

Instructions for downloading and installing the Espressif IDF SDK and tool chains are outside of the scope of this document. Please consult the IDF SDKGetting Started guide for more information.

6.4.2 Build Instructions

Change directories to the `src/platforms/esp32` directory under the AtomVM source tree root:

```
shell$ cd <atomvm-source-tree-root>
shell$ cd src/platforms/esp32
```

Start by updating the default build configuration of local `sdkconfig` file via the `idf.py reconfigure` command:

```
shell$ idf.py set-target esp32
shell$ idf.py reconfigure
```

> Note. For those familiar with esp-idf the build can be customized using `menuconfig` instead of `reconfigure`
>
> `shell$ idf.py menuconfig`
>
> This command will bring up a curses dialog box where you can make adjustments such as not including AtomVM components that are not desired in a particular build. You can also change the behavior of a crash in the VM to print the error and reboot, or halt after the error is printed. Extreme caution should be used when changing any non AtomVM settings. You can quit the program by typing `Q`. Save the changes, and the program will exit.

You can now build AtomvVM using the build command:

```
shell$ idf.py build
...
```

> Note. You may specify `-j <n>`, where `<n>` is the number of CPUs you would like to assign to run the build in parallel.

This command, once completed, will create the Espressif bootloader, partition table, and AtomVM binary. The last line of the output should read something like the following:

```
To flash all build output, run 'idf.py flash' or:
python /path/to/esp-idf-sdk/components/esptool_py/esptool/esptool.py --chip esp32
--port /dev/ttyUSB0 --baud 115200 --before default_reset --after hard_reset
write_flash
-z --flash_mode dio --flash_freq 40m --flash_size detect
0x1000
/path-to-atomvm-source-tree/Atomvm/src/platforms/esp32/build/bootloader/bootloader.bin
0x10000
/path-to-atomvm-source-tree/Atomvm/src/platforms/esp32/build/atomvvm-esp32.bin
0x8000 /path-to-atomvm-source-tree/Atomvm/src/platforms/esp32/build/partitions.bin
```

At this point, you can run `idf.py flash` to upload the 3 binaries up to your ESP32 device, and in some development scenarios, this is a preferable shortcut.

However, first, we will build a single binary image file containing all of the above 3 binaries, as well as the AtomVM core libraries. See Section 6.4.5, below. But first, it is helpful to understand a bit about how the AtomVM partitioning scheme works, on the ESP32.

6.4.3 Flash Layout

The AtomVM Flash memory is partitioned to include areas for the above binary artifacts created from the build, as well areas for runtime information used by the ESP32 and compiled Erlang/Elixir code.

The flash layout is roughly as follows (not to scale):

```
+-----------------+  ------------- 0x0000
|    secure       |
|     boot        | 4KB
|                 |
```

```
+-----------------+  ------------- 0x1000
|                 |             ^
|   boot loader   | 28KB        |
|                 |             |
+-----------------+             |
| partition table | 3KB         |
+-----------------+             |
|                 |             |
|       NVS       | 24KB        |
|                 |             |
+-----------------+             |
|     PHY_INIT    | 4KB         |
+-----------------+             | AtomVM
|                 |             | binary
|                 |             | image
|                 |             |
|     AtomVM      |             |
|     Virtual     | 1.75MB      |
|     Machine     |             |
|                 |             |
|                 |             |
+-----------------+             |
|     lib.avm     | 256KB       v
+-----------------+  ------------- 0x210000
|                 |             ^
|                 |             |
|     main.avm    | 1MB+        | Erlang/Elixir
|                 |             | Application
|                 |             |
|                 |             v
+-----------------+  ------------- end
```

The following table summarizes the partitions created on the ESP32 when deploying AtomVM:

Partition	Offset	Length	Description
Secure Boot	0x00	4kB	Initialization vectors and other data needed for ESP32 secure boot.
Bootloader	0x1000	28kB	The ESP32 bootloader, as built from the IDF-SDK. AtomVM does not define its own bootloader.
Partition Table	0x8000	3kB	The AtomVM-defined partition table.
NVS	0x9000	24kB	Space for non-volatile storage.
PHY_INIT	0xF000	4kB	Initialization data for physical layer radio signal data.
AtomVM virtual machine	0x10000	1.75mB	The AtomVM virtual machine (compiled from C code).
lib.avm	0x1D0000	256k	The AtomVM BEAM library, compiled from Erlang and Elixir files in the AtomVM source tree.
main.avm	0x210000	1mB	The user application. This is where users flash their compiled Erlang/Elixir code

6.4.4 The `lib.avm` and `main.avm` partitions

The `lib.avm` and `main.avm` partitions are intended to store Erlang/Elixir libraries (compiled down to BEAM files, and assembled as AVM files).

The `lib.avm` partition is intended for core Erlang/Elixir libraries that are built as part of the AtomVM build. The release image of AtomVM (see below) includes both the AtomVM virtual machine and the `lib.avm` partition, which includes the BEAM files from the `estdlib` and `eavmlib` libraries.

In contrast, the `main.avm` partition is intended for user applications. Currently, the `main.avm` partition starts at address `0x210000`, and it is to that location to which application developers should

flash their application AVM files.

The AtomVM search path for BEAM modules starts in the `main.avm` partition and falls back to `lib.avm`. Users should not have a need to override any functionality in the `lib.avm` partition, but if necessary, a BEAM module of the same name in the `main.avm` partition will be loaded instead of the version in the `lib.avm` partition.

> Note. The location of the `main.avm` partition may change over time, depending on the relative sizes of the AtomVM binary and `lib.avm` partitions.

6.4.5 Building a Release Image

The `<atomvm-source-tree-root>/tools/release/esp32` directory contains the `mkimage.sh` script that can be used to create a single AtomVM image file, which can be distributed as a release, allowing application developers to develop AtomVM applications without having to build AtomVM from scratch.

> Note. Before running the `mkimage.sh` script, you must have a complete build of both the esp32 project, as well as a full build of the core Erlang libraries in the `libs` directory. The script configuration defaults to assuming that the core Erlang libraries will be written to the `build/libs` directory in the AtomVM source tree. You should pass the `--build_dir <path>` option to the `mkimage.sh` script, with `<path>` pointing to your AtomVM build directory, if you target a different build directory when running CMake.

Running this script will generate a single `atomvm-<sha>.img` file in the `build` directory of the esp32 source tree, where `<sha>` is the git hash of the current checkout. This image contains the ESP32 bootloader, AtomVM executable, and the `eavmlib` and `estdlib` Erlang libraries in one file, which can then be flashed to address `0x1000`.

The `mkimage.sh` script is run from the src/platform/esp32 directory as follows:

```
shell$ ./build/mkimage.sh
Writing output to /home/frege/AtomVM/src/platforms/esp32/build/atomvm-602e6bc.img
=============================================
Wrote bootloader at offset 0x1000 (4096)
Wrote partition-table at offset 0x8000 (32768)
Wrote AtomvVM Virtual Machine at offset 0x10000 (65536)
Wrote AtomvVM Core BEAM Library at offset 0x110000 (1114112)
```

Users can then use the `esptool.py` directly to flash the entire image to the ESP32 device, and then flash their applications to the `main.app` partition at address `0x210000`,

But first, it is a good idea to erase the flash, e.g.,

```
shell$ esptool.py --chip esp32 --port /dev/ttyUSB0 erase_flash
esptool.py v2.1
Connecting........_
Chip is ESP32D0WDQ6 (revision 1)
Uploading stub...
Running stub...
Stub running...
Erasing flash (this may take a while)...
Chip erase completed successfully in 5.4s
Hard resetting...
```

You can then use the `./build/flash.sh` tool (or `esptool.py` directly, if you prefer), to flash the entire image to your device:

```
shell$ FLASH_OFFSET=0x1000 ./build/flash.sh ./build/atomvm-602e6bc.img
esptool.py v2.8-dev
Serial port /dev/tty.SLAB_USBtoUART
Connecting........_
Chip is ESP32D0WDQ6 (revision 1)
Features: WiFi, BT, Dual Core, Coding Scheme None
Crystal is 40MHz
```

```
MAC: 30:ae:a4:1a:37:d8
Uploading stub...
Running stub...
Stub running...
Changing baud rate to 921600
Changed.
Configuring flash size...
Auto-detected Flash size: 4MB
Wrote 1163264 bytes at 0x00001000 in 15.4 seconds (603.1 kbit/s)...
Hash of data verified.
Leaving...
Hard resetting via RTS pin...
```

Note. Flashing the full AtomVM image will delete all entries in non-volatile storage. Only flash the full image if you have a way to recover and re-write any such data.

A `flashimage.sh` is also generated with each build that will flash the full image using the correct flash offset for the chip the build was configured for.

```
shell$ ./build/flashimage.sh
```

Finally, you can then flash your own application, e.g.,

```
shell$ ./build/flash.sh ../../../build/examples/erlang/esp32/blink.avm
%%
%% Flashing examples/erlang/esp32/blink.avm (size=4k)
%%
esptool.py v2.8-dev
Serial port /dev/tty.SLAB_USBtoUART
Connecting........_
Chip is ESP32D0WDQ6 (revision 1)
Features: WiFi, BT, Dual Core, Coding Scheme None
Crystal is 40MHz
MAC: 30:ae:a4:1a:37:d8
Uploading stub...
Running stub...
Stub running...
Changing baud rate to 921600
Changed.
Configuring flash size...
Auto-detected Flash size: 4MB
Wrote 16384 bytes at 0x00210000 in 0.2 seconds (611.7 kbit/s)...
Hash of data verified.
Leaving...
Hard resetting via RTS pin...
```

Note. Since the Erlang core libraries are flashed to the ESP32 device, it is not necessary to include core libraries in your application AVM files. Users may be interested in using downstream development tools, such as the Elixir ExAtomVM Mix task, or the Erlang AtomVM Rebar3 Plugin for doing day-to-day development of applications for the AtomVM platform.

6.4.6 Adding custom Nifs, Ports, and third-party components

While AtomVM is a functional implementation of the Erlang virtual machine, it is nonetheless designed to allow developers to extend the VM to support additional integrations with peripherals and protocols that are not otherwise supported in the core virtual machine.

AtomVM supports extensions to the VM via the implementation of custom native functions (Nifs) and processes (AtomVM Ports), allowing users to extend the VM for additional functionality, and you can add your own custom Nifs, ports, and additional third-party components to your ESP32 build by adding them to the `components` directory, and the ESP32 build will compile them automatically.

For more information about building components for the IDF SDK, consult the IDF SDK Build System documentation.

The instructions for adding custom Nifs and ports differ in slight detail, but are otherwise quite similar. In general, they involve:

1. Adding the custom Nif or Port to the `comonents` directory of the AtomVM source tree;
2. Adding the component to the corresponding `main/component_nifs.txt` or `main/component_ports.txt` files;
3. Building the AtomVM binary.

> Note. The Espressif SDK and tool chains do not, unfortunately, support dynamic loading of shared libraries and dynamic symbol lookup. In fact, dynamic libraries are not supported at all on the ESP32 using the IDF SDK; instead, any code that is needed at runtime must be statically linked into the application.

Custom Nifs and Ports are available through third parties. Follow the instructions provided with these custom components for detailed instruction for how to add the Nif or Port to your build.

More detailed instructions follow, below, for implementing your own Nif or Port.

Adding a custom AtomVM Nif

To add support for a new peripheral or protocol using custom AtomVM Nif, you need to do the following:

- Choose a name for your nif (e.g, "my_nif"). Call this `<moniker>`.
- In your source code, implement the following two functions:
  - `void <moniker>_nif_init(GlobalContext *global);`
    - This function will be called once, when the application is started.
  - `const struct Nif *<moniker>_nif_get_nif(const char *nifname);`
    - This function will be called to locate the Nif during a function call.

Example:

```
void my_nif_init(GlobalContext *global);
const struct Nif *my_nif_get_nif(const char *nifname);
```

> Note. Instructions for implementing Nifs is outside of the scope of this document.

- Add your `<moniker>` to the `main/component_nifs.txt` file in the `src/platforms/esp32` directory.

> Note. The `main/component_nifs.txt` file will not exist until after the first clean build.

Adding a custom AtomVM Port

To add support for a new peripheral or protocol using an AtomVM port, you need to do the following:

- Choose a name for your port (e.g, "my_port"). Call this `<moniker>`.
- In your source code, implement the following two functions:
  - `void <moniker>_init(GlobalContext *global);`
    - This function will be called once, when the application is started.
  - `Context *<moniker>_create_port(GlobalContext *global, term opts);`
    - This function will be called to locate the Nif during a function call.

Example:

```
void my_port_init(GlobalContext *global);
Context *my_port_create_port(GlobalContext *global, term opts);
```

Note. Instructions for implementing Ports is outside of the scope of this document.

- Add your `<moniker>` to the `main/component_ports.txt` file in the `src/platforms/esp32` directory.

  Note. The `main/component_ports.txt` file will not exist until after the first clean build.

6.5 Building for STM32

TODO

Chapter 7

AtomVM Internals

7.1 What is an Abstract Machine?

AtomVM is an "abstract" or "virtual" machine, in the sense that it simulates, in software, what a physical machine would do when executing machine instructions. In a normal computing machine (e.g., a desktop computer), machine code instructions are generated by a tool called a compiler, allowing an application developer to write software in a high-level language (such as C). (In rare cases, application developers will write instructions in assembly code, which is closer to the actual machine instructions, but which still requires a translation step, called "assembly", to translate the assembly code into actual machine code.) Machine code instructions are executed in hardware using the machine's Central Processing Unit (CPU), which is specifically designed to efficiently execute machine instructions targeted for the specific machine architecture (e.g., Intel x86, ARM, Apple M-series, etc.) As a result, machine code instructions are typically tightly packed, encoded instructions that require minimum effort (on the part of the machine) to unpack an interpret. These a low level instructions unsuited for human interpretation, or at least for most humans.

AtomVM and virtual machines generally (including, for example, the Java Virtual Machine) perform a similar task, except that i) the instructions are not machine code instructions, but rather what are typically called "bytecode" or sometimes "opcode" instructions; and ii) the generated instructions are themselves executed by a runtime execution engine written in software, a so-called "virtual" or sometimes "abstract" machine. These bytecode instructions are generated by a compiler tailored specifically for the virtual machine. For example, the `javac` compiler is used to translate Java source code into Java VM bytecode, and the `erlc` compiler is used to translate Erlang source code into BEAM opcodes.

AtomVM is an abstract machine designed to implement the BEAM instruction set, the 170+ (and growing) set of virtual machine instructions implemented in the Erlang/OTP BEAM.

> Note that there is no abstract specification of the BEAM abstract machine and instruction set. Instead, the BEAM implementation by the Erlang/OTP team is the definitive specification of its behavior.

At a high level, the AtomVM abstract machine is responsible for:

- Loading and execution of the BEAM opcodes encoded in one or more BEAM files;
- Managing calls to internal and external functions, handling return values, exceptions, and crashes;
- Creation and destruction of Erlang "processes" within the AtomVM memory space, and communication between processes via message passing;
- Memory management (allocation and reclamation) of memory associated with Erlang "processes"
- Pre-emptive scheduling and interruption of Erlang "processes"
- Execution of user-defined native code (Nifs and Ports)

- Interfacing with the host operating system (or facsimile)

This document provides a description of the AtomVM abstract machine, including its architecture and the major components and data structures that form the system. It is intended for developers who want to get involved in bug fixing or implementing features for the VM, as well as for anyone interested in virtual machine internals targeted for BEAM-based languages, such as Erlang or Elixir.

7.2 AtomVM Data Structures

This section describes AtomVM internal data structures that are used to manage the load and runtime state of the virtual machine. Since AtomVM is written in C, this discussion will largely be in the context of native C data structures (i.e., `structs`). The descriptions will start at a fairly high level but drill down to some detail about the data structures, themselves. This narrative is important, because memory is limited on the target architectures for AtomVM (i.e., micro-controllers), and it is important to always be aware of how memory is organized and used in a way that is as space-efficient as possible.

7.2.1 The GlobalContext

We start with the top level data structure, the `GlobalContext` struct. This object is a singleton object (currently, and for the foreseeable future), and represents the root of all data structures in the virtual machine. It is in essence in 1..1 correspondence with instances of the virtual machine.

> Note. Given the design of the system, it is theoretically possible to run multiple instances of the AtomVM in one process space. However, no current deployments make use of this capability.

In order to simplify the exposition of this structure, we break the fields of the structure into manageable subsets:

- Process management – fields associated with the management of Erlang (lightweight) "processes"
- Atoms management – fields associated with the storage of atoms
- Module Management – fields associated with the loading of BEAM modules
- Reference Counted Binaries – fields associated with the storage of binary data shared between processes
- Other data structures

These subsets are described in more detail below.

> Note. Not all fields of the `GlobalContext` structure are described in this document.

Process Management

As a BEAM implementation, AtomVM must be capable of spawning and managing the lifecycle of Erlang lightweight processes. Each of these processes is encapsulated in the `Context` structure, described in more detail in subsequent sections.

The `GlobalContext` structure maintains a list of running processes and contains the following fields for managing the running Erlang processes in the VM:

- `processes_table` the list of all processes running in the system
- `waiting_processes` the subset of processes that are waiting to run (e.g., waiting for a message or timeout condition). This set is the complement of the set of ready processes.
- `ready_processes` the subset of processes that are ready to run. This set is the complement of the set of waiting processes.

Each of these fields are doubly-linked list (ring) structures, i.e, structs containing a `prev` and `next`

pointer field. The `Context` data structure begins with two such structures, the first of which links the `Context` struct in the `processes_table` field, and the second of which is used for either the `waiting_processes` or the `ready_processes` field.

> Note. The C programming language treats structures in memory as contiguous sequences of fields of given types. Structures have no hidden pramble data, such as you might find in C++ or who knows what in even higher level languages. The size of a struct, therefore, is determined simply by the size of the component fields.

The relationship between the `GlobalContext` fields that manage BEAM processes and the `Context` data structures that represent the processes, themselves, is illustrated in the following diagram:

Error opening image file: cannot identify image file '/home/runner/-work/AtomVM/AtomVM/build/doc/src/_static/globalcontext-processes.svg'

> Note. The `Context` data structure is described in more detail below.

Module Management

An Aside: What's in a HashTable?

7.2.2 Modules

7.2.3 Contexts

7.3 Runtime Execution Loop

7.4 Module Loading

7.5 Function Calls and Return Values

7.6 Exception Handling

7.7 The Scheduler

7.8 Stacktraces

Stacktraces are computed from information gathered at load time from BEAM modules loaded into the application, together with information in the runtime stack that is maintained during the execution of a program. In addition, if a BEAM file contains a `Line` chunk, additional information is added to stack traces, including the file name (as defined at compile time), as well as the line number of a function call.

> Note. Adding line information to a BEAM file adds non-trivial memory overhead to applications and should only be used when necessary (e.g., during the development process). For applications to make the best use of memory in tightly constrained environments, packagers should consider removing line information all together from BEAM files and rely instead on

logging or other mechanisms for diagnosing problems in the field.

Newcomers to Erlang may find stacktraces slightly confusing, because some optimizations taken by the Erlang compiler and runtime can result in stack frames "missing" from stack traces. For example, tail-recursive function calls, as well as function calls that occur as the last expression in a function clause, don't involve the creation of frames in the runtime stack, and consequently will not appear in a stacktrace.

7.8.1 Line Numbers

Including file and line number information in stacktraces adds considerable overhead to both the BEAM file data, as well as the memory consumed at module load time. The data structures used to track line numbers and file names are described below and are only created if the associated BEAM file contains a `Line` chunk.

The line-refs table

The line-refs table is an array of 16-bit integers, mapping line references (as they occur in BEAM instructions) to the actual line numbers in a file. (Internally, BEAM instructions do not reference line numbers directly, but instead are indirected through a line index). This table is stored on the `Module` structure.

This table is populated when the BEAM file is loaded. The table is created from information in the `Line` chunk in the BEAM file, if it exists. Note that if there is no `Line` chunk in a BEAM file, this table is not created.

The memory cost of this table is `num_line_refs * 2` bytes, for each loaded module, or 0, if there is no `Line` chunk in the associated BEAM file.

The filenames table

The filenames table is a table of (usually only 1?) file name. This table maps filename indices to `ModuleFilename` structures, which is essentially a pointer and a length (of type `size_t`). This table generally only contains 1 entry, the file name of the Erlang source code module from which the BEAM file was generated. This table is stored on the `Module` structure.

Note that a `ModuleFilename` structure points to data directly in the `Line` chunk of the BEAM file. Therefore, for ports of AtomVM that memory-map BEAM file data (e.g., ESP32), the actual file name data does not consume any memory.

The memory cost of this table is `num_filenames * sizeof(struct ModuleFilename)`, where `struct ModuleFilename` is a pointer and length, for each loaded module, or 0, if there is no `Line` chunk in the associated BEAM file.

The line-ref-offsets list

The line-ref-offsets list is a sequence of `LineRefOffset` structures, where each structure contains a ListHead (for list book-keeping), a 16-bit line-ref, and an unsigned integer value designating the code offset at which the line reference occurs in the code chunk of the BEAM file. This list is stored on the `Module` structure.

This list is populated at code load time. When a line reference is encountered during code loading, a `LineRefOffset` structure is allocated and added to the line-ref-offsets list. This list is used at a later time to find the line number at which a stack frame is called, in a manner described below.

The memory cost of this list is `num_line_refs * sizeof(struct LineRefOffset)`, for each loaded module, or 0, if there is no `Line` chunk in the associated BEAM file.

7.8.2 Raw Stacktraces

Chapter 8

Memory Management

Like most managed execution environments, AtomVM provides automated memory management for compiled Erlang/Elixir applications that run on the platform, allowing developers to focus on the logic of application programs, instead of the minutiae of managing the allocation and disposal of memory in the process heap of the program.

Because Erlang/Elixir, and the BEAM, specifically, is a shared-nothing, concurrency-based language, AtomVM can manage memory independently, for each unit of concurrency, viz., the Erlang process. While there is some global state, internally, that AtomVM manages (e.g., to manage all running processes in the system), memory management for each individual process can be performed independently of any other process.

AtomVM internally uses a "Context" structure, to manage aspects of a process (including memory management), and we use "execution context" and "Erlang process" interchangeably in this document. As usual, an Erlang process should be distinguished from the Operating System (OS) process in which Erlang processes run.

For any given execution context, there are three regions of memory that are relevant: i) the stack, ii) the heap, and iii) registers. The stack and heap actually occupy one region of memory allocated in the OS process heap (via malloc or equiv), and grow in opposite directions towards each other. Registers in AtomVM are a fixed size array of 16 elements.

The fundamental unit of memory that occupies space in the stack, heap, and registers is the `term`, which is typedef'd internally to be an integral type that fits in a single word of machine memory (i.e., a C `int`). Various tricks are used, described below, to manage and reference multi-word terms, but in general, a term (or in some cases, a term pointer) is intended to fit into a single word or memory.

This document describes the memory layout for each execution context (i.e., Erlang/Elixir process), how memory is allocated and used, how terms are represented internally, and how AtomVM makes room for more terms, as memory usage increases and as terms go out of scope and are no longer used by the application, and can hence be garbage collected.

8.1 The Context structure

8.1.1 The Heap and Stack

The heap and stack for each AtomVM process are stored in a single allocated block of memory (e.g., via the `malloc` C function) in the heap space of the AtomVM program, and the AtomVM runtime manages the allocation of portions of this memory during the execution of a program. The heap starts at the bottom of the block of memory, and grows incrementally towards the top of the allocated block, as memory is allocated in the program. Each word in the heap and stack (or in some cases, a sequence of words) represent a term that has been allocated.

The heap contains all of the allocated terms in an execution context. In some cases, the terms occupy more than one word of memory (e.g., a tuple), but in general, the heap contains a record of memory in use by the program.

The heap grows incrementally, as memory is allocated, and terms are allocated sequentially, in increasing memory addresses. There is, therefore, no memory fragmentation, properly speaking, at least insofar as a portion of memory might be in use and then freed. However, it is possible that previously allocated blocks of memory in the context heap are no longer referenced by the program. In this case, the allocated blocks are "garbage", and are reclaimed at the next garbage collection.

> Note. It is possible for the AtomVM heap, as provided by the underlying operating system, to become fragmented, as the execution context stack and heap are allocated via `malloc` or equiv. But that is a different kind of fragmentation that does not refer to the allocated block used by an individual AtomVM process.

The stack grows from the top of the allocated block toward the heap in decreasing addresses. Terms in the stack, as opposed to the heap, are either single-word terms, i.e., simple terms like small integers, process ids, etc, or *pointers* to terms in the heap. In either case, they only occupy one word of memory.

The region between the stack and heap is the free space available to the Erlang/Elixir process.

The following diagram illustrates an allocated block of memory that stores terms (or term pointers) in the heap and stack:

```
+================================+ <- heap_start --
|             word[0]            |      ^      ^
+--------------------------------+      |      |
|             word[1]            |      |      |
+--------------------------------+      |      |
|             word[2]            |      | heap |
+--------------------------------+      |      |
|               ...              |      |      |
+--------------------------------+      |      |
|                                |      v      |
+--------------------------------+ <- heap_ptr |
|                                |      ^      |
|                                |      |      |
|                                |      |      |
|                                |      | free |
|                                |      |      |
|                                |      |      |
|                                |      v      |
+--------------------------------+ <- e ----   |
|                                |      ^      |
+--------------------------------+      |      |
|                                |      |      |
+--------------------------------+      | stack|
|                                |      |      |
+--------------------------------+      |      |
|           word[n-1]            |      v      v
+================================+ <- stack_base --
```

The initial size of the allocated block for the stack and heap in AtomVM is 8 words. As heap and stack allocations grow, eventually, the amount of free space will decrease to the point where a garbage collection is required. In this case, a new but larger (typically by 2x) block of memory is allocated by the AtomVM OS process, and terms are copied from the old stack and heap to the new stack and heap. Garbage collection is described in more detail below.

8.1.2 Registers

Registered are allocated in an array of 16 terms (words) and are referenced by the `x` field in the Context data structure:

```
+---------+---------+---------+--------+
|   x[0]  |   x[1]  |   ...   |  x[15] |
+---------+---------+---------+--------+
```

Like terms in the stack, terms in registers are either single-word terms, i.e., simple terms like small integers, process ids, etc, or *pointers* to terms in the heap, in a manner described in more detail below. In either case, they only occupy one word of memory.

Registers are used as part of the BEAM instruction set to store and retrieve values that are passed between BEAM instruction opcodes.

8.1.3 Process Dictionary

AtomVM processes support a process dictionary, or map of process-specific data, as supported via the `erlang:put/2` and `erlang:get/1` functions.

The Process Dictionary contains a list of key-value pairs, where each key and value is a single-word term, either a simple term like an atom or pid, or a reference to an allocated object in the process heap. (see below)

8.1.4 Heap Fragments

AtomVM makes use of heap fragments in some edge cases, such as loading external terms from the literals table in a BEAM file. Heap fragments are individually allocated blocks of memory that contain may contain multi-word term structures. The data in heap fragments are copied into the heap during a garbage collection event, and then deleted, so heap fragments are generally short lived. However, during execution of a program, there may be references to term structures in such fragments from the stack, registers, the process dictionary, or from nested terms in the process heap.

8.1.5 Mailbox

Each Erlang process contains a process mailbox, which is a linked-list structure of messages. Each message in this list contains a term structure, which is a copy of a term sent to it, e.g., via the `erlang:send/2` operation, or `!` operator.

The representation of terms in a message is identical to that in the heap and heap fragments, with the exception that messages in the process mailbox are not garbage collected, in the way that the process heap is. Instead, messages in the process mailbox are copied to the process heap when the message is read off the mailbox (e.g., via `receive ... end`). Messages (and their term contents) are then destroyed once they are no longer needed, and after being copied into the heap.

8.1.6 Memory Graph

Memory is allocated in the execution context heap, and structured types, such as tuples and lists, generally include references to the blocks of memory that have beenn previously allocated.

For example, if we look at the memory allocated for the term

```
{foo, [{bar, self()}]}
```

we would generally see something like the following in the execution context heap:

```
|           ...             |
|                           |
+---------------------------+
|          tuple            |<--+
+---------------------------+   |
|          bar              |   |
+---------------------------+   |
|          <0.1.0>          |   |
+---------------------------+   |
|          []               |<- | --+
+---------------------------+   |   |
|          tuple ptr        |---+   |
+---------------------------+       |
|          tuple            |       |
+---------------------------+       |
```

```
|          foo              |       |
+---------------------------+       |
|          list ptr         |-------+
+---------------------------+
|                           |
|           ...             |
0123456789012345678901234567890123456789012345678 9
```

The tuple `{bar, self()}` is allocated in a block, and the list `[{bar, self()}]` (or, technically, `[{bar, self()} | []]`) contains elements that *point* to it elements (in this case, `[]` and `{bar, self()}` – note that in general, in AtomVM, the address of the tail of a list occupies the first byte in the list – more details on that below). Finally, the tuple `{foo, [{bar, self()}]}` contains the atom `foo` and a *pointer* to the list it contains.

In this way, the set of allocated blocks in the execution context heap forms a directed graph of objects, whose nodes are structured terms (lists, tuples, etc) and whose leaves are simple terms, like atoms, pids, and so forth. Note that because BEAM-based languages such as Erlang and Elixir are true functional programming languages, these directed graphs have no cycles.

The stack, registers, and process dictionary contain pointers to terms in the heap. We call these terms "root" nodes, and any term in the heap that is referenced by a root node, or any term that is so referenced by such a term, is in the path of a root node. Some terms in the heap are not in the path of a root node. We call these terms "garbage".

Note that the values in the stack and register root nodes change over time as the result of the execution of Erlang opcodes, and are dependent on the BEAM output of the Erlang compiler, along with inputs to the program being executed. Thus, a term in the process heap may become garbage, once it is no longer reachable from the root set. But once garbage, the termill will always remain garbage, at least until it is reclaimed during a garbage collection event. For more information about how the garbage collector works, see the Garbage Collection section, below.

8.2 Simple Terms

The fundamental unit of memory in AtomVM is the `term` object, which is designed to fit either into a single machine work (single-word terms), or into multiple words (so called "boxed terms" and lists).

This section enumerates the AtomVM term types, and how they are represented in memory.

> Note. The term type is overloaded in some cases to store raw pointers to memory addresses, but this is rare and well controlled.

The following term types take up a single word, referred to as "immediates" in the BEAM documentation[1]. The low-order bits of the word are used to represent the type of the term, and the high order bits represent the term contents, in a manner described in the following sections.

8.2.1 Atoms

An atom is represented as a single word, with the low-order 6 bits having the value `0xB` (`001011b`). The high order word-size-6 bits are used to represent the index of the atom in the global atom table:

```
                          |< 6  >|
+=========================+======+
|       atom index        |001011| <- 0xB
+=========================+======+
|                                |
|<---------- word-size --------->|
```

There may therefore only be `2^{word-size-6}` atoms in an AtomVM program (e.g., on a 32-bit platform, `67,108,864`). Plenty to work with!

> Note. The global atom table is a table of all allocated atoms, and is generally (at least in the limit, as modules are loaded) a fixed size table. Management of the global atom table is

outside of the scope of this document.

8.2.2 Integers

An integer is represented as a single word, with the low-order 4 bits having the value `0xF` (`1111b`). The high order word-size-6 bits are used to represent the integer value:

```
                            |< 4>|
+===========================+====+
|       integer value       |1111| <- 0xF
+===========================+====+
|                               |
|<---------- word-size --------->|
```

The magnitude of an integer is therefore limited to `2^{word-size - 4}` in an AtomVM program (e.g., on a 32-bit platform, `+- 134,217,728`).

Note. Arbitrarily large integers (bignums) are not currently supported in AtomVM.

8.2.3 nil

The special value `nil` (typically the tail of the tail … of the tail of a list, or `[]`) is the special value 0x3B:

```
+================================+
|000         ...       0000111011| <- 0x3B
+================================+
|                                |
|<---------- word-size --------->|
```

8.2.4 Pids

A Pid is represented as a single word, with the low order 4 bits indicating the Pid term type (`0x03`), and (for now), the high order `word-size - 4` bits store the local process id:

```
                            |< 4>|
+===========================+====+
|     local process id      |0011| <- 0x3
+===========================+====+
|                               |
|<---------- word-size --------->|
```

There may therefore only be `2^{word-size - 4}` Pids in an AtomVM program (e.g., on a 32-bit platform, `268,435,456`).

Note. Global process IDs are not currently supported, but they may be in the future, which may result in segmentation of the high order `word-size - 4` bits.

8.3 Boxed terms

Some term types cannot fit in a single word, and must therefore used a sequence of contiguous words to represent the term contents. These terms are called "Boxed" terms. Boxed terms use the low-order 6 bits of the first byte (`boxed[0]`) to represent the term type, and the high order `word-size - 6` bits to represent the remaining size (in words) of the boxed term, not including the first word.

8.3.1 Boxed term pointers

Before discussing the different types of boxed terms in detail, let us first see how boxed terms are referenced from the stack, registers, process dictionary, and from embedded terms in the heap. We call such references to boxed terms boxed term pointers.

A boxed term pointer is a single-word term that contains the address of the referenced term in the high-order `word-size - 2` bits, and `0x2` (`10b`) in the low-order 2 bits.

```
                              |2 |
+=============================+==+
|       term address          |10| <- term pointer type (2 bits)
+=============================+==+
|                               |
|<---------- word-size --------->|
```

Because terms (and hence the heap) are always aligned on boundaries that are divisible by the word size, the low-order 2 bits of a term address are always 0. Consequently, the high-order word-size - 2 (`1,073,741,824`, on a 32-bit platform) are sufficient to address any term address in the AtomVM address space, for 32-bit and greater machine architectures.

8.3.2 References

A reference (e.g., created via `erlang:make_ref/0`) stores a 64-bit incrementing counter value (a "ref tick"). On 64 bit machines, a Reference takes up two words – the boxed header and the 64-bit value, which of course can fit in a single word. On 32-bit platforms, the high-order 28 bits are stored in `boxed[1]`, and the low-order 32 bits are stored in `boxed[2]`:

```
                          |< 6  >|
+=========================+======+
|        boxed-size       |010000| boxed[0]
+-------------------------+------+
|     high-order ref-ticks       | boxed[1]
+================================+
|      low-order ref-ticks       | boxed[2] (32-bit only)
+= = = = = = = = = = = = = = = ==+
|                                |
|<---------- word-size --------->|
```

8.3.3 Tuples

Tuples are represented as boxed terms containing a boxed header (`boxed[0]`), a type tag of `0x00` (`000000b`), followed by a sequence of `n`-many words, which may either (copies of) single-word terms, or boxed term pointers, where `n` is the arity of the tuple:

```
                          |< 6  >|
+=========================+======+
|    boxed-size (n)       |000000| boxed[0]
+-------------------------+------+
|             element-1          | boxed[1]
+--------------------------------+
|             element-2          | boxed[2]
+--------------------------------+
|               ...              | boxed[i]
+--------------------------------+
|             element-n          | boxed[n]
+================================+
|                                |
|<---------- word-size --------->|
```

8.3.4 Maps

Maps are represented as boxed terms containing a boxed header (`boxed[0]`), a type tag of `0x3C` (`111100b`), followed by:

- a term pointer to a tuple of arity `n` containing the keys in the map;
- a sequence of `n`-many words, containing the values of the map corresponding (in order) to

the keys in the reference tuple.

The keys and values are single word terms, i.e., either immediates or pointers to boxed terms or lists.

```
        +=========================+======+
+-----> |    boxed-tuple (n)      |000000|
|       +-------------------------+------+
|       |             key-1              |
|       +--------------------------------+
|       |             key-2              |
|       +--------------------------------+
|       |               ...              |
|       +--------------------------------+
|       |             key-n              |
|       +================================+
|       |                                |
|                         ...
|       |                         |< 6  >|
|       +=========================+======+
|       |    boxed-size (n)       |111100| boxed[0]
|       +-------------------------+------+
+-----------------<   keys               | boxed[1]
        +--------------------------------+
        |             value-1            | boxed[2]
        +--------------------------------+
        |               ...              | ...
        +--------------------------------+
        |             value-n            | boxed[2 + n]
        +================================+
        |                                |
        |<---------- word-size --------->|
```

The tuple of keys may or may not be contiguous with the boxed term holding the map itself (and in general will not be, after garbage collection). In addition, maps that are modified [sic] via the `:=` operator (or via `=>`, when the key already exists in the source map) share the keys tuple, for space efficiency.

8.3.5 Binaries

Binaries are stored in several different ways, depending on their size and the kinds of data to which they refer.

Binary data less than 64 bytes in length are stored in the process heap, as so-called Heap Binaries.

Binary data greater or equal to 64 bytes is stored in two manners, depending on whether the data stored is constant data (e.g., literal binary data compiled directly into a BEAM file), or dynamically allocated data, e.g., as the result of a call to the `erlang:list_to_binary/1` Nif.

Non-const binaries are stored outside of the heap in dynamically allocated memory and are reference-counted, whereby references to dynamically allocated blocks are tracked from pointers in heap storage. This way, large blocks of binary data can be efficiently shared between processes; only a relatively small term that contains a reference to the dynamically allocated storage needs to be copied. When the reference count of non-literal binary reaches 0, the dynamically allocated memory is free'd.

Const binaries share similar features to non-const binaries in the process heap; however, instead of pointing to dynamically allocated memory that requires reference counting and memory management, the boxed term in the process heap points directly to constant memory (e.g., a term literal stored in a memory-mapped BEAM file). This is especially useful in memory constrained applications, such as the ESP32 micro-controller, where the BEAM file contents are not read into memory, but are instead directly mapped from flash storage.

Finally, a special kind of binary is used in the heap to maintain the state of a match context, when, for example, matching binary terms using Erlang bit syntax. Like non-const binaries, creation and destruction of match context binaries will affect the reference count on the binaries to which they

refer.

The following sub-sections describe these storage mechanisms and memory management in more detail.

Heap Binaries

Heap binaries are represented as boxed terms containing a boxed header (`boxed[0]`), a type tag of `0x024` (`100100b`), followed by the size in bytes of the binary, and then a sequence of `n`-many words, which contains the sequence of `size`-many bytes (`<= word-size * n`):

```
                          |< 6  >|
+=========================+======+
|    boxed-size (n)       |100100| boxed[0]
+-------------------------+------+
|         size (in bytes)        | boxed[1]
+--------------------------------+
| byte-1, byte-2, byte-3,  ...   | boxed[2]
+--------------------------------+
|               ...              | boxed[i]
+------------------+------------+
| ..., byte-{size-1}| -unused-   | boxed[n+1]
+==================+============+
|                               |
|<---------- word-size --------->|
```

> Note. If the number of bytes in a binary is not evenly divisible by the machine word size, then the remaining sequence of bytes in the last word are unused.

Reference Counted Binaries

Reference counted binaries are represented as boxed terms containing a boxed header (`boxed[0]`), a type tag of `0x020` (`100000b`), followed by the size in bytes of the binary data, a word containing a set of flags, and then a pointer to the off-heap data.

Currently, only the low-order bit of the flags field is used. A `0` value of indicates that the referenced binary is non-literal.

The off-heap data is a block of allocated data, containing:

- a ListHead structure, used to maintain a list of dynamically allocated data (mostly for book-keeping purposes);
- a reference count (unsigned integer);
- the size of the stored data;
- the stored data, itself.

All of the above data is allocated in a single block, so that it can be easily `free`'d when no longer referenced.

The reference count is initialized to 1, under the principle that that reference count is incremented for any occurrence of boxed terms that reference the same data in any heap space, including process heaps, mailbox messages, heap fragments, and so forth. Decrementing reference counts and `free`'ing data in off-heap storage is discussed in more detail below, in the Garbage Collection section.

```
                              |< 6  >|
+--> +=========================+======+
|    |    boxed-size (5)       |100000| boxed[0]
|    +-------------------------+------+
|    |         size (in bytes)        | boxed[1]
|    +--------------------------------+
|    |             flags             0| boxed[2]
|    +--------------------------------+                    off-heap storage
|    |              ptr >-------------- boxed[3] ---> +----------------------+
------
```

```
|    +--------------------------------+                |         prev         |
^
|    |              cdr               | boxed[4]       +----------------------+
| ListHead
|    +--------------------------------+                |         next         |
v
+-----------------< car               | boxed[5]       +----------------------+
------
     +==================================+                |   reference-count    |
     |<---------- word-size --------->|                +----------------------+
                                                        |         size         |
                                                        +----------------------+
                                                        |         data         |
                                                                 ...
                                                        |                      |
                                                        +----------------------+
```

Note. The size of a reference counted binary is stored both in the process heap (in the boxed term), as well as in the off-heap storage. The size count in the off-heap storage is needed in order to report the amount of data in use by binaries (e.g., via `erlang:memory/0,1`).

In addition, a reference-counted boxed term contains a cons-cell appended to the end of the boxed term, which is used by the garbage collector for tracking references. The `car` of this cell points to the boxed term, itself, and the `cdr` points to the "previous" cons cell associated with a reference counted binary in the heap, if there is one, or the empty list (`nil`), otherwise. The cons cell forms an element in the "Mark and Sweep Object" (MSO) list, used to reclaim unreferenced storage during a garbage collection event.. See the Garabage Collection section, below, for more information about the critical role of this structure in the process of reclaiming unused memory in the AtomVM virtual machine.

Const Binaries

Const binaries are stored in the same manner as Reference Counted binaries, with the following exceptions:

- The low order bit of the flags field (`boxed[2]`) is `1`, to indicate that the reference binary is constant;
- The ptr field (`boxed[3]`) points directly to the constant storage (e.g., literal data stored in a memory-mapped BEAM file);
- The trailing cons cell elements are unused, as dynamic memory management for static storage is uncesessary. These values are initialized to `nil`.

This heap structure has the following representation:

```
                          |< 6  >|
+=========================+======+
|    boxed-size (5)       |100000| boxed[0]
+-------------------------+------+
|         size (in bytes)        | boxed[1]
+--------------------------------+
|             flags             1| boxed[2]
+--------------------------------+                        static storage
|              ptr >-------------- boxed[3] -------> +----------------------+
+--------------------------------+                    |         data         |
|             unused             | boxed[4]           |                      |
+--------------------------------+                                 ...
|             unused             | boxed[5]           |                      |
+================================+                    +----------------------+
|<---------- word-size --------->|
```

Match Binaries

Match binaries are represented as boxed terms containing a boxed header (`boxed[0]`), a type tag of `0x04` (`000100b`), and the following elements:

- a reference to either a binary or another match binary that refers to a binary;
- an offset in the referenced binary used by the match opcodes;
- a saved state used for backtracking unmatched clause heads;

Like a reference counted binary, a match binary includes a trailing cons cell, whose `car` element points to the actual referenced binary (if the referenced binary is a reference-counted binary), and whose `cdr` points to the "previous" cons cell associated with a reference counted binary in the heap.

> Note. If the referenced binary is not reference-counted, the trailing cons cell elements are unused and are initialized to `nil`.

```
some
binary                            |< 6  >|
^       +=========================+======+
|       |    boxed-size (5)       |100100| boxed[0]
|       +-------------------------+------+
|       |      match-or-binary-ref       | boxed[1]
|       +--------------------------------+
|       |             offset             | boxed[2]
|       +--------------------------------+
|       |             saved              | boxed[3]
|       +--------------------------------+
|       |              cdr               | boxed[4]
|       +--------------------------------+
+--------------------< car               | boxed[5]
        +================================+
        |<---------- word-size --------->|
```

A reference to a reference-counted binary counts as a reference, in which case the creation or copying of a match binary results in the increment of the reference-counted binary's reference count, and the garbage collection of a match binary results in a decrement (and possible `free`ing) of a reference-counted binary. The trailing cons cell becomes an element of the context (or message) MSO list, and plays a critical role in garbage collection. See the garbage collection section below for more information about the role of this structure.

Sub-Binaries

Sub-binaries are represented as boxed terms containing a boxed header (`boxed[0]`), a type tag of `0x28` (`001000b`)

A sub-binary is a boxed term that points to a reference-counted binary, recording the offset into the binary and the length (in bytes) of the sub-binary. An invariant for this term is that the `offset + length` is always less than or equal to the length of the referenced binary.

```
    some
    refc
    binary                            |< 6  >|
    ^       +=========================+======+
    |       |    boxed-size (3)       |001000| boxed[0]
    |       +-------------------------+------+
    |       |              len               | boxed[1]
    |       +--------------------------------+
    |       |             offset             | boxed[2]
    |       +--------------------------------+
    +----------------< binary-ref            | boxed[3]
            +================================+
            |<---------- word-size --------->|
```

Note than when a sub-binary is copied between processes (e.g., via `erlang:send`, or `!`), the sub-binary boxed term, as well as the boxed-term that manages the reference-counted binary is copied, as well. Thus, sending a sub-binary to another process will result in an increment of the reference count on the referenced binary, and similarly, garbage collection of the sub-binary will result in a decrement of the referenced binary's reference count.

A sub-binary may be created from both const (literal) and non-const reference-counted binaries. For performance reasons, sub-binaries do not reference heap binaries.

Sub-binaries are created via the `binary:part/3` and `binary:split/2` Nifs, as well as via the `/binary` bit syntax specifier.

8.4 Lists

A list is, very simply, a cons cell, i.e., a sequence of two words, whose first word is a term (single word or term pointer) representing the tail (`cdr`) of the list, and the second of which represents the head (`car`) of the list.

```
+================================+
|                tail            | list_elem[0]
+--------------------------------+
|                head            | list_elem[1]
+================================+
|                                |
|<---------- word-size --------->|
```

> Note. Lists are typically terminated with the empty list (`[]`), represented by the nil term, described above. However, nothing in Erlang requires that a sequence of cons cells is `nil`-terminated.

Unlike boxed terms, the low-order two bits of list pointers are `0x1` (`01b`):

```
+==============================+==+
|       term address           |01| <- list pointer type (2 bits)
+==============================+==+
|                                |
|<---------- word-size --------->|
```

8.4.1 Strings

Strings are just lists of integers, but they are efficiently allocated at creation time so that a contiguous block of cons cells are created in the heap. They otherwise have the same properties of a list described above.

```
+================================+
|    address-of-next-cons     |01| elem[1]
+--------------------------------+
|             int-value          |
+--------------------------------+
|    address-of-next-cons     |01| elem[2]
+--------------------------------+
|             int-value          |
+--------------------------------+
|                ...          |01| elem[i]
+--------------------------------+
|                ...             |
+--------------------------------+
|                nil             | elem[n]
+--------------------------------+
|             int-value          |
+================================+
```

```
|                                |
|<---------- word-size --------->|
```

Note. String elements may not remain contiguous after a garbage collection event.

8.4.2 Functions

Functions are represented as boxed terms containing a boxed header (`boxed[0]`), a type tag of `0x14` (`010100b`), followed by the raw memory address of the Module data structure in which the function is defined, and the function index (so that the function can be located).

In addition, if there are any terms that are used outside of the scope of the function (i.e., closures), these terms are copied from registers into the function objects

```
                          |< 6  >|
+=========================+======+
|    boxed-size (n)       |010100| boxed[0]
+-------------------------+------+
|         module address         | boxed[1]
+--------------------------------+
|         function index         | boxed[2]
+--------------------------------+
|            closure_1           | boxed[3]
+- - - - - - - - - - - - - - - - +
|               ...              |
+- - - - - - - - - - - - - - - - +
|            closure_k           | boxed[n-1]
+= = = = = = = = = = = = = = = = +
|                                |
|<---------- word-size --------->|
```

8.5 Special Stack Types

Some terms are only used in the stack.

8.5.1 Continuation Pointer

A continuation pointer is a raw address. Because words are aligned on word boundaries, the low order two bits of a continuation pointer are always `0x0` (`(00000000)b`):

```
+================================+
|            raw address      |00|
+================================+
```

8.5.2 Catch Labels

A catch label is used to indicate a position in code to which to jump in a try-catch expression. The term occupies a single term, with the low order 6 bits having the value `0x1B`, the high order 8 bits holding the module index (`m_i`), and the middle 18 bits holding the catch label index (`l_i`):

```
|<   8  >|<     18       >|< 6  >|
+========+================+======+
|   m_i  |     l_i        |011011|
+========+================+======+
|                                |
|<---------- word-size --------->|
```

Module and catch label indices are stored outside of the process heap and are outside of the scope of this document.

8.6 Garbage Collection

Garbage collection refers to the process of removing no-longer referenced term data stored in the heap, making room for new storage, as the program requires. AtomVM implements Tracing Garbage Collection, as does Erlang Garbage Collection. Unlike some garbage collection systems (e.g., as implemented by the Java Virtual Machine), garbage collection in Erlang-based systems, is performed independently on the heap allocated for each active Erlang process; there is no single shared heap for all running Erlang processes.

A given process heap and stack occupy a single region of malloc'd memory, and it is the job of the Erlang VM to manage memory within the allocated regions. Because this region is fixed, every allocation in the heap or stack results in less free space for the Erlang process. When free space reaches a limit, AtomVM will run a garbage collection event, which will allocate a new block of memory to hold the new heap and stack (typically, enough to allocate a requested object, plus a little extra), and then copy terms from the old heap and stack to the new heap and stack. Any terms that no longer have references from term pointers in the old stack or registers are not copied to the new stack, and are therefore "collected" as garbage. In addition, any objects in the old heap that reference objects in shared memory (see reference counted binaries, above) are also managed as part of this process, in a manner described below.

```
                              +---------+ ------
                              |   new   |    ^
                              |   heap  |    |
                              +---------+    |
       ---- +----------+      |         |    |
        ^   |          |      |         |    |
        |   |   old    |      |         |    |  new
old     |   |   heap   |      |         |    |  malloc'd
malloc'd|   |          | ===> |         |    |  region
region  |   +==========+  gc  |  free   |    |
        |   |   old    |      |         |    |
        v   |   stack  |      |         |    |
       ---- +----------+      |         |    |
                              +---------+    |
                              |   new   |    |
                              |   stack |    v
                              +---------+ -----


+---+---+---+-------------------+---+
| 0 | 1 | 2 |                   | 15|
+---+---+---+-------------------+---+
registers


process dictionary
+--------+--------+
|   k1   |   v1   |
+--------+--------+
|   k2   |   v2   |
+--------+--------+
|       ...       |
```

Terms stored in the stack, registers, and process dictionary are either single-word terms (like atoms or pids) or term references, i.e., single-word terms that point to boxed terms or list cells in the heap. These terms constitute the "roots" of the memory graph of all "reachable" terms in the process.

8.6.1 When does garbage collection happen?

Garbage collection typically occurs as the result of a request for an allocation of a multi-word term in

the heap (e.g., a tuple, list, or binary, among other types), and when there is currently insufficient space in the free space between the current heap and the current stack to accommodate the allocation.

Garbage collection is a *synchronous* operation in each Context (Erlang process), but conceptually no other execution contexts are impacted (i.e., no global locks, other than those required for memory allocation in the OS process heap).

> Note. Currently, AtomVM does not support symmetric multi-processing, or execution of multiple processes in parallel on separate machine cores.

8.6.2 Garbage Collection Steps

Garbage collection in AtomVM can be broken down into the following phases:

- Allocation of a new block of memory to store the new heap and stack;
- A "shallow copy" of all root terms (from the stack, registers, and process dictionary) into the heap, as well as updates to the references in the stack, registers, and process dictionary;
- An iterative "scan and copy" of the new heap, until all "live" terms are copied to the new heap;
- A sweep of the "Mark Sweep Object" list;
- Deletion of the old heap.

The following subsections describe these phases in more detail.

Allocation

Garbage collection typically occurs as the result of a request for space on an Erlang process's heap. The amount of space requested is dependent on the kind of term being allocated, but in general, AtomVM will check the amount of free space in the heap, and if it is below the amount of requested space plus some extra (currently, 16 words), then a garbage collection will occur, with the requested allocation space being the current size of the heap, plus the requested size, plus an extra 16 words.

Allocation is a straightforward `malloc` in the (operating system) process heap of the requested set of words. This block of storage will become the "new heap", as opposed to the existing, or "old heap".

Shallow Copy

The garbage collector starts by traversing the current root set, i.e., the terms contained in the stack, registers, and keys and values in the process dictionary, and performs a "shallow copy" of the terms that are in or referenced from these root terms from the old heap to the new heap, while at the same time updating the values in the root set, as some of these values may be pointers into the old heap, and therefore need to be updated to pointers in the new heap.

A shallow copy of a term depends on the type of the term being copied. If the term is a single-word term, like an atom or pid, then the term only resides in the root set, itself, and nothing needs to be copied from the old heap to the new heap. (The term *may* occur in the heap elsewhere, but as an element of another term, like a tuple, for example.)

On the other hand, if the term in the root set points to a boxed term in the old heap, then three things happen:

- The boxed term is copied from the old heap to the new heap. Note that if the term being copied contains pointers to other boxed terms in the old heap, the pointers are *not* updated (yet); they will be as part of the iterative scan and copy (see below);
- The first word of the existing boxed term that was copied is *over-written* with a marker value (`0x2b`) in the old heap, and the second word is over-written with the address of the copied boxed term in the new heap.
- The term in the root set is updated with the address of the copied boxed term in the new heap.

This process is best illustrated with a motivating example:

```
{foo, <<1,2,3,4,...,1024>>}
```

Suppose this term resides in the old heap, and some `register[i]` is a root term pointer to this tuple in the heap:

```
+-> |              |        |             |
|   |              |        |             |
|   |              |        |             |  USED
|   |              |        |             |
          ...                      ...
|   |              |        |             |
|   +--------------+        +=============+ <-- heap
|   |     tuple    |<---+   |             |     addr
|   +--------------+    |   |             |
|   |   atom foo   |    |   |             |
|   +--------------+    |   |             |
+----< refc binary |    |   |             |  FREE
    +--------------+    |   |             |
    |              |    |   |             |
    |     ...      |    |   |     ...     |
    |              |    |   |             |
        old heap        |       new heap
                        |
            ---+----------------+---
           ... |     old-ptr    | ...
            ---+----------------+---
                  register[i]
```

The boxed term is copied to the new heap, overwritten with the marked header `0x2b`, along with a pointer to the new term, and the root term is updated with the same address:

```
+-> |              | <--------+   |             |
|   |              |          |   |             |
|   |              |          |   |             |
|   |              |          |   |             |
           ...                          ...
|   |              |          |   |             |
|   +--------------+ >>>>>>>>>|>> +-------------+
|   |     0x2b     |   +--+---|-> |   tuple     | USED
|   +--------------+   |  |   |   +-------------+
|   |     ptr  --------+  |   |   |  atom foo   |
|   +--------------+      |   |   +-------------+
+----< refc binary |      |   +----< refc binary|
    +--------------+ >>>>>|>>>>>>>+=============+ <-- new
    |              |      | COPY  |             |   heap
    |     ...      |      |       |             |   addr
         old heap         |       |             |
                          |       |             | FREE
                          |
                          |
             ---+----------------+---
            ... |     new-ptr    | ...
             ---+----------------+---
                   register[i]
```

Note that the first term of the tuple (atom `foo`) is copied to the new heap, but the pointer to the refc binary is out of date – it still points to a value in the old heap. This will be corrected in the iterative scan and copy phase, below.

After a shallow copy of the root set, all terms immediately reachable from the root set have been copied to the new heap, and any boxed terms they reference have been marked as being moved. The new heap consists of a set of contiguous copied boxed terms from the old heap, starting from the base address of the heap, to some higher address in the heap, but less than or equal to the maximum heap size on the new heap.

Iterative Scan and Copy

The iterative scan and copy phase works as follows:

- Starting with the newly created region used in the shallow copy phase in the new heap, iterate over every term in the region (call this the "scan&copy" region);
- If any term in this region is a reference to a term on the old heap that has *not* been marked as copied, perform a shallow copy of it (as described above) to the new heap, but starting at the next free address below the region being iterated over;
- Note that after iterating over all such terms in the scan and copy region, all terms are "complete", in that there are no references to boxed terms in the old heap in that region. We have, however, created a new region which may have references to boxed terms in the old heap;
- So we repeat the process on the new region, which will complete the current scan&copy requion, but which in turn may create a new region of copied terms;
- The process is repeated until no new reqions have been introduced.

The following sequence of iterative additions to the new heap illustrates this process:

```
+---------------+ ===> +---------------+ ===> +---------------+ ...
|   scan&copy   |      |    complete   |      |    complete   |
|   region      |      |    region     |      |    region     |
|               |      |               |      |               |
|               |      |               |      |               |
|               |      |               |      |               |
|               |      |               |      |               |
|               |      |               |      |               |
+---------------+      +---------------+      +---------------+
                       |     newly     |      |   scan&copy   |
                       |    copied     |      |   region      |
                       |     terms     |      |               |
                       +---------------+      +---------------+
                                              |newlycpiedterms|
                                              +---------------+



 ... ===> +---------------+ ===> +---------------+
          |    complete   |      |    complete   |
          |    region     |      |    region     |
          |               |      |               |
          |               |      |               |
          |               |      |               |
          |               |      |               |
          |               |      |               |
          |               |      |               |
          |               |      |               |
          |               |      |               |
          +---------------+      |               |
          |  scan&copy    |      |               |
          +---------------+      +---------------+
```

At the end of the iterative scan and copy, all reachable terms in the old heap will be copied to the new heap, and no boxed terms in the old heap will contain pointers to terms in the old heap. Any terms that have not been copied to the new heap are "garbage", as there are no longer any paths to them from the root set, and can therefore be destroyed,

MSO Sweep

As mentioned in the section above on binaries, AtomVM supports refrence-counted binaries, whereby binaries of a sufficiently large size (>64 bytes) are allocated outside of the process heap, and are

instead referenced from boxed terms in the heap. This way, binaries, which are immutable objects, can be shared between processes without incurring the time and space cost of a large data copy.

In order to manage the memory associated with such binaries, AtomVM tracks references to these off-heap binaries via the "Mark and Sweep Object" list, a list that keeps track of which boxed terms in the process heap have a reference to an off-heap binary. When such a boxed term is copied (e.g., from a heap to a mailbox on a `send`, or from a mailbox to a heap on a `receive`), the reference count is incremented on the off-heap binary.

The MSO list is formed via the cons cells that are appended to reference counted binary boxed terms in the process heap. The list is initially empty (nil), but as reference counted binaries are added to the process heap, they are pre-pended to the MSO list for the process (on the mailbox message, as reference-counted binaries in the mailbox need to be managed, as well).

The following diagram illustrates a set of two reference counted binaries in a process heap:

```
        |                   |
+-----> +-------------------+
|       |       refc        |
|       |      binary       |
|       |                   |
|       |                   |
|       +-------------------+ <----+
|       |        nil        |      |
|       +-------------------+      |
+---------------< car       |      |
        +-------------------+      |
        |                   |      |
                ...
        |                   |      |
+-----> +-------------------+      |
|       |       refc        |      |
|       |      binary       |      |
|       |                   |      |
|       |                   |      |
|       +-------------------+ <----|-------+
|       |        cdr >-------------+       |
|       +-------------------+              |
+--------------< car        |              |
        +-------------------+              |
        |                   |              |
        |                   |              |
                ...                        |
                                           |
                                   +-------^--------+
                                   |   mso_list     |
                                   +----------------+
```

After the new heap has been scanned and copied, as described above, the MSO list is traversed to determine if any reference-counted binaries are no longer referenced from the process heap. If any reference counted binaries in the heap have not been marked as moved from the old heap, they are, effectively, no longer referenced from the root set, and the reference count on the corresponding off-heap binary can be decremented. Furthermore, when the reference count reaches 0, the binaries can then be deleted.

> Note. Const binaries, while they have slots for entry into the MSO list, nonetheless are never "stitched" into the MSO list, as the binary data they poont to is const, endures for the lifecycle of the program, and is never deleted. Match binaries, on the other hand, do count as references, and can therefore be stitched into the MSO list. However, when they are, the reference counted binaries they point to are the actual binaries in the process heap, not the match binaries, as with the case of refc binaries on the process heap.

Deletion

Once all terms have been copied from the old heap to the new heap, and once the MSO list has been

swept for unreachable references, the old heap is simply discarded via the `free` function.

Chapter 9

Packbeam Format

AtomVM makes use of the packbeam format for aggregating beam and other file types into a single file that is used as the code base for an AtomVM application. Typically, on an embedded device, packbeam files are uploaded (e.g., via serial connection) to a specific location on flash media. The AtomVM runtime will locate the entrypoint into the application, and use the beam and other files flashed to the local media to run the uploaded application.

AtomVM provides a simple tool for generating packbeam files, but other tools have emerged for manipulation packbeam files using standard Erlang and Elixir tool chains, notably `Mix` and `rebar3`.

This document describes the packbeam format, so that both AtomVM and upstream/downstream tooling have a reference document on which to base implementations.

9.1 Overview

Packbeam files are binary-encoded aggregations of BEAM and plain data files. At a high level, a packbeam file consists of a packbeam header, followed by a sequence of files (beam or otherwise), each of which is prefixed with a header, including data about the file (name, size, flags, etc).

All binary integer values are 32-bit, in network order (big-endian). Headers and encoded files are padded when necessary and aligned on 4-byte boundaries.

At present, the AtomVM runtime treats data in packbeam files as *read-only* data. There is no support for modifying the contents on an AtomVM file by the runtime.

9.2 Packbeam Header

All AtomVM files begin with the packbeam header, a fixed 24-byte sequence of octets:

```
0x23, 0x21, 0x2f, 0x75,
0x73, 0x72, 0x2f, 0x62,
0x69, 0x6e, 0x2f, 0x65,
0x6e, 0x76, 0x20, 0x41,
0x74, 0x6f, 0x6d, 0x56,
0x4d, 0x0a, 0x00, 0x00
```

The ASCII encoding of this sequence is

```
#!/usr/bin/env AtomVM\n
```

followed by two nil (`0x00`) bytes.

The packbeam header is followed by a sequence of 0 or more encoded files. The number of files in a packbeam file is not indicated in the packbeam header; however, packbeam files do contain a special `end` file header, marking the end of the sequence of encoded files.

9.3 File encodings

Each embedded file in a packbeam file contains a file header, followed by the file contents.

9.3.1 File Header

The file header consists of the following 4 fields:

- `size` (32 bit, big-endian)
- `flags` (32-bit, big endian)
- `reserved` (32-bit, big-endian, currently unused)
- `module_name` (null-terminated sequence of bytes)

The `size` field indicates the size (in bytes) of the encoded file following the header. This size includes the file content length, in addition to any padding that may have been added to the file, in order for it to align on a 4-byte boundary.

Currently, the two low-order bits of the `flags` field are used. `0x02` indicates the file is a BEAM file, and `0x01` indicates that the file contains a `start/0` function, and is therefore suitable as an entry-point to start code execution.

> When AtomVM starts, it will scan the BEAM files in the AtomVM file, from start to finish, with which it is initialized to find the entrypoint to start code execution. It will start execution on the first BEAM file with a `start/0` function, i.e., whose flags mask against `0x03`. It is conventional, but not required, for the first file in an AtomVM file to be a BEAM file that has a `start/0` entrypoint.

The `reserved` field is currently unused.

The `module_name` is variable length, null terminated sequence of characters. Because the module name is variable-length, the header may be padded with null characters (`0x00`), in order to align the start of the file contents on a 4-byte boundary.

9.3.2 Example

The following BEAM header indicates a BEAM file with a length of 308 bytes (`0x00000134`), with a `start/0` entrypoint (`0x00000003`), and named `mylib.beam` (`0x6D796C69 622E6265 616D00`). The header has a 1-byte padding of null (`0x00`) characters.

```
00000134 00000003 00000000 6D796C69 622E6265 616D0000
```

9.3.3 BEAM files

BEAM files obey IFF encoding as detailed here, but certain information in BEAM files is stripped out in order to minimize the amount of data stored on flash.

The following BEAM chunks are included in BEAM files:

- `AtU8`
- `Code`
- `ExpT`
- `LocT`
- `ImpT`
- `LitU`
- `FunT`

- `StrT`
- `LitT`

Any other chunks are stripped out of the BEAM files before insertion into AVM files.

In addition, data in the literals table (`LitT`) are uncompressed before insertion into AVM files, as the AtomVM runtime does not include support for `zlib` decompression.

BEAM files may be padded at the end with a sequence of 1-3 null (`0x00`) characters, in order to align on 4-byte boundaries.

> Note. The `module_name` field in the file header will only contain the "base" name of the BEAM file, i.e., the file name stripped of any path information.

9.3.4 Normal Files

Normal files (e.g., text files, data files, etc.) can be stored in packbeam AVM files, as well as BEAM files. For example, a normal file might contain static configuration information, or data that is interpreted at runtime.

Normal files contain a 32-bit big-endian size prefix, indicating the size of the file data (without padding). Note that the `size` field in the file header includes the size of the data with padding, if applicable.

The AtomVM runtime provides access to data files via the `atomvm:read_priv/2` NIF. This function will create a path name formed by the `App` (atom) and `Path` (string) terms provided by this function, separated by `"/priv/"`. For example, the expression

```
atomvm:read_priv(mylib, "sample.txt")
```

yields a `binary` containing the contents of `mylib/priv/sample.txt`, if it exists, in the AtomVM packbeam file.

As a consequence, normal files should be included in packbeam files using module names that obey the above patterns.

> Note. Normal file names may encode virtual directory names, such as `mylib/priv/another/sample/text/file`. There is no requirement that the `Path` component of a normal file be a simple file name.

9.3.5 `end` file

Packbeam files end with a special `end` header. The `size` field of the `end` header is 0 bytes.

Example

The following sequence of bytes encodes the `end` header:

```
00000000 00000000 00000000 656E6400
```

Chapter 10

API Reference Documentation

10.1 Erlang Libraries

10.1.1 estdlib

The estdlib library

Modules

Module base64

- Description
- Function Index
- Section 8.4.2

An implementation of a subset of the Erlang/OTP base64 interface.

Description

This module is designed to be API-compatible with the Erlang/OTP base64 module, with the following exceptions:

- No support for decoding data with whitespace in base64 data
- No support for mime decoding functions

Function Index

Function Details

decode/1

`Data`: the data to decode

returns: the base-64 data decoded, as a binary

Base-64 decode a binary or string, outputting a binary.

This function will raise a badarg exception if the supplied data is not valid base64-encoded data.

decode_to_string/1

`Data`: the data to decode

returns: the base-64 data decoded, as a string

Base-64 decode a binary or string, outputting a string.

This function will raise a badarg exception if the supplied data is not valid base64-encoded data.

encode/1

`Data`: the data to encode

returns: the base-64 data encoded, as a binary

Base-64 encode a binary or string, outputting a binary.

encode_to_string/1

`Data`: the data to encode

returns: the base-64 data encoded, as a string

Base-64 encode a binary or string, outputting a string.

Module calendar

- Data Types
- Function Index
- Section 8.4.2

Data Types

day()

day_of_week()

gregorian_days()

month()

year()

Function Index

Function Details

date_to_gregorian_days/1

date_to_gregorian_days/3

day_of_the_week/1

day_of_the_week/3

Module erlang

- Description
- Function Index
- Section 8.4.2

An implementation of the Erlang/OTP erlang module, for functions that are not already defined as NIFs.

Function Index

Function Details

apply/3

```
apply(Module, Function, Args) -> any()
```

is_map/1

`Map`: the map to test

returns: `true` if `Map` is a map; `false`, otherwise.

Return `true` if `Map` is a map; `false`, otherwise.

This function may be used in a guard expression.

map_get/2

`Key`: the key to get`Map`: the map from which to get the value

returns: the value in `Map` associated with `Key`, if it exists.

throws `{badkey, Key} | {badmap, Map}`

Get the value in `Map` associated with `Key`, if it exists.

This function throws a `{badkey, Key}` exception if 'Key' does not occur in `Map` or a `{badmap, Map}` if `Map` is not a map.

This function may be used in a guard expression.

map_is_key/2

`Key`: the key`Map`: the map

returns: `true` if `Key` is associated with a value in `Map`; `false`, otherwise.

throws `{badmap, Map}`

Return `true` if `Key` is associated with a value in `Map`; `false`, otherwise.

This function throws a `{badmap, Map}` exception if `Map` is not a map.

This function may be used in a guard expression.

map_size/1

`Map`: the map

returns: the size of the map

throws `{badmap, Map}`

Returns the size of (i.e., the number of entries in) the map

This function throws a `{badmap, Map}` exception if `Map` is not a map.

This function may be used in a guard expression.

max/2

```
max(A, B) -> any()
```

Return the minimum value of two terms

Terms are compared using > and follow the ordering principles defined in https://www.erlang.org/-doc/reference_manual/expressions.html#term-comparisons

md5/1

`Data`: data to compute hash of, as a binary.

returns: the md5 hash of the input Data, as a 16-byte binary.

Computes the MD5 hash of an input binary, as defined by https://www.ietf.org/rfc/rfc1321.txt

min/2

```
min(A, B) -> any()
```

Return the minimum value of two terms

Terms are compared using < and follow the ordering principles defined in https://www.erlang.org/-doc/reference_manual/expressions.html#term-comparisons

process_info/2

`Pid`: the process pid.`Key`: key used to find process information.

returns: process information for the specified pid defined by the specified key.

Return process information.

This function returns information about the specified process. The type of information returned is

dependent on the specified key.

The following keys are supported:

- **heap_size** the number of words used in the heap (integer)
- **stack_size** the number of words used in the stack (integer)
- **message_queue_len** the number of messages enqueued for the process (integer)
- **memory** the estimated total number of bytes in use by the process (integer)

Specifying an unsupported term or atom raises a bad_arg error.

send_after/3

`Time`: time in milliseconds after which to send the message.`Dest`: Pid or server name to which to send the message.`Msg`: Message to send to Dest after Time ms.

returns: a reference that can be used to cancel the timer, if desired.

Send Msg to Dest after Time ms.

start_timer/3

`Time`: time in milliseconds after which to send the timeout message.`Dest`: Pid or server name to which to send the timeout message.`Msg`: Message to send to Dest after Time ms.

returns: a reference that can be used to cancel the timer, if desired.

Start a timer, and send {timeout, TimerRef, Msg} to Dest after Time ms, where TimerRef is the reference returned from this function.

system_flag/2

`Key`: key used to change system flag.`Value`: value to change

returns: previous value of the flag.

Update system flags.

This function allows to modify system flags at runtime.

The following key is supported on SMP builds:

- **schedulers_online** the number of schedulers online

Specifying an unsupported atom key will result in a bad_arg error. Specifying a term that is not an atom will result in a bad_arg error.

system_info/1

`Key`: key used to find system information.

returns: system information defined by the specified key.

Return system information.

This function returns information about the system on which AtomVM is running. The type of information returned is dependent on the specified key.

The following keys are supported on all platforms:

- **process_count** the number of processes running in the node (integer)
- **port_count** the number of ports running in the node (integer)
- **atom_count** the number of atoms currently allocated (integer)
- **system_architecture** the processor and OS architecture (binary)
- **version** the version of the AtomVM executable image (binary)
- **wordsize** the number of bytes in a machine word on the current platform (integer)
- **schedulers** the number of schedulers, equal to the number of online processors (integer)

- **schedulers_online** the current number of schedulers (integer)

The following keys are supported on the ESP32 platform:

- **esp32_free_heap_size** the number of (noncontiguous) free bytes in the ESP32 heap (integer)
- **esp_largest_free_block** the number of the largest contiguous free bytes in the ESP32 heap (integer)
- **esp_get_minimum_free_size** the smallest number of free bytes in the ESP32 heap since boot (integer)

Additional keys may be supported on some platforms that are not documented here.

Specifying an unsupported atom key will results in returning the atom 'undefined'.

Specifying a term that is not an atom will result in a bad_arg error.

Module gen_event

- Function Index
- Section 8.4.2

Function Index

Function Details

add_handler/3

```
add_handler(EventMgrRef, Handler, Args) -> any()
```

delete_handler/3

```
delete_handler(EventMgrRef, Handler, Args) -> any()
```

notify/2

```
notify(EventMgrRef, Event) -> any()
```

start/0

```
start() -> any()
```

start/2

```
start(EventMgrName, Options) -> any()
```

start_link/0

```
start_link() -> any()
```

start_link/2

```
start_link(EventMgrName, Options) -> any()
```

stop/1

```
stop(EventManagerRef) -> any()
```

sync_notify/2

```
sync_notify(EventMgrRef, Event) -> any()
```

Module gen_server

- Description
- Data Types
- Function Index
- Section 8.4.2

An implementation of the Erlang/OTP gen_server interface.

Description

This module implements a strict susbset of the Erlang/OTP gen_server interface, supporting operations for local creation and management of gen_server instances.

This module is designed to be API-compatible with gen_server, with exceptions noted below.

Caveats:

- Support only for locally named procs
- No support for abcast
- No support for enter_loop
- No support for format_status
- No support for multi_call

Data Types

from()

options()

server_ref()

Function Index

Function Details

call/2

Equivalent to `call(ServerRef, Request, 5000)`.

Send a request to a gen_server instance, and wait for a reply.

call/3

`ServerRef`: a reference to the gen_server acquired via start`Request`: the request to send to the gen_server

returns: the reply sent back from the gen_server; {error, Reason}, otherwise.

Send a request to a gen_server instance, and wait for a reply.

This function will send the specified request to the specified gen_server instance, and wait at least Timeout milliseconds for a reply from the gen_server.

cast/2

`ServerRef`: a reference to the gen_server acquired via start`Request`: the request to send to the gen_server

returns: ok | {error, Reason}

Send a request to a gen_server instance.

This function will send the specified request to the specified gen_server instance, but will not wait for a reply.

init_it/4

```
init_it(Starter, Module, Args, Options) -> any()
```

init_it/5

```
init_it(Starter, Name, Module, Args, Options) -> any()
```

reply/2

`Reply`: the reply to send to the client

returns: an arbitrary term, that should be ignored

Send a reply to a calling client.

This function will send the specified reply back to the specified gen_server client (e.g, via call/3). The return value of this function can be safely ignored.

start/3

`Module`: the module in which the gen_server callbacks are defined`Args`: the arguments to pass to the module's init callback`Options`: the options used to create the gen_server

returns: the gen_server pid, if successful; {error, Reason}, otherwise.

Start an un-named gen_server.

This function will start a gen_server instance.

Note. *The Options argument is currently ignored.*

start/4

`ServerName`: the name with which to register the gen_server`Module`: the module in which the gen_server callbacks are defined`Args`: the arguments to pass to the module's init callback`Options`: the options used to create the gen_server

returns: the gen_server pid, if successful; {error, Reason}, otherwise.

Start a named gen_server.

This function will start a gen_server instance and register the newly created process with the process registry. Subsequent calls may use the gen_server name, in lieu of the process id.

Note. *The Options argument is currently ignored.*

start_link/3

`Module`: the module in which the gen_server callbacks are defined`Args`: the arguments to pass to the module's init callback`Options`: the options used to create the gen_server

returns: the gen_server pid, if successful; {error, Reason}, otherwise.

Start and link an un-named gen_server.

This function will start a gen_server instance.

Note. *The Options argument is currently ignored.*

start_link/4

`ServerName`: the name with which to register the gen_server`Module`: the module in which the gen_server callbacks are defined`Args`: the arguments to pass to the module's init callback`Options`: the options used to create the gen_server

returns: the gen_server pid, if successful; {error, Reason}, otherwise.

Start and link a named gen_server.

This function will start a gen_server instance and register the newly created process with the process registry. Subsequent calls may use the gen_server name, in lieu of the process id.

Note. *The Options argument is currently ignored.*

stop/1

Equivalent to `stop(ServerRef, normal, infinity)`.

Stop a previously started gen_server instance.

stop/3

`ServerRef`: a reference to the gen_server acquired via start

returns: ok, if the gen_server stopped; {error, Reason}, otherwise.

Stop a previously started gen_server instance.

This function will stop a gen_server instance, providing the supplied Reason to the gen_server's

terminate/2 callback function. If the gen_server is named, then the gen_server name may be used to stop the gen_server.

Module gen_statem

- Description
- Data Types
- Function Index
- Section 8.4.2

An implementation of the Erlang/OTP gen_statem interface.

Description

This module implements a strict susbset of the Erlang/OTP gen_statem interface, supporting operations for local creation and management of gen_statem instances.

This module is designed to be API-compatible with gen_statem, with exceptions noted below.

Caveats:

- No support for start_link
- Support only for locally named gen_statem instances
- Support only for state function event handlers
- No support for keeep_state or repeat_state return values from Module:StateName/3 callbacks
- No support for postpone or hibernate state transition actions
- No support for state enter calls
- No support for multi_call

Data Types

options()

server_ref()

Function Index

Function Details

call/2

Equivalent to `call(ServerRef, Request, infinity)`.

Send a request to a gen_statem instance, and wait for a reply.

call/3

`ServerRef`: a reference to the gen_statem acquired via start`Request`: the request to send to the gen_statem`Timeout`: the amount of time in milliseconds to wait for a reply

returns: the reply sent back from the gen_statem; {error, Reason}, otherwise.

Send a request to a gen_statem instance, and wait for a reply..

This function will send the specified request to the specified gen_statem instance, and wait at least Timeout milliseconds for a reply from the gen_statem.

cast/2

```
cast(ServerRef, Request) -> any()
```

Send a request to a gen_statem instance.

This function will send the specified request to the specified gen_statem instance, but will not wait for a reply.

reply/2

`reply(Client, Reply) -> any()`

Send a reply to a calling client.

This function will send the specified reply back to the specified gen_statem client (e.g, via call/3). The return value of this function can be safely ignored.

start/3

`Module`: the module in which the gen_statem callbacks are defined`Args`: the arguments to pass to the module's init callback`Options`: the options used to create the gen_statem

returns: the gen_statem pid, if successful; {error, Reason}, otherwise.

Start an un-named gen_statem.

This function will start a gen_statem instance.

Note. *The Options argument is currently ignored.*

start/4

`ServerName`: the name with which to register the gen_statem`Module`: the module in which the gen_statem callbacks are defined`Args`: the arguments to pass to the module's init callback`Options`: the options used to create the gen_statem

returns: the gen_statem pid, if successful; {error, Reason}, otherwise.

Start a named gen_statem.

This function will start a gen_statem instance and register the newly created process with the process registry. Subsequent calls may use the gen_statem name, in lieu of the process id.

Note. *The Options argument is currently ignored.*

start_link/3

`Module`: the module in which the gen_statem callbacks are defined`Args`: the arguments to pass to the module's init callback`Options`: the options used to create the gen_statem

returns: the gen_statem pid, if successful; {error, Reason}, otherwise.

Start an un-named gen_statem.

This function will start a gen_statem instance.

This version of the start function will link the started gen_statem process to the calling process.

Note. *The Options argument is currently ignored.*

start_link/4

`ServerName`: the name with which to register the gen_statem`Module`: the module in which the gen_statem callbacks are defined`Args`: the arguments to pass to the module's init callback`Options`: the options used to create the gen_statem

returns: the gen_statem pid, if successful; {error, Reason}, otherwise.

Start a named gen_statem.

This function will start a gen_statem instance and register the newly created process with the process registry. Subsequent calls may use the gen_statem name, in lieu of the process id.

This version of the start function will link the started gen_statem process to the calling process.

Note. *The Options argument is currently ignored.*

stop/1

Equivalent to `stop(ServerRef, normal, infinity)`.

Stop a previously started gen_statem.

stop/3

`ServerRef`: a reference to the gen_statem acquired via start

returns: ok, if the gen_statem stopped; {error, Reason}, otherwise.

Stop a previously started gen_statem instance.

This function will stop a gen_statem instance, providing the supplied Reason to the . If the gen_statem is a named gen_statem, then the gen_statem name may be used to stop the gen_statem.

Module gen_tcp

- Description
- Function Index
- Section 8.4.2

A partial implementation of the Erlang/OTP gen_tcp interface.

Description

This module provides an implementation of a subset of the functionality of the Erlang/OTP gen_tcp interface. It is designed to be API-compatible with gen_tcp, with exceptions noted below.

This interface may be used to send and receive TCP packets, as either binaries or strings. Active and passive modes are supported for receiving data.

Caveats:

- Limited support for socket tuning parameters
- No support for **controlling_process/2**

Note. *Port drivers for this interface are not supportedon all AtomVM platforms.*

Function Index

Function Details

accept/1

`ListenSocket`: the listening socket.

returns: a connection-based (tcp) socket that can be used for reading and writing

Accept a connection on a listening socket.

accept/2

`ListenSocket`: the listening socket.`Timeout`: amount of time in milliseconds to wait for a connection

returns: a connection-based (tcp) socket that can be used for reading and writing

Accept a connection on a listening socket.

close/1

`Socket`: the socket to close

returns: ok.

Close the socket.

connect/3

`Address`: the address to which to connect`Port`: the port to which to connect`Options`: options for controlling the behavior of the socket (see below)

returns: {ok, Socket}} | {error, Reason}

Connect to a TCP endpoint on the specified address and port.

If successful, this function will return a Socket which can be used with the send/2 and revc/2 and recv/3 functions in this module.

The following options are supported:

- **active** Active mode (default: true)
- **buffer** Size of the receive buffer to use in active mode (default: 512)
- **binary** data is received as binaries (as opposed to lists)
- **list** data is received as lists (default)

If the socket is connected in active mode, then the calling process will receive messages of the form {tcp, Socket, Packet} when data is received on the socket. If active mode is set to false, then applications need to explicitly call one of the recv operations in order to receive data on the socket.

controlling_process/2

`controlling_process(Socket, Pid) -> any()`

Assign a controlling process to the socket. The controlling process will receive messages from the socket.

This function will return `{error, not_owner}` if the calling process is not the current controlling proces.

By default, the controlling process is the process associated with the creation of the Socket.

listen/2

`Port`: the port number on which to listen. Specify 0 to use an OS-assigned port number, which can then be retrieved via the inet:port/1 function.`Options`: A list of configuration parameters.

returns: a listening socket, which is appropriate for use in accept/1

Create a server-side listening socket.

This function is currently unimplemented

recv/2

Equivalent to `recv(Socket, Length, infinity)`.

Receive a packet over a TCP socket from a source address/port.

recv/3

`Socket`: the socket over which to receive a packet`Length`: the maximum length to read of the received packet`Timeout`: the amount of time to wait for a packet to arrive

returns: {ok, Address, Port, Packet}} | {error, Reason}

Receive a packet over a TCP socket from a source address/port.

This function is used when the socket is not created in active mode. The received packet data returned from this call, and should be of length no greater than the specified length. This function will return {error, closed} if the server gracefully terminates the server side of the connection.

This call will block until data is received or a timeout occurs.

Note. *Currently, the Timeout parameter isignored.*

send/2

`Socket`: The Socket obtained via connect/3`Packet`: the data to send

returns: ok | {error, Reason}

Send data over the specified socket to a TCP endpoint.

If successful, this function will return the atom ok; otherwise, an error with a reason.

Module gen_udp

- Description
- Data Types
- Function Index
- Section 8.4.2

An implementation of the Erlang/OTP gen_udp interface.

Description

This module provides an implementation of a subset of the functionality of the Erlang/OTP gen_udp interface. It is designed to be API-compatible with gen_udp, with exceptions noted below.

This interface may be used to send and receive UDP packets, as either binaries or strings. Active and passive modes are supported for receiving data.

Caveats:

- Currently no support for IPv6
- Currently limited support for socket tuning parameters
- Currently no support for closing sockets

Note. *Port drivers for this interface are not supportedon all AtomVM platforms.*

Data Types

packet()

proplist()

reason()

Function Index

Function Details

close/1

`Socket`: the socket to close

returns: ok, if closing the socket succeeded, or {error, Reason}, if closing the socket failed for any reason.

Close the socket.

controlling_process/2

```
controlling_process(Socket, Pid) -> any()
```

Assign a controlling process to the socket. The controlling process will receive messages from the socket.

This function will return `{error, not_owner}` if the calling process is not the current controlling proces.

By default, the controlling process is the process associated with the creation of the Socket.

open/1

Create a UDP socket. This function will instantiate a UDP socket that may be used to send or receive UDP messages.

open/2

returns: an opaque reference to the socket instance, used in subsequent commands.

throws `bad_arg`

Create a UDP socket. This function will instantiate a UDP socket that may be used to send or receive

UDP messages. This function will raise an exception with the bad_arg atom if there is no socket driver supported for the target platform.

Note. *The Params argument is currently ignored.*

recv/2

Equivalent to `recv(Socket, Length, infinity)`.

Receive a packet over a UDP socket from a source address/port.

recv/3

`Socket`: the socket over which to receive a packet`Length`: the maximum length to read of the received packet`Timeout`: the amount of time to wait for a packet to arrive

returns: {ok, Address, Port, Packet}} | {error, Reason}

Receive a packet over a UDP socket from a source address/port. The address and port of the received packet, as well as the received packet data, are returned from this call. This call will block until data is received or a timeout occurs.

Note. *Currently Length and Timeout parameters areignored.*

Note. *Currently the length of the received packetis limited to 128 bytes.*

send/4

`Socket`: the socket over which to send a packet`Address`: the target address to which to send the packet`Packet`: the packet of data to send

returns: ok | {error, Reason}

Send a packet over a UDP socket to a target address/port.

Note. *Currently only ipv4 addresses are supported.*

Module inet

- Data Types
- Function Index
- Section 8.4.2

Data Types

address()

ipv4_address()

octet()

port_number()

socket()

Function Index

Function Details

close/1

`Socket`: the socket to close

returns: ok.

Close the socket.

peername/1

`Socket`: the socket

returns: The address and port of the remote end of an established connection.

The address and port representing the "remote" end of a connection. This function should be called on a running socket instance.

port/1

`Socket`: the socket from which to obtain the port number

returns: the port number associated with the local socket

Retrieve the actual port number to which the socket is bound. This function is useful if the port assignment is done by the operating system.

sockname/1

`Socket`: the socket

returns: The address and port of the local end of an established connection.

The address and port representing the "local" end of a connection. This function should be called on a running socket instance.

Module io

- Function Index
- Section 8.4.2

Function Index

Function Details

format/1

Equivalent to format(Format, []).

format/2

`Format`: format string`Args`: format argument

returns: string

Format string and data to console. See io_lib:format/2 for information about formatting capabilities.

get_line/1

```
get_line(Prompt) -> any()
```

put_chars/1

```
put_chars(Chars) -> any()
```

Module io_lib

- Function Index
- Section 8.4.2

Function Index

Function Details

format/2

`Format`: format string`Args`: format argument

returns: string

Format string and data to a string. Approximates features of OTP io_lib:format/2, but only supports ~p and ~n format specifiers. Raises `badarg` error if the number of format specifiers does not match the length of the Args.

Module lists

- Description
- Function Index

- Section 8.4.2

An implementation of the Erlang/OTP lists interface.

Description

This module implements a strict susbset of the Erlang/OTP lists interface.

Function Index

Function Details

all/2

`Fun`: the predicate to evaluate`List`: the list over which to evaluate elements

returns: true if Fun(E) evaluates to true, for all elements in List

Evaluates to true iff Fun(E) =:= true, for all E in List

any/2

`Fun`: the predicate to evaluate`List`: the list over which to evaluate elements

returns: true if Fun(E) evaluates to true, for at least one in List

Evaluates to true iff Fun(E) =:= true, for some E in List

delete/2

`E`: the member to delete`L`: the list from which to delete the value

returns: the result of removing E from L, if it exists in L; otherwise, L.

Remove E from L

filter/2

`Pred`: the predicate to apply to elements in List`List`: list

returns: all values in L for which Pred is true.

Filter a list by a predicate, returning the list of elements for which the predicate is true.

flatten/1

`L`: the list to flatten

returns: flattened list

recursively flattens elements of L into a single list

foldl/3

`Fun`: the function to apply`List`: the list over which to fold

returns: the result of folding Fun over L

Fold over a list of terms, from left to right, applying Fun(E, Accum) to each successive element in List

foldr/3

Equivalent to `foldl(Fun, Acc0, reverse(List))`.

Fold over a list of terms, from right to left, applying Fun(E, Accum) to each successive element in List

foreach/2

`Fun`: the predicate to evaluate`List`: the list over which to evaluate elements

returns: ok

Applies given fun to each list element

join/2

`Sep`: the separator`List`: list

returns: the result of inserting Sep between every element of List.

Inserts Sep between every element of List.

keydelete/3

`K`: the key to match`I`: the position in the tuple to compare (1..tuple_size)`L`: the list from which to delete the element

returns: the result of deleting any element in L who's Ith element matches K

Delete the entry in L whose Ith element matches K.

keyfind/3

`K`: the key to match`I`: the position in the tuple to compare (1..tuple_size)`L`: the list from which to find the element

returns: the tuple in L who's Ith element matches K; the atom false, otherwise

Find the entry in L whose Ith element matches K.

keymember/3

`K`: the key to match`I`: the position in the tuple to compare (1..tuple_size)`L`: the list from which to find the element

returns: true if there is a tuple in L who's Ith element matches K; the atom false, otherwise

Returns true if a Ith element matches K.

keyreplace/4

`K`: the key to match`I`: the position in the tuple to compare (1..tuple_size)`L`: the list from which to find the element

returns: result of replacing the first element in L who's Ith element matches K.

Returns the result of replacing the first element in L who's Ith element matches K.

map/2

`Fun`: the function to apply`List`: the list over which to map

returns: the result of mapping over L

Map a list of terms, applying Fun(E)

member/2

`E`: the member to search for`L`: the list from which to get the value

returns: true if E is a member of L; false, otherwise.

Determine whether a term is a member of a list.

nth/2

`N`: the index in the list to get`L`: the list from which to get the value

returns: the value in the list at position N.

Get the value in a list at position N.

Returns the value at the specified position in the list. The bhavior of this function is undefined if N is outside of the {1..length(L)}.

reverse/1

`L`: the list to reverse

returns: the elements of L in reverse order

Reverse the elements of L.

search/2

`Pred`: the predicate to apply to elements in List`List`: search

returns: the first {value, Val}, if Pred(Val); false, otherwise.

If there is a Value in List such that Pred(Value) returns true, returns {value, Value} for the first such Value, otherwise returns false.

seq/2

```
seq(From, To) -> any()
```

Returns a sequence of integers in a specified range.

This function is equivalent to `lists:seq(From, To, 1)`.

seq/3

```
seq(From, To, Incr) -> any()
```

Returns a sequence of integers in a specified range incremented by a specified value.

sort/1

```
sort(List) -> any()
```

Returns a sorted list, using < operator to determine sort order.

sort/2

```
sort(Fun, List) -> any()
```

Returns a sorted list, using Fun(A, B) to determine sort order.

Module maps

- Description
- Data Types
- Function Index
- Section 8.4.2

A *naive* implementation of the Erlang/OTP `maps` interface.

Description

The `maps` module provides several convenience operations for interfacing with the Erlang map type, which associates (unique) keys with values.

Note that the ordering of entries in a map is implementation-defined. While many operations in this module present entries in lexical order, users should in general make no assumptions about the ordering of entries in a map.

This module implements a susbset of the Erlang/OTP `maps` interface. Some OTP functions are not implemented, and the approach favors correctness and readability over speed and performance.

Data Types

iterator()

key()

map_or_iterator()

value()

Function Index

Function Details

filter/2

`Pred`: a function used to filter entries from the map`MapOrIterator`: the map or map iterator to filter

returns: a map containing all elements in `MapOrIterator` that satisfy `Pred`

throws `{badmap, Map} | badarg`

Return a map who's entries are filtered by the supplied predicate.

This function returns a new map containing all elements from the input `MapOrIterator` that satisfy the input `Pred`.

The supplied predicate is a function from key-value inputs to a boolean value.

This function throws a `{badmap, Map}` exception if `Map` is not a map or map iterator, and a `badarg` exception if the input predicate is not a function.

find/2

`Key`: the key to find`Map`: the map in which to search

returns: `{ok, Value}` if `Key` is in `Map`; `error`, otherwise.

throws `{badmap, Map}`

Returns `{ok, Value}` if `Key` is in `Map`; `error`, otherwise.

This function throws a `{badmap, Map}` exception if `Map` is not a map.

fold/3

`Fun`: function over which to fold values`Init`: the initial value of the fold accumulator`MapOrIterator`: the map or map iterator over which to fold

returns: the result of folding over all elements of the supplied map.

throws `{badmap, Map} | badarg`

Fold over the entries in a map.

This function takes a function used to fold over all entries in a map and an initial accumulator value to use as the value supplied to the first entry in the map.

This function throws a `badmap` exception if `Map` is not a map or map iterator, and a `badarg` exception if the input function is not a function.

from_list/1

`List`: a list of `[{Key, Value}]` pairs

returns: the map containing the entries from the list of supplied key-value pairs.

throws `badarg`

This function constructs a map from the supplied list of key-value pairs.

If the input list contains duplicate keys, the returned map will contain the right-most entry.

This function will raise a `badarg` exception if the input is not a proper list or contains an element that is not a key-value pair.

get/2

`Key`: the key to get`Map`: the map from which to get the value

returns: the value in `Map` associated with `Key`, if it exists.

throws `{badkey, Key} | {badmap, Map}`

Get the value in `Map` associated with `Key`, if it exists.

This function throws a `{badkey, Key}` exception if 'Key' does not occur in `Map` or a `{badmap, Map}` if `Map` is not a map.

get/3

`Key`: the key`Map`: the map`Default`: default value

returns: the value in `Map` associated with `Key`, or `Default`, if the key is not associated with a value in

`Map`.

throws `{badmap, Map}`

Get the value in `Map` associated with `Key`, or `Default`, if the key is not associated with a value in `Map`.

This function throws a `{badmap, Map}` exception if `Map` is not a map.

is_key/2

`Key`: the key`Map`: the map

returns: `true` if `Key` is associated with a value in `Map`; `false`, otherwise.

throws `{badmap, Map}`

Return `true` if `Key` is associated with a value in `Map`; `false`, otherwise.

This function throws a `{badmap, Map}` exception if `Map` is not a map.

iterator/1

`Map`: the map

returns: an iterator structure that can be used to iterate over associations in a map.

throws `{badmap, Map}`

Return an iterator structure that can be used to iterate over associations in a map.

In general, users shouuld make no assumptions about the order in which entries appear in an iterator. The order of entries in a map is implementation-defined.

This function throws a `{badmap, Map}` exception if `Map` is not a map.

See also: `next/1`.

keys/1

`Map`: the map

returns: the list of keys that occur in this map.

throws `{badmap, Map}`

Returns the list of keys that occur in this map.

No guarantees are provided about the order of keys returned from this function.

This function throws a `{badmap, Map}` exception if `Map` is not a map.

map/2

`Fun`: the function to apply to every entry in the map`Map`: the map to which to apply the map function

returns: the result of applying `Fun` to every entry in `Map`

throws `{badmap, Map} | badarg`

Returns the result of applying a function to every element of a map.

This function throws a `badmap` exception if `Map` is not a map or map iterator, and a `badarg` exception if the input function is not a function.

merge/2

`Map1`: a map`Map2`: a mpa

returns: the result of merging entries from `Map1` and `Map2`.

throws `{badmap, Map}`

Merge two maps to yield a new map.

If `Map1` and `Map2` contain the same key, then the value from `Map2` will be used.

This function throws a `badmap` exception if neither `Map1` nor `Map2` is a map.

new/0

returns: a new map

Return a new (empty) map.

next/1

`Iterator`: a map iterator

returns: the key and value, along with the next iterator in the map, or the atom `none` if there are no more items over which to iterate.

throws `badarg`

Returns the next key and value in the map, along with a new iterator that can be used to iterate over the remainder of the map.

This function throws a `badarg` exception if the supplied iterator is not of the expected type. Only use iterators that are returned from functions in this module.

put/3

`Key`: the key`Value`: the value`Map`: the map

returns: A copy of `Map` containing the `{Key, Value}` association.

throws `{badmap, Map}`

Return the map containing the `{Key, Value}` association.

If `Key` occurs in `Map` then it will be over-written. Otherwise, the returned map will contain the new association.

This function throws a `{badmap, Map}` exception if `Map` is not a map.

remove/2

`Key`: the key to remove`MapOrIterator`: the map or map iterator from which to remove the key

returns: a new map without `Key` as an entry.

throws `{badmap, Map}`

Remove an entry from a map using a key.

If `Key` does not occur in `Map`, then the returned Map has the same entries as the input map or map iterator.

Note. This function extends the functionality of the OTP `remove/2` function, since the OTP interface only takes a map as input.

This function throws a `badmap` exception if `Map` is not a map or map iterator.

size/1

`Map`: the map

returns: the size of the map

throws `{badmap, Map}`

Returns the size of (i.e., the number of entries in) the map

This function throws a `{badmap, Map}` exception if `Map` is not a map.

to_list/1

returns: a list of `[{Key, Value}]` tuples

Return the list of entries, expressed as `{Key, Value}` pairs, in the supplied map.

No guarantees are provided about the order of entries returned from this function.

This function throws a `{badmap, Map}` exception if `Map` is not a map.

update/3

`Key`: the key to update`Value`: the value to update`Map`: the map to update

returns: a new map, with `Key` updated with `Value`

throws `{badmap, Map}`

Returns a new map with an updated key-value association.

This function throws a `badmap` exception if `Map` is not a map and `{badkey, Key}` if key doesn`t exist

values/1

`Map`: the map

returns: the list of values that occur in this map.

throws `{badmap, Map}`

Returns the list of values that occur in this map.

No guarantees are provided about the order of values returned from this function.

This function throws a `{badmap, Map}` exception if `Map` is not a map.

Module math

- Function Index
- Section 8.4.2

Function Index

Function Details

acos/1

acosh/1

asin/1

asinh/1

atan/1

atan2/2

atanh/1

ceil/1

cos/1

cosh/1

exp/1

floor/1

fmod/2

log/1

log10/1

log2/1

pi/0

pow/2

sin/1

sinh/1

sqrt/1

tan/1

tanh/1

Module proplists

- Description
- Data Types
- Function Index
- Section 8.4.2

An implementation of the Erlang/OTP proplists interface.

Description

This module implements a strict susbset of the Erlang/OTP proplists interface.

Data Types

property()

Function Index

Function Details

get_value/2

Equivalent to `get_value(Key, List, undefined)`.

Get a value from a property list.

get_value/3

`Key`: the key with which to find the value`Default`: the default value to return, if Key is not in the property list.

returns: the value in the property list under the key, or Default, if Key is not in List.

Get a value from a property list.

Returns the value under the specified key, or the specified Default, if the Key is not in the supplied List. If the Key corresponds to an entry in the property list that is just a single atom, this function returns the atom true.

Module string

- Description
- Function Index
- Section 8.4.2

An implementation of the Erlang/OTP string interface.

Description

This module implements a strict susbset of the Erlang/OTP string interface.

Function Index

Function Details

split/2

```
split(String, Pattern) -> any()
```

split/3

```
split(String, Pattern, Where) -> any()
```

to_upper/1

```
to_upper(S) -> any()
```

trim/1

```
trim(String) -> any()
```

trim/2

```
trim(String, X2) -> any()
```

Module supervisor

- Function Index
- Section 8.4.2

Function Index

Function Details

handle_call/3

```
handle_call(Msg, _from, State) -> any()
```

handle_cast/2

```
handle_cast(Msg, State) -> any()
```

handle_info/2

```
handle_info(Msg, State) -> any()
```

init/1

```
init(X1) -> any()
```

start_link/2

```
start_link(Module, Args) -> any()
```

start_link/3

```
start_link(SupName, Module, Args) -> any()
```

Module timer

- Function Index
- Section 8.4.2

Function Index

Function Details

sleep/1

10.1.2 eavmlib

The eavmlib library

Modules

Module atomvm

- Description
- Data Types

- Function Index
- Section 8.4.2

AtomVM-specific APIs.

Description

This module contains functions that are specific to the AtomVM platform.

Data Types

platform_name()

Function Index

Function Details

platform/0

returns: The platform name.

Return the platform moniker. You may use this function to uniquely identify the platform type on which your application is running.

rand_bytes/1

`Len`: non-negative integer

returns: Binary containing random sequence of bytes of length Len.

Returns a binary containing random sequence of bytes of length Len. Supplying a negative value will result in a badarg error. This function will use a cryptographically strong RNG if available. Otherwise, the random value is generated using a PRNG.

random/0

returns: random 32-bit integer.

Returns a random 32-bit integer value. This function will use a cryptographically strong RNG if available. Otherwise, the random value is generated using a PRNG.

read_priv/2

`App`: application name.`Path`: path to the resource.

returns: Binary containing the resource content.

This function allows to fetch priv/ resources content.

Module avm_pubsub

- Function Index
- Section 8.4.2

Function Index

Function Details

handle_call/3

```
handle_call(X1, From, Table) -> any()
```

handle_info/2

```
handle_info(Info, Table) -> any()
```

init/1

```
init(X1) -> any()
```

pub/3

```
pub(PubSub, Topic, Term) -> any()
```

start/0

```
start() -> any()
```

start/1

```
start(LocalName) -> any()
```

sub/2

```
sub(PubSub, Topic) -> any()
```

sub/3

```
sub(PubSub, Topic, Pid) -> any()
```

terminate/2

```
terminate(Reason, State) -> any()
```

unsub/2

```
unsub(PubSub, Topic) -> any()
```

unsub/3

```
unsub(PubSub, Topic, Pid) -> any()
```

Module console

- Description
- Function Index
- Section 8.4.2

This modules supports output of string data to the console.

Function Index

Function Details

flush/0

returns: ok if the data was written, or {error, Reason}, if there was an error.

Flush any previously written data to the console.

print/1

`String`: the string data to write to the console

returns: ok if the data was written, or {error, Reason}, if there was an error.

Write a string to the console.

See also: erlang:display/1.

puts/1

`String`: the string data to write to the console

returns: ok if the data was written, or {error, Reason}, if there was an error.

Write a string to the console.

Note. *This operation will only write string data.The output is not suffixed with a newline character or sequence.To print an erlang term, use erlang:display/1.*

See also: erlang:display/1.

Module esp

- Description

- Data Types
- Function Index
- Section 8.4.2

ESP32-specific APIs.

Description

This module contains functions that are specific to the ESP32 platform.

Data Types

esp_reset_reason()

esp_wakeup_cause()

Function Index

Function Details

freq_hz/0

returns: Clock frequency (in hz)

Return the clock frequency on the chip

nvs_erase_all/0

Equivalent to nvs_erase_all(?ATOMVM_NVS_NS).

nvs_erase_all/1

`Namespace`: NVS namespace

returns: ok

Erase all values in the specified namespace.

nvs_erase_key/1

Equivalent to nvs_erase_key(?ATOMVM_NVS_NS, Key).

nvs_erase_key/2

`Namespace`: NVS namespace`Key`: NVS key

returns: ok

Erase the value associated with a key. If a value does not exist for the specified key, no action is performed.

nvs_get_binary/1

Equivalent to nvs_get_binary(?ATOMVM_NVS_NS, Key).

nvs_get_binary/2

`Namespace`: NVS namespace`Key`: NVS key

returns: binary value associated with this key in NV storage, or undefined if there is no value associated with this key.

Get the binary value associated with a key, or undefined, if there is no value associated with this key.

nvs_get_binary/3

`Namespace`: NVS namespace`Key`: NVS key`Default`: default binary value, if Key is not set in Namespace

returns: binary value associated with this key in NV storage, or Default if there is no value associated with this key.

Get the binary value associated with a key, or Default, if there is no value associated with this key.

nvs_reformat/0

returns: ok

Reformat the entire NVS partition. WARNING. This will result in deleting all NVS data and should be used with extreme caution!

nvs_set_binary/2

Equivalent to nvs_set_binary(?ATOMVM_NVS_NS, Key, Value).

nvs_set_binary/3

`Namespace`: NVS namespace`Key`: NVS key`Value`: binary value

returns: ok

Set an binary value associated with a key. If a value exists for the specified key, it is over-written.

reset_reason/0

returns: the reason for the restart

Returns the reason for the restart

restart/0

Restarts the ESP device

sleep_enable_ext0_wakeup/2

returns: Configure ext0 wakeup

sleep_enable_ext1_wakeup/2

returns: Configure ext1 wakeup

wakeup_cause/0

returns: the cause for the wake up

Returns the cause for the wakeup

Module gpio

- Description
- Data Types
- Function Index
- Section 8.4.2

GPIO driver module.

Description

This module provides functions for interacting with micro-controller GPIO (General Purpose Input and Output) pins.

Data Types

direction()

gpio()

high_level()

level()

low_level()

pin()

pull()

trigger()

Function Index

Function Details

attach_interrupt/2

`Trigger`: is the state that will trigger an interrupt

returns: ok | error

Convenience function for `gpio:set_int/3`

This is a convenience function for `gpio:set_int/3` that allows an interrupt to be set using only the pin number and trigger as arguments.

This function should only be used when only one gpio trigger is used in an application. If multiple pins are being configured with interrupt triggers gpio:set_int/3 should be used otherwise there is a race condition when start() is called internally by this function.

close/1

`GPIO`: pid that was returned from gpio:start/0

returns: ok atom

Stop the GPIO interrupt port

This function disables any interrupts that are set, stops the listening port, and frees all of its resources.

deep_sleep_hold_dis/0

returns: ok

Disable all gpio pad functions during Deep-sleep.

deep_sleep_hold_en/0

returns: ok

Enable all hold functions to continue in deep sleep.

The gpio pad hold function works in both input and output modes, but must be output-capable gpios.

When the chip is in Deep-sleep mode, all digital gpio will hold the state before sleep, and when the chip is woken up, the status of digital gpio will not be held. Note that the pad hold feature only works when the chip is in Deep-sleep mode, when not in sleep mode, the digital gpio state can be changed even you have called this function.

Power down or call `gpio_hold_dis` will disable this function, otherwise, the digital gpio hold feature works as long as the chip enters Deep-sleep.

detach_interrupt/1

returns: ok | error

Convenience function for `gpio:remove_int/2`

This is a convenience function for `gpio:remove_int/2` that allows an interrupt to be removed using only the pin number as an argument.

Unlike `gpio:attach_interrupt/2` this function can be safely used regardless of the number of interrupt pins used in the application.

digital_read/1

returns: high | low

Read the digital state of a GPIO pin

Read if an input pin state is high or low. Warning: if a is not previously configured as an input using `gpio:set_pin_mode/2` it will always read as low.

digital_write/2

returns: ok | error

Set GPIO digital output level

Set a pin to `high` (1) or `low` (0).

hold_dis/1

returns: ok | error

Release a pin from a hold state.

When the chip is woken up from Deep-sleep, the gpio will be set to the default mode, so, the gpio will output the default level if this function is called. If you don't want the level changes, the gpio should be configured to a known state before this function is called. e.g. If you hold gpio18 high during Deep-sleep, after the chip is woken up and `gpio:hold_dis` is called, gpio18 will output low level(because gpio18 is input mode by default). If you don't want this behavior, you should configure gpio18 as output mode and set it to hight level before calling `gpio:hold_dis`.

hold_en/1

returns: ok | error

Hold the state of a pin

The gpio pad hold function works in both input and output modes, but must be output-capable gpios.

If pad hold enabled: In output mode: the output level of the pad will be force locked and can not be changed. In input mode: the input value read will not change, regardless the changes of input signal.

The state of digital gpio cannot be held during Deep-sleep, and it will resume the hold function when the chip wakes up from Deep-sleep. If the digital gpio also needs to be held during Deep-sleep `gpio:deep_sleep_hold_en` should also be called.

open/0

returns: Pid

Start the GPIO driver port

The GPIO port driver will be stared and registered as `gpio`. If the port has already been started through the `gpio:open/0` or `gpio:start/0` the command will fail. The use of `gpio:open/0` or `gpio:start/0` is required before using any functions that require a GPIO pid as a parameter.

read/2

`GPIO`: pid that was returned from gpio:start/0

returns: high | low

Read the digital state of a GPIO pin

Read if an input pin state is `high` or `low`. Warning: if a is not previously configured as an input using `gpio:set_direction/3` it will always read as low.

remove_int/2

`GPIO`: pid that was returned from `gpio:start/0`

returns: ok | error

Remove a GPIO interrupt

Removes an interrupt from the specified pin.

set_direction/3

`GPIO`: pid that was returned from `gpio:start/0Direction`: is `input`, `output`, or `output_od`

returns: ok | error

Set the operational mode of a pin

Pins can be used for input, output, or output with open drain.

set_int/3

`GPIO`: pid that was returned from `gpio:start/0Trigger`: is the state that will trigger an interrupt

returns: ok | error

Set a GPIO interrupt

Available triggers are `none` (which is the same as disabling an interrupt), `rising`, `falling`, `both` (rising or falling), `low`, and `high`. When the interrupt is triggered it will send a tuple: `{gpio_interrupt, Pin}` to the process that set the interrupt. Pin will be the number of the pin that triggered the interrupt.

set_level/3

`GPIO`: pid that was returned from `gpio:start/0`

returns: ok | error

Set GPIO digital output level

Set a pin to `high` (1) or `low` (0).

set_pin_mode/2

`Direction`: is `input`, `output`, or `output_od`

returns: ok | error

Set the operational mode of a pin

Pins can be used for input, output, or output with open drain.

set_pin_pull/2

`Pull`: is the internal resistor state

returns: ok | error

Set the internal resistor of a pin

Pins can be internally pulled `up`, `down`, `up_down` (pulled in both directions), or left `floating`.

start/0

returns: Pid

Start the GPIO driver port

Returns the pid of the active GPIO port driver, otherwise the GPIO port driver will be stared and registered as `gpio`. The use of `gpio:open/0` or `gpio:start/0` is required before using any functions that require a GPIO pid as a parameter.

stop/0

returns: ok atom

Stop the GPIO interrupt port

This function disables any interrupts that are set, stops the listening port, and frees all of its resources.

Module http_server

- Function Index
- Section 8.4.2

Function Index

Function Details

parse_query_string/1

```
parse_query_string(L) -> any()
```

reply/3

```
reply(StatusCode, Reply, Conn) -> any()
```

start_server/2

```
start_server(Port, Router) -> any()
```

Module i2c

- Description
- Data Types
- Function Index
- Section 8.4.2

AtomvVM I2c interface.

Description

This module provides and interface into the AtomVM I2C driver.

Use this module to communicate with devices connected to your ESP32 device via the 2-wire I2C interface.

Using this interface, you can read or write data to an I2C device at a given I2C address. In addition, you may read from or write to specific registers on the I2C device.

Data Types

address()

freq_hz()

i2c()

param()

params()

pin()

register()

Function Index

Function Details

begin_transmission/2

`Address`: I2C Address of the device (typically fixed for the device type)

returns: `ok` or `error`

Begin a transimission of I2C commands

This command is typically followed by one or more calls to `write_byte/2` and then a call to `end_transmission/1`

close/1

`I2C`: I2C instance created via `open/1`

returns: `ok` atom

Closes the connection to the I2C driver

This function will close the connection to the I2C driver and free any resources in use by it.

end_transmission/1

returns: `ok` or `error`

End a transimission of I2C commands

This command is typically preceded by a call to `begin_transmission/2` and one or more calls to `write_byte/2`.

open/1

`Param`: Initialization parameters

returns: process id of the driver.

Open a connection to the I2C driver

This function will open a connection to the I2C driver.

read_bytes/3

`Address`: I2C Address of the device (typically fixed for the device type)`Count`: The number of bytes to read

returns: the read binary data or `error`

Read a block of bytes from the I2C device.

This command is not wrapped in a `begin_transmission/2` and `end_transmission/1` call.

read_bytes/4

`Address`: I2C Address of the device (typically fixed for the device type)`Register`: The register address in the device from which to read data`Count`: The number of bytes to read

returns: the read binary data or `error`

Read a block of bytes from the I2C device starting at a specified register address

This command is not wrapped in a `begin_transmission/2` and `end_transmission/1` call.

write_byte/2

`Byte`: value to write

returns: `ok` or `error`

Write a byte to the device.

This command must be wrapped in a `begin_transmission/2` and `end_transmission/1` call.

write_bytes/2

returns: `ok` or `error`

Write a sequence of bytes to the device.

This command must be wrapped in a `begin_transmission/2` and `end_transmission/1` call.

write_bytes/3

`Address`: I2C Address of the device (typically fixed for the device type)`BinOrInt`: The binary or byte value to write

returns: `ok` or `error`

Write a block of bytes to the I2C device.

This command is not wrapped in a `begin_transmission/2` and `end_transmission/1` call.

write_bytes/4

`Address`: I2C Address of the device (typically fixed for the device type)`Register`: The register address in the device to which to write data`BinOrInt`: The binary or byte value to write

returns: `ok` or `error`

Write a block of bytes to the I2C device starting at a specified register address.

This command is not wrapped in a `begin_transmission/2` and `end_transmission/1` call.

Module json_encoder

- Function Index
- Section 8.4.2

Function Index

Function Details

encode/1

```
encode(Value) -> any()
```

Module ledc

- Description
- Data Types
- Function Index
- Section 8.4.2

LED Controller low-level APIs.

Description

The functions in this module broadly reflect the ESP IDF-SDK LED Controller API.

See the IDF-SDK LEDC documentation for more information about these APIs.

Data Types

channel()
channel_cfg()
channel_config()
duty()
duty_cfg()
duty_resolution()
duty_resolution_cfg()
fade_mode()
freq_hz()
freq_hz_cfg()
gpio_num()
gpio_num_cfg()
hpoint()
hpoint_cfg()
ledc_error_code()
speed_mode()
speed_mode_cfg()
timer_config()
timer_num()
timer_num_cfg()
timer_sel()
timer_sel_cfg()

Function Index

Function Details

channel_config/1

`Config`: timer configuration

returns: ok | {error, ledc_error_code()}

LEDC timer configuration.

Configure LEDC timer with the given source timer/frequency(Hz)/duty_resolution.

fade_func_install/1

`Flags`: Flags used to allocate the interrupt. One or multiple (ORred) ESP_INTR_FLAG_* values. See esp_intr_alloc.h for more info.

returns: ok | {error, ledc_error_code()}

Install LEDC fade function.

This function will occupy interrupt of LEDC module.

fade_func_uninstall/0

returns: ok

Uninstall LEDC fade function.

fade_start/3

`SpeedMode`: Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.`Channel`: LEDC channel index (0-7).`FadeMode`: Whether to block until fading done.

returns: ok | {error, ledc_error_code()}

Start LEDC fading.

Note. Call ledc:fade_func_install() once before calling this function. Call ledc:fade_start() after this to start fading.

get_duty/2

`SpeedMode`: Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.`Channel`: LEDC channel index (0-7).

returns: ok | {error, ledc_error_code()}

LEDC get duty.

get_freq/2

`SpeedMode`: Select the LEDC channel group with specified speed mode.`TimerNum`: LEDC timer index (0-3).

returns: ok | {error, ledc_error_code()}

LEDC get channel frequency (Hz)

set_duty/3

`SpeedMode`: Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.`Channel`: LEDC channel index (0-7).`Duty`: Set the LEDC duty, the range of duty setting is [0, (2**duty_resolution)]

returns: ok | {error, ledc_error_code()}

LEDC set duty.

set_fade_with_step/5

`SpeedMode`: Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.`Channel`: LEDC channel index (0-7).`TargetDuty`: Target duty of fading.(0..(2^duty_resolution-1)))`Scale`: Controls the increase or decrease step scale.`CycleNum`: increase or decrease the duty every cycle_num cycles

returns: ok | {error, ledc_error_code()}

Set LEDC fade function

Note. Call ledc:fade_func_install() once before calling this function. Call ledc:fade_start() after this to start fading.

set_fade_with_time/4

`SpeedMode`: Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.`Channel`: LEDC channel index (0-7).`TargetDuty`: Target duty of fading.(0..(2^duty_resolution-1)))`MaxFadeTimeMs`: The maximum time of the fading (ms).

returns: ok | {error, ledc_error_code()}

Set LEDC fade function, with a limited time.

Note. Call ledc:fade_func_install() once before calling this function. Call ledc:fade_start() after this to start fading.

set_freq/3

`SpeedMode`: Select the LEDC channel group with specified speed mode.`TimerNum`: LEDC timer index (0-3).

returns: ok | {error, ledc_error_code()}

LEDC get channel frequency (Hz)

stop/3

`SpeedMode`: Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.`Channel`: LEDC channel index (0-7).`IdleLevel`: Set output idle level after LEDC stops.

returns: ok | {error, ledc_error_code()}

LEDC stop. Disable LEDC output, and set idle level.

timer_config/1

`Config`: timer configuration

returns: ok | {error, Reason}

LEDC timer configuration.

Configure LEDC timer with the given source timer/frequency(Hz)/duty_resolution.

update_duty/2

`SpeedMode`: Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.`Channel`: LEDC channel index (0-7).

returns: ok | {error, ledc_error_code()}

LEDC update channel parameters.

Module logger

- Description
- Data Types
- Function Index
- Section 8.4.2

A simple logging module.

Description

This module can be used to log messages to the console.

Most applications should use the logging macros defined in logger.hrl to do logging.

Data Types

config()

config_item()

level()

location()
sink()

Function Index

Function Details

get_filter/0

returns: the current filter set in the logger

Get the current filter in the logger.

get_levels/0

returns: the current list of levels set in the logger

Get the current levels in the logger.

get_sinks/0

returns: the current list of sinks set in the logger

Get the current sinks in the logger.

log/3

`Location`: the location in the source module

Log a message the the specified location with the specified level.

Users should use the logging macros in logger.hrl instead of calling this function directly.

set_filter/1

`Filter`: the filter to set

returns: ok

Set the filter in the logger. Currently, a filter is a set of module names. If the filter is non-empty, then a log will be triggered only if the module being logged is in the filter list; otherwise, if the filter is empty, then the log will be triggered (if the level matches, as well)

set_levels/1

`Levels`: the list of levels to set

returns: ok

Set the levels in the logger.

set_sinks/1

`Sinks`: the list of sinks to set

returns: ok

Set the sinks in the logger.

start/0

Equivalent to `start([{levels, [info, warning, error]}]).`

Start the logger.

start/1

`Config`: the logging configuration

returns: {ok, Pid}

Start the logger.

This function will start the logger with the specified configuration.

stop/0

Stop the logger.

This function is rarely used.

Module network

- Data Types
- Function Index
- Section 8.4.2

Data Types

ap_config()
ap_config_property()
ap_max_connections_config()
ap_ssid_hidden_config()
ap_sta_connected_config()
ap_sta_disconnected_config()
ap_sta_ip_assigned_config()
ap_started_config()
dhcp_hostname_config()
ip_info()
ipv4_address()
ipv4_info()
mac()
network_config()
octet()
psk_config()
sntp_config()
sntp_config_property()
ssid_config()
sta_config()
sta_config_property()
sta_connected_config()
sta_disconnected_config()
sta_got_ip_config()

Function Index

Function Details

start/1

`Config`: The network configuration

returns: ok, if the network_fsm was started, or {error, Reason} if a failure occurred (e.g., due to malformed network configuration).

Start a network_fsm.

This function will start a network_fsm, which will attempt to connect to an AP endpoint in the background. Specify callback functions to receive definitive information that the connection succeeded. See the AtomVM Network FSM Programming Manual for more information.

start_link/1

stop/0

returns: ok, if the network_fsm was stopped, or {error, Reason} if a failure occurred.

Stop a network_fsm.

wait_for_ap/0

Equivalent to wait_for_ap(15000).

wait_for_ap/1

Equivalent to wait_for_ap([], Timeout) or wait_for_ap(StaConfig, 15000).

wait_for_ap/2

`ApConfig`: The AP network configuration`Timeout`: amount of time in milliseconds to wait for a connection

returns: ok, when the nework_fsm has started the AP, or {error, Reason} if a failure occurred (e.g., due to malformed network configuration).

Start a network_fsm in access point mode and wait the AP to be up and running

This function will start a network_fsm in AP mode, and will wait until the network is up and ready to be connected. This is a convenience function, for applications that do not need to be notified of connectivity changes in the network.

wait_for_sta/0

Equivalent to wait_for_sta(15000).

wait_for_sta/1

Equivalent to wait_for_sta([], Timeout) or wait_for_sta(StaConfig, 15000).

wait_for_sta/2

`StaConfig`: The STA network configuration`Timeout`: amount of time in milliseconds to wait for a connection

returns: {ok, IpInfo}, if the network_fsm was started, or {error, Reason} if a failure occurred (e.g., due to malformed network configuration).

Start a network_fsm in station mode and wait for a connection to be established

This function will start a network_fsm in station mode, and will wait for a connection to be established. This is a convenience function, for applications that do not need to be notified of connectivity changes in the network.

Module network_fsm

- Function Index
- Section 8.4.2

Function Index

Function Details

start/1

```
start(Config) -> any()
```

stop/0

```
stop() -> any()
```

wait_for_ap/0

```
wait_for_ap() -> any()
```

wait_for_ap/1

```
wait_for_ap(ApConfig) -> any()
```

wait_for_ap/2

```
wait_for_ap(ApConfig, Timeout) -> any()
```

wait_for_sta/0

```
wait_for_sta() -> any()
```

wait_for_sta/1

```
wait_for_sta(StaConfig) -> any()
```

wait_for_sta/2

```
wait_for_sta(StaConfig, Timeout) -> any()
```

Module port

- Function Index
- Section 8.4.2

Function Index

Function Details

call/2

call/3

Module spi

- Description
- Data Types
- Function Index
- Section 8.4.2

This module provides an interface into the Serial Peripheral Interface (SPI) supported on many devices.

Description

This module currently support the SPI "leader" (historically known as the "master") interface, allowing the leader to connect to one or more "follower" (historically known as "slave") devices.

Users interact with this interface by creating an instance of the driver via the `open/1` function, with returns an opaque reference to the driver instance. The `open/1` function takes a complex map structure, which configures the driver to connect to follower devices. See the `open/1` documentation for details about the structure of this configuration map.

Subsequent read and write operations use the SPI instance returned from the `open/1` function. Users may read from a specific follower device at a specific address, write to the device at an address, or simultaneously read from and write to the device in a single transaction.

Data Types

address()

bus_config()

device_config()

device_name()

params()

spi()

spi_peripheral()

transaction()

Function Index

Function Details

close/1

`SPI`: SPI instance created via `open/1`

Close the SPI driver.

Close the SPI driver and free any resources in use by the driver.

The SPI instance will no longer be valid and usable after this function has been called.

open/1

`Params`: Initialization parameters

returns: process id of the driver.

throws `badarg`

Open a connection to the SPI driver

This function will open a connection to the SPI driver.

Supply a set of parameters to initialize the driver.

The parameters list must contain an SPI Bus configuration, together with a properties list containing one or more device configurations. This list must contain atom keys as names, which are used to identify the device in the subsequent read and write operations. You may use any atom value of your choosing.

The SPI Bus configuration is a properties list containing the following entries:

Each device configuration is a properties list containing the following entries:

Example:

```
Params = [
   {bus_config, [
       {miso_io_num, 16},
       {mosi_io_num, 17},
       {sclk_io_num, 5}
   },
   {device_config, [
       {device1, [
           {spi_cs_io_num, 18}
       ]},
       {device2, [
           {spi_cs_io_num, 19}
       ]}
   ]}
]
```

Note that `device1` and `device2` are atom names used to identify the device for read and write operations.

This function raises an Erlang exception with a `badarg` reason, if initialization of the SPI Bus or any device fails.

The `write/3` and `write_read/3` functions in this module are designed to provide the maximum mount of flexibility when interfacing with the SPI device. The both make use of a map structure to encapsulate an SPI transaction.

An SPI transaction may contain a command, and address, and/or a blob of data, each of which is optional and each of which depends on how users interact with the device. Consult the data sheet for your SPI device to understand which fields should be used with your device.

The fields of a transaction map are as follows:

read_at/4

`SPI`: SPI instance created via `open/1``Address`: SPI Address from which to read

returns: `{ok, Value}` or `error`

Read a value from and address on the device.

write/3

`SPI`: SPI instance created via `open/1``DeviceName`: SPI device name (use key in `device_config`)`Transaction`: transaction map.

returns: `ok` or `{error, Reason}`, if an error occurred.

Write data to the SPI device, using the intructions encoded in the supplied transaction.

The supplied `Transaction` encodes information about how data is to be written to the selected SPI device. See the description above for the fields that may be specified in this map.

When a binary is supplied in the `write_data` field, the data is written to the SPI device in the natural order of the binary. For example, if the input binary is `<<16#57, 16#BA>>`, then the first byte is `0x57` and the second byte is `0xBA`.

The value of the `write_bits` field, if specified, must be less than or equal to `8 * byte_size(write_data)`. If `write_bits` is less than `8 * byte_size(write_data)`, only the first `write_bits` bits from `write_data` will be written.

This function will return a tuple containing the `error` atom if an error occurred writing to the SPI device at the specified address. The returned reason term is implementation-defined.

write_at/5

`SPI`: SPI instance created via `open/1``Address`: SPI Address to which to write

returns: `{ok, Value}` or `error`

Write a value to and address on the device.

The value returned from this function is dependent on the device and address. Consult the documentation for the device to understand expected return values from this function.

write_read/3

`SPI`: SPI instance created via `open/1``DeviceName`: SPI device name (use key in `device_config`)`Transaction`: transaction.

returns: `{ok, binary()}` or `{error, Reason}`, if an error occurred.

Write data to the SPI device, using the intructions encoded in the supplied transaction. device, and simultaneously read data back from the device, returning the read data in a binary.

The supplied `Transaction` encodes information about how data is to be written to the selected SPI device. See the description above for the fields that may be specified in this map.

When a binary is supplied in the `write_data` field, the data is written to the SPI device in the natural order of the binary. For example, if the input binary is `<<16#57, 16#BA>>`, then the first byte is `0x57` and the second byte is `0xBA`.

The value of the `write_bits` field, if specified, must be less than or equal to `8 * byte_size(write_data)`. If `write_bits` is less than `8 * byte_size(write_data)`, only the first `write_bits` bits from `write_data` will be written.

The return value contains a sequence of bytes that have been read from the SPI device. The number of bytes returned will be `ceil(read_bits / 8)`. Only the first `read_bits` will be populated.

This function will return a tuple containing the `error` atom if an error occurred writing to the SPI device at the specified address. The returned reason term is implementation-defined.

Module timestamp_util

- Description
- Data Types
- Function Index
- Section 8.4.2

Utility functions for comparing timestamps.

Description

This module contains functions that are useful for comparing timestamps without running the risk of integer overflow.

Note that the functions in this module may be obsoleted in future versions of AtomVM, as support for arbitrary sized integers is added; however, the functions may still be useful in their own right.

Data Types

megasecs()

microsecs()

secs()

timestamp()

Function Index

Function Details

delta/2

`TS2`: a timestamp`TS1`: a timestamp

returns: TS2 - TS1, as a timestamp

Computes the difference between TS2 and TS1, as a timestamp.

delta_ms/2

`TS2`: a timestamp`TS1`: a timestamp

returns: TS2 - TS1, in milliseconds

Computes the difference between TS2 and TS1, in milliseconds.

Module uart

- Function Index
- Section 8.4.2

Function Index

Function Details

close/1

```
close(Pid) -> any()
```

open/2

```
open(Name, Opts) -> any()
```

read/1

```
read(Pid) -> any()
```

write/2

```
write(Pid, B) -> any()
```

10.1.3 alisp

The alisp library

Modules

Module alisp

- Function Index
- Section 8.4.2

Function Index

Function Details

booleanize/1

```
booleanize(V) -> any()
```

eval/1

```
eval(S) -> any()
```

run/1

```
run(S) -> any()
```

Module alisp_stdlib

- Function Index
- Section 8.4.2

Function Index

Function Details

''/1*

```
*(L) -> any()
```

'+'/1

```
+(L) -> any()
```

'-'/1

```
-(T) -> any()
```

'='/1

```
=(T) -> any()
```

'remove-if'/1

```
remove-if(X1) -> any()
```

'remove-if-not'/1

```
remove-if-not(X1) -> any()
```

append/1

```
append(X1) -> any()
```

binaryp/1

```
binaryp(X1) -> any()
```

car/1

```
car(X1) -> any()
```

cdr/1

```
cdr(X1) -> any()
```

cons/1

```
cons(X1) -> any()
```

floatp/1

```
floatp(X1) -> any()
```

identity/1

```
identity(X1) -> any()
```

integerp/1

```
integerp(X1) -> any()
```

last/1

```
last(X1) -> any()
```

list/1

```
list(List) -> any()
```

listp/1

```
listp(X1) -> any()
```

mapcar/1

```
mapcar(X1) -> any()
```

numberp/1

```
numberp(X1) -> any()
```

pidp/1

```
pidp(X1) -> any()
```

print/1

```
print(X1) -> any()
```

refp/1

```
refp(X1) -> any()
```

tuple/1

```
tuple(List) -> any()
```

tuplep/1

```
tuplep(X1) -> any()
```

Module arepl

- Function Index
- Section 8.4.2

Function Index

Function Details

start/0

```
start() -> any()
```

Module sexp_lexer

- Function Index
- Section 8.4.2

Function Index

Function Details

string/1

```
string(Bin) -> any()
```

Module sexp_parser

- Function Index
- Section 8.4.2

Function Index

Function Details

parse/1

```
parse(T) -> any()
```

Module sexp_serializer

- Function Index
- Section 8.4.2

Function Index

Function Details

serialize/1

```
serialize(Value) -> any()
```

10.1.4 etest

The etest library

Modules

Module etest

- Description
- Function Index
- Section 8.4.2

This modules provides a basic testing framework for AtomVM Erlang libraries.

Function Index

Function Details

assert_equals/2

`X`: a term`Y`: a term

returns: ok if X and Y are equal; fail otherwise.

assert_failure/1

`F`: a function to evaluate

returns: ok if evaluating F results in Error being thrown; fail, otherwise

assert_failure/2

`F`: a function to evaluate

returns: ok if evaluating F results in Error being thrown; fail, otherwise

assert_match/2

`X`: a term

returns: ok if X and Y unify; fail otherwise.

assert_true/1

returns: ok if X is true; fail otherwise.

test/1

`Tests`: a list of test modules

returns: ok if all of the tests pass, or the atom fail, if any of the tests failed.

Test a sequence of test modules.

This function will execute the test/0 function for each module provided in the input list of test modules. If all of the tests return the atom ok, then this function returns ok. If any of the test modules return a value other than ok, then this function returns the atom fail.

10.2 AtomVM 'C' Internal Libraries

10.2.1 libAtomVM

[libAtomVM (Doxygen theme)]

10.2.2 libAtomVM Source Files

[libAtomVM (Doxygen theme)]

Chapter 11

Contributing Guidelines

Make sure to understand the license and the contribution guidelines before contributing and last but not least be kind.

11.1 Git Recommended Practices

- Commit messages should have a summary and a description
- Avoid trailing white spaces
- Always `git pull --rebase`
- Clean up your branch history with `git rebase -i`
- All your intermediate commits should build

11.2 Coding Style

11.2.1 C Code

Indentation

- K&R indentation and braces style
- Mandatory braces
- 4 spaces indentation

Good:

```
void f(int reverse)
{
    if (reverse) {
        puts("!dlroW olleH");
    } else {
        puts("Hello world");
    }
}
```

Bad:

```
void f(int reverse) {
    if (reverse)
        puts ("!dlroW olleH");
    else
```

```
        puts ("Hello world");
}
```

Names

- Struct names are PascalCase (e.g. Context)
- Scalar types are lower case (e.g. term)
- All other names (e.g. functions and variables) are snake_case (e.g. term_is_integer)
- Always prefix function names (e.g. term_is_nil, term_is_integer, context_new, context_destroy)

Other Coding Conventions

- Pointer * should be with the variable name rather than with the type (e.g. `char *name`, not `char* name`)
- Avoid long lines, use intermediate variables with meaningful names.

11.2.2 Elixir Code

Just use Elixir formatter enforced style.

- genindex
- modindex
- search